

UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

LAUREA MAGISTRALE in
INFORMATICA

Optimization of Logistic Operations in Decentralized Manual Warehouses

Supervisor:

Prof. Alberto Ceselli

Candidate:

Abdelrahman Sobh

Mat. Nr.: 30061A

Milano, A.Y. 2025/2026

Abdelrahman Sobh

Optimization of Logistic Operations in Decentralized Manual Warehouses

Master's thesis, 13 july 2026

Supervisor:

Prof. Alberto Ceselli

Faculty of Science and Technology

Department of Computer Science “Giovanni degli Antoni” Via Celoria 18 - 20133 Milan

Acknowledgements

I would like to express my sincere gratitude to my supervisor, prof. Alberto Ceselli, for his guidance and support throughout this research. I am also grateful to all my professors for their dedication to teaching and the knowledge they shared.

I would also like to thank Beta 80 SPA for their collaboration and for providing the real-world datasets essential for this business-based study.

Contents

Acknowledgements	i
Contents	ii
List of Figures	vi
1 Introduction	1
2 Problem Definition and Mathematical Formulation	5
2.1 Problem Definition	5
System Entities	6
Operational Constraints and Dynamics	6
Graph Conceptualization	7
Edge Classes and Temporal Dynamics	8
System Objective	8
Problem Classification within Combinatorial Optimization	9
2.2 Mathematical Formulation	10
Sets and Indices	10
Parameters	10
Decision Variables	11
Objective Function	11
The Trade-off between Makespan and Operator Activation	12
Constraints	12
2.3 Additional Problem Formulation: Real-World Constraints	15
Operational Characteristics and Skill Compatibility	15
3 Mixed-Integer Programming Model	17
3.1 Optimality: Justification for Mixed-Integer Programming (MIP)	17
Solution Methodology: Branch-and-Bound Algorithm	18

3.2	Computational Implementation	19
	Solver Configurations and Hyperparameters	21
3.3	Primary Solving Approach	22
3.4	Limitations of the Exact MIP Solver in Practical Business Scenarios	23
3.5	The Dual Role of the MIP Solver	25
	Ground Truth for Imitation Learning (Training Data Generation)	25
	Exact Baseline for Evaluation	26
4	Graph Neural Networks and the One-Shot Architecture	27
4.1	Imitation Learning: The Scope of the Neural Solver	27
4.2	Introduction to Graph Neural Networks (GNNs)	29
	Core GNN Architectures	29
	Overview of Message Passing Framework (MPNN)	30
	Neighborhood Determination and Scalable Message Passing	32
4.3	Message Passing Bi-Directionality in Warehouse Heterogeneous Graphs	34
	Justification for GATv2 in Warehouse Routing	35
4.4	Multi-Layer Perceptrons (MLP) in Graph Neural Networks	36
4.5	Proposed Architecture: One-Shot GNN and Coupled Multi-Task MLP Heads	38
	1. Operator Activation Head	39
	2. Order Assignment Head	40
	3. Temporal Sequence Head	40
	Addressing Gradient Domination and Multi-Task MLP Conflicts via Explicit Logical Coupling	41
4.6	Multi-Objective Loss Function and Constraint Penalties	42
	Limitations and Transition to Recurrent Modeling	45
5	Graph-to-Sequence Neural Network Architecture	47
5.1	Proposed Architecture: Node-Aware Graph2Seq with Coupled Multi-Task MLP Heads	47
	Operator State Update via GRU	49
5.2	The Static Spatial Encoder	51
5.3	The Dynamic Recurrent Decoder	51
	Operator State Update via GRU	51
	Global Fleet Context Integration	52
5.4	Iterative Multi-Task Prediction Heads	52
	1. Activation Head	52

2. Assignment Head	53
3. Sequence Head	54
Dynamic Masking of Completed Actions	54
5.5 Autoregressive Training via Teacher Forcing and BPTT	55
Step-wise Loss Evaluation and Target Masking	56
6 Inference and Scheduling Methods	58
6.1 One-Shot GNN Inference and Neural Decoding	59
One-Shot GNN Model and Greedy Decoder	59
6.2 Autoregressive Graph2Seq Inference	62
Iterative Greedy Sequence Decoder with Non-Overlapped Chunking	63
6.3 Algorithmic Baselines for Large-Scale Benchmarking	67
The Computational Limits of Exact Solvers	67
The Standalone Heuristic Greedy Solver	67
7 Key Performance Indicators and Lower Bound Calculation for Benchmarking	70
7.1 Key Performance Indicators (KPIs)	70
Makespan (C_{max})	71
Total Flow Time (TFT)	71
Activation Score	71
Weighted Combination Score	71
Coefficient of Variation (CV)	72
7.2 Theoretical Lower Bound (LB) Calculation	72
7.3 Benchmarking Protocol and Gap Analysis	74
Mini-Batch Evaluation: MIP Optimality Imitation	74
Large-Scale Evaluation: Heuristic Dominance	74
Generalization Proof via Lower Bound Gaps	75
Detailed Per-Horizon Evaluation	75
8 Experimental Setup and Training Strategies	77
8.1 Dataset Structure and Cleaning	77
Heterogeneous Graph Generation	78
8.2 Ground Truth and Mini-Batch Generation	79
Isomorphic Spatial Augmentation	80
8.3 Learning Rates and Optimization Strategy	82
8.4 Model Quantitative Evaluation Metrics	84

Precision, Recall, and the F1-Score	85
Metric Selection for Task-Specific Heads	85
8.5 Hyperparameter Tuning	86
Hyperparameters to Be Tuned	89
Should We Tune the Number of Training Iterations (Epochs)?	91
Tuning Task-Specific Decision Thresholds	92
8.6 Training Curves and Monitoring	93
Model-Based Training and Validation Curves Analysis	94
Overall Model Performance	94
Activation Head Performance	95
Assignment Head Performance	97
Sequence Head Performance	98
Summary of the Pipeline	101
9 Mini-Batch Experiments and Qualitative Analysis	102
9.1 Mini-Batch Validation Results Analysis: Efficiency vs. Fairness	104
Primary Metrics Evaluation	104
Weighted Combination Score	106
Efficiency and Fairness: TFT and CV Scores	107
Pareto Trade-off: Observations on the Relationship between TFT and CV	110
9.2 Optimality Gap Analysis	113
Primary Objective Optimality Gaps	114
Efficiency and Fairness Optimality Gaps	114
Model Comparison and Selection	115
9.3 Recurrent Model: Per-Horizon Optimality Gaps Analysis	116
Per-Horizon Primary Objective KPIs	116
Per-Horizon Efficiency and Fairness KPIs	117
Interpretation of Time Horizon Effects	118
9.4 Recurrent Model: Lower Bound Gaps Analysis	119
Primary Objective Lower Bound Gaps	120
Efficiency and Fairness Lower Bound Gaps	122
10 Full Problem Experiments and Qualitative Analysis	123
10.1 Recurrent Model: Test Set Lower Bound Gaps Analysis	124
10.2 Generalization via Validation-Test Lower Bound Gaps	125
Validation-Test Lower Bound Gaps on Primary Objective KPIs	126
Validation-Test Lower Bound Gaps on Efficiency and Fairness KPIs	127

10.3	Recurrent Model: Per-Horizon Lower Bound Gaps Analysis (Full Batch)	128
10.4	Heuristic Greedy Baseline Comparison	129
	Recurrent Model to Greedy Baseline Score Curves	130
	Primary Objective Baseline Gaps	133
	Efficiency and Fairness Baseline Gaps	134
11	Discussion	136
11.1	The Superiority of Recurrent Decoding at Scale	136
	Inference Speed and Real-World Reliability	137
11.2	The Weighted Combination Score as the Ultimate Business Metric	137
12	Computational Environment and Used Technologies	139
12.1	Computational Environment	139
13	Conclusion and Future Work	141
13.1	Open Challenges and Future Directions	141
	Architectural Refinements and Advanced Learning Paradigms	141
	Adapting to Changing Business Perspectives	144
	Fault Tolerance and Dynamic Rescheduling	144
	Bibliography	145

List of Figures

1.1	Euler diagram for P, NP, NP-complete, and NP-hard set of problems.	2
2.1	System Graph Representation.	7
3.1	Simplified Illustration of the B&B Tree.	19
4.1	GNN: Single Node Message Aggregation from Its Local Neighborhood	31
4.2	GNN: The Process of Updating Node Embedding through Multiple Layers. [16].	32
4.3	Graph Representation for Message Passing.	35

4.4	Example of a Multi-Layer Perceptron (MLP) Architecture with Two Hidden Layers.	37
4.5	One-Shot Architecture of the GNN-MLP Model.	39
4.6	ReLU and Softplus Curves.	44
5.1	Recurrent Architecture of the Graph2Seq Model.	48
5.2	Gated Recurrent Units (GRU) Architecture.	49
5.3	Representation of the Dynamic Masking Mechanism across Consecutive Decoding Steps.	54
5.4	Unrolled Representation of a GRU over Time to Illustrate BPTT.	56
5.5	Teacher Forcing Pipeline.	57
6.1	One-Shot GNN Model and Greedy Decoding Procedure	60
6.2	Graph2Seq Model and Iterative Greedy Sequence Decoding Procedure	65
8.1	Precision-recall curve with an example optimal decision threshold.	93
8.2	Overall Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	95
8.3	Activation Head Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	96
8.4	Assignment Head Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	97
8.5	Sequence Head Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	99
8.6	Area under Precision-Recall Curve (PRC) for different model heads.	100
9.1	Makespan & Activation Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	105
9.2	Weighted Combination Score Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	106
9.3	TFT Score Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	108
9.4	CV Score Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	109
9.5	Primary Objective Optimality Gaps Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	113
9.6	Efficiency to Fairness Optimality Gaps Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.	115

9.7	Recurrent Model: Per-Horizon Optimality Gaps on Primary Objective KPIs.	116
9.8	Recurrent Model: Per-Horizon Optimality Gaps on Efficiency and Fairness KPIs.	117
9.9	Primary Objective Lower-Bound KPIs (Recurrent Model)	120
9.10	Lower Bound Gaps on Primary Objective KPIs	120
9.11	Trade-off: Makespan and Activation Lower Bound Gap Curves	121
9.12	Lower Bound Gaps on Efficiency and Fairness KPIs	122
10.1	Test Set Lower Bound Gaps on Primary Objective KPIs	124
10.2	Test Set Lower Bound Gaps on Efficiency and Fairness KPIs	125
10.3	Validation vs. Test Lower Bound Gaps (Primary Objective KPIs)	126
10.4	Validation vs. Test Lower Bound Gaps (Efficiency and Fairness KPIs)	127
10.5	Per-Horizon LB Gaps on Primary Objective KPIs	128
10.6	Per-Horizon LB Gaps on Efficiency and Fairness KPIs	129
10.7	Makespan and Activation Scores: Recurrent Model vs. Greedy Baseline	131
10.8	Weighted Combination Score: Recurrent Model vs. Greedy Baseline	131
10.9	TFT and CV Curves: Recurrent Model vs. Greedy Baseline	132
10.10	Recurrent Model vs. Greedy Baseline Gaps (Primary Objective KPIs)	133
10.11	Recurrent Model vs. Greedy Baseline Gaps (Efficiency and Fairness)	134
11.1	Weighted Combination Score Comparison: Lower Bound, Recurrent Model, and Heuristic Greedy Algorithm	138
13.1	Warehouse Map	143

1.

Introduction

The efficient allocation of resources to tasks under operational constraints is a fundamental challenge that pervades a wide range of real-world domains, from logistics and manufacturing to healthcare and emergency response. In many such domains, the assignment problem is further complicated by spatial dependencies in which resources must physically travel between locations, and the time cost of these movements directly influences the feasibility, efficiency, productivity and the overall quality of any proposed schedule. When formalized rigorously, these problems belong to the class of **combinatorial optimization problems** (COPs), many of which are NP-hard (fig.1.1) and therefore intractable by exhaustive search even for moderate instance sizes.

NP-hard problem: A problem H is scientifically defined as NP-hard if every problem L in the complexity class NP is polynomial-time reducible to H. [2]

Combinatorial optimization deals with problems with an objective function that has to be minimized or maximized over a set of combinatorial alternatives. The most naive way of obtaining the optimal solution is to list all the feasible solutions, evaluating them using the objective function and selecting the optimal one. Nevertheless, a brute-force approach lacks practicality when the size of the problem is too large, as the time needed to solve it grows exponentially with the problem size. [3]

Classical optimization methods, most notably **mixed-integer programming** (MIP), have long served as the gold standard for solving such problems to provable optimality. MIP solvers such as CPLEX and Gurobi exploit the mathematical structure of the problem to prune the search space efficiently, yielding exact or bounded-approximate solutions. However, their computational cost grows prohibitively with the size and complexity of the instance; since they use the **branch and bound** (B&B) algorithm,

rendering them impractical in time-critical or large-scale deployment scenarios for real-world where decisions must be made rapidly and repeatedly. [4]

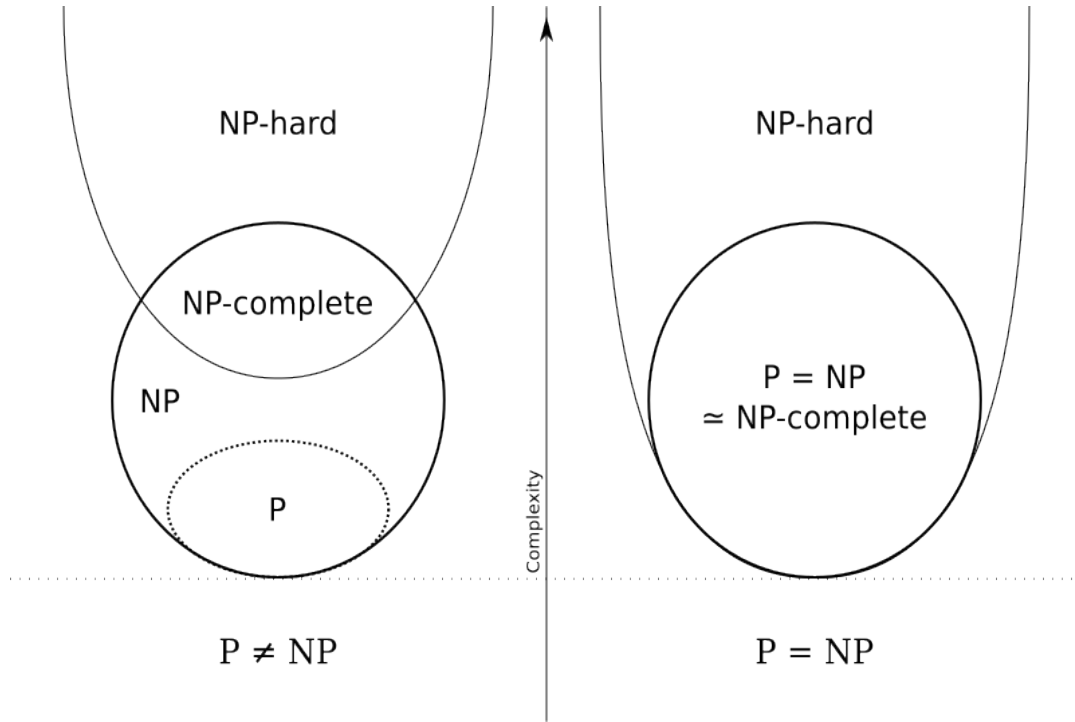


Figure 1.1: Euler diagram for P, NP, NP-complete, and NP-hard set of problems.

Recently, the intersection of machine learning and combinatorial optimization has opened a promising alternative approach called "**neural combinatorial optimization(NCO)**" [3, 5]. Rather than solving each problem instance from scratch, learned solvers attempt to extract generalizable patterns from historical instances and exploit this knowledge to guide or replace the search process at inference time. Among the architectures that have proven a particular effectiveness for this purpose are **graph neural networks** (GNNs), which operate natively on graph-structured data and are naturally suited to encode the relational structure of scheduling and routing problems.

Neural combinatorial optimization attempts to learn good heuristics for solving a set of problems using neural network models. Recently, its good performance has encouraged many practitioners to develop neural architectures for a wide variety of combinatorial problems. However, the incorporation of such algorithms in the conventional optimization framework has raised many questions related to their (1) performance (2) training data transferability (3) computational cost (4) generalization to large size instances (5) the experimental comparison with other methods such as exact algorithms, heuristics

and metaheuristics.

The field of neural combinatorial optimization has emerged as a major research direction at the intersection of machine learning and mathematical optimization. Rather than treating each instance as an independent problem to be solved from scratch, neural combinatorial optimization seeks to exploit data-driven regularities in order to accelerate decision making, improve heuristics, or directly construct high-quality solutions. This field encompasses several complementary subareas, including end-to-end learned solution construction, machine learning guidance for exact optimization methods, graph-based representation learning for structured decision problems, and hybrid approaches that integrate neural models with classical optimization algorithms.

The present thesis is situated primarily in this third and fourth line of research, as it investigates the use of GNNs to learn assignment and scheduling policies by imitating the behaviour of MIP-based solvers for a structured scheduling problem while preserving a comparison with exact MIP-based optimization.

This thesis investigates the application of GNNs to a real-world operator-mission assignment problem characterized by spatial travel times, processing constraints, and planning horizon limits. We propose a neural solver based on graph attention networks v2 (GATv2) and connected multi-layer perceptron (MLP) heads, trained to approximate high-quality predictions. The training can be integrated with a separate post-processing procedure (decoder) that picks and checks the produced predictions in order to build the final solution; otherwise, an end-to-end (E2E) training can solve the decoding problem by fusing the decoding step into the model, so the final solution is automatically produced from initial raw data. At inference time, solutions are constructed through a greedy scheduling procedure that explicitly accounts for operation's required time, operator availability, and real-world physical constraints.

The central hypothesis driving this work is that a GNN-based solver can achieve high-quality solutions having competitive scores compared to MIP-based solutions, as measured by optimality gaps across key performance indicators (KPI) such as makespan, resource activation rate, total flow time (TFT), and coefficient of variation (CV), while operating at a fraction of the computational cost. To validate this claim, we conduct a systematic experimental evaluation on both small-scale mini-batch instances and large-scale full-size problem sets, comparing the proposed method against the MIP baseline and theoretical lower bounds.

The **remainder** of our thesis is organized as follows:

- Part 1 (Modeling): formalizes the scheduling problem as a graph model and derives its algebraic mathematical programming formulation.
- Part 2 (Algorithms & Metrics): describes the exact MIP model, the GNN architectures, the inference methodology, the lower bound computation strategy, and the KPI definitions used throughout the evaluation.
- Part 3 (Experiments): presents the experimental results on real-world datasets, beginning with mini-batch experiments and scaling up to the full problem, with quantitative metrics and comparative qualitative analyses.

2. Problem Definition and Mathematical Formulation

In this chapter, we provide a formal definition of the combinatorial optimization scheduling problem in the context of manual warehouses at various scales of logistic operations. We model this type of COP as a graph problem incorporating entities (operators, missions, depots), and real-world constraints. Furthermore, We formally define the spatially constrained operator-mission assignment problem and translates its operational dynamics into an algebraic MIP model.

2.1 Problem Definition

In manual warehouses, a scale-variant batch of logistic operations (picking, unpicking) need to be geographically distributed on a fleet of a limited number of forklift operators without exceeding their shift. The primary goal is to find an optimal schedule that minimizes the overall *makespan* of these operations, minimizing also *the total number of operators* required to fulfill these operations (called "missions" or "orders"). Furthermore, the workload distribution need also to be balanced for all activated operators. Once the batch is processed and the optimized schedule is built, each operator is informed of his missions by the start of the next work day.

The problem extends classical bipartite matching by introducing spatiotemporal routing dynamics, meaning the feasibility and cost of an assignment depend heavily on the

sequential physical movement of the operators.

Makespan (C_{max}): The makespan, defined as $\max(C_1, \dots, C_n)$, is equivalent to the completion time of the last job to leave the system. A minimum makespan usually implies a good utilization of the machine(s). [6]

Where C_j is the completion time of mission j and n is the total number of missions that need to be scheduled.

System Entities

The operational environment consists of three primary entities: a **central depot**, a set of **missions**, and a pool of available **operators**. The central depot, conceptually designated as node 0, serves as the universal starting location for all available resources at the beginning of the scheduling period. Missions represent localized tasks that require a specific duration of active processing time to be completed. Operators are the mobile agents responsible for executing these missions, moving through the environment and processing tasks sequentially until the operational time horizon is reached.

Operational Constraints and Dynamics

The temporal progression of the system is governed by a combination of travel times and processing times. When an operator is assigned their first mission, they must incur an initial travel time penalty representing the distance from the base depot to the task location. For all subsequent assignments, the travel time is calculated based on the distance between the operator's last completed mission and their newly assigned target. Furthermore, all assignments are strictly bound by a planning horizon constraint, meaning an operator cannot be assigned a mission if the combined travel and processing time exceeds the remaining available time in the current shift.

Graph Conceptualization

The spatially constrained operator-mission assignment problem can be formally represented as an optimization problem on a **heterogeneous** directed multigraph [fig.2.1], denoted as $G=(V, E)$, where V is its vertex set and E represents its edges. The heterogeneity of the graph arises from the presence of two functionally distinct sets of vertices (nodes) and two distinct classes of edges, representing different physical and temporal properties of the system. [7]

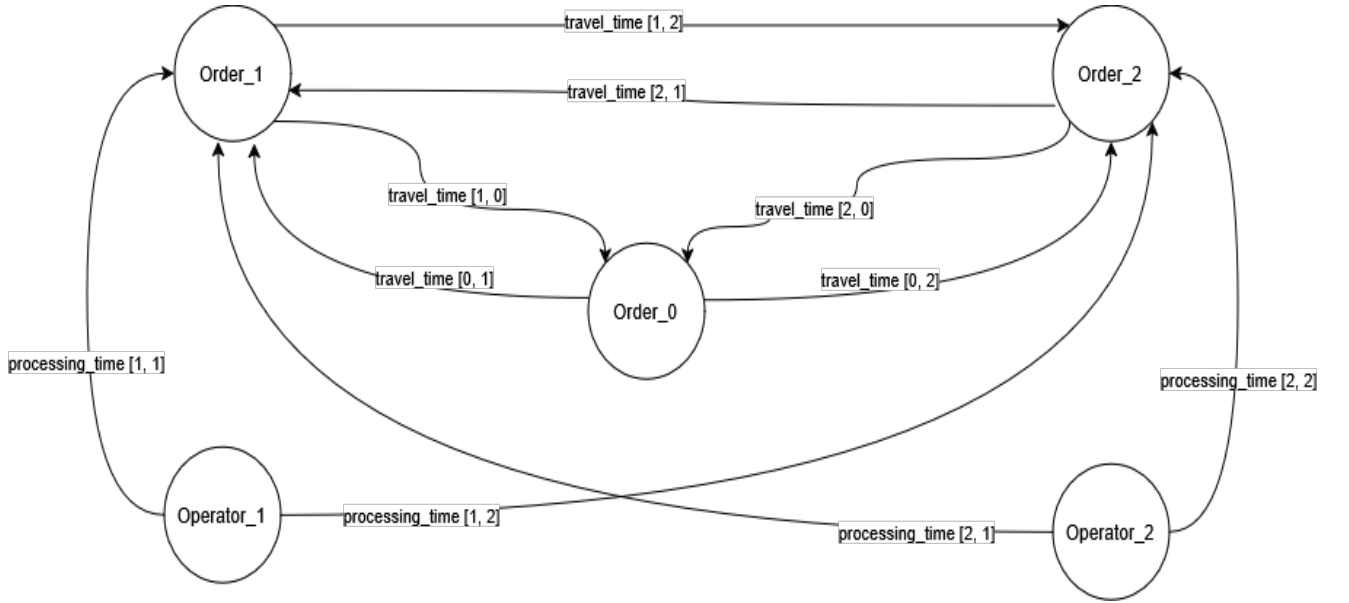


Figure 2.1: System Graph Representation.

The vertex set V is partitioned into two disjoint subsets, $V = I_{max} \cup J'$, representing the physical entities in the system:

Operator Nodes (I_{max}): The set of mobile agents available to be activated to execute tasks (e.g., $Operator_1$, $Operator_2$, ...). An individual operator is denoted as $i \in I_{max}$.

Order Nodes (J'): The set of geographically distributed tasks (e.g., $Order_1$, $Order_2$, ...) that require processing with spatial features. This set explicitly includes a specialized *depot node* (represented as $Order_0$), which serves as the physical origin point for all operators at the beginning of the scheduling horizon. An individual assignable order is denoted as $j \in J$, where $J = J' - \{Order_0\}$.

Edge Classes and Temporal Dynamics

The edge set E is similarly partitioned into two functional categories, $E = E_T \cup E_P$, which encode the spatial and processing constraints of the scheduling environment:

Travel Time Edges (E_T): These directed edges exclusively connect the nodes within the order subset J , such that $E_T \subseteq J \times J$. An edge $(j, j') \in E_T$ represents the spatial routing cost—specifically the time required to travel from the location of order j to order j' . Crucially, the graph includes travel edges originating from the depot node Order_0 to the other orders (e.g., $\text{travel_time}[0, 1]$), representing the initial dispatch time penalty incurred when an operator i is assigned their first task. Meanwhile, travel edges directed to the depot node Order_0 from other orders (e.g., $\text{travel_time}[1, 0]$) indicates the return-to-base constraint for an operator i .

Processing Time Edges (E_P): These directed edges connect operator nodes to order nodes, such that $E_P \subseteq I_{\max} \times J$. An edge $(i, j) \in E_P$ encodes the specific duration required for operator i to complete order j (e.g., $\text{processing_time}[1, 1]$).

While travel times in the current graph are assumed to be symmetric ($E_{Tj,j'} = E_{Tj',j}$), the problem is modeled on a directed graph (digraph). This explicit representation via dual arcs allows for the future integration of asymmetric routing constraints without any loss of generality.

System Objective

The optimization objective is to determine an optimal schedule—represented as a sequence of traversed edges—that maps the operators in $I_{\max}$ to the orders in J . A valid schedule must ensure that for any given operator $i \in I_{\max}$, the total time consumed does not exceed the maximum allowable operational horizon limit, denoted as H_{fixed} .

Specifically, the sum of the traversed travel time edges (E_T) and processing time edges (E_P) for operator i must satisfy:

$$\sum_{(j,j') \in E_{T,i}} \text{travel_time}(j, j') + \sum_{(i,j) \in E_{P,i}} \text{processing_time}(i, j) \leq H_{\text{fixed}}$$

The goal is to construct this schedule such that it minimizes both operator activation rate and global temporal metrics.

Problem Classification within Combinatorial Optimization

In the taxonomy of classical operations research, the problem addressed in this thesis can be formally classified as a multi-objective vehicle routing and scheduling problem with route duration limits and compatibility constraints. While the fundamental spatial dynamics are rooted in the well-known capacitated vehicle routing problem (CVRP). [8]

The operational requirements of the system introduce several layers of complexity that extend significantly beyond standard routing formulations:

Joint routing and scheduling: Standard routing problems typically assume that reaching a node satisfies the demand. In contrast, this problem entails both spatial traversal and discrete task execution. Therefore, the model must explicitly compute continuous temporal variables (start and completion times) to ensure sequence feasibility, placing it in the domain of scheduling.

Route duration constraints: Rather than bounding the physical cargo capacity of a vehicle, the system strictly bounds the maximum allowable shift duration for each operator. This aligns the formulation with resource-constrained or distance-constrained VRP variants, where the total temporal length of a route dictates its feasibility.

Multi-objective min-max optimization: Traditional VRPs primarily seek to minimize the aggregate sum of travel distances or fleet size. The proposed system navigates a multi-objective space, balancing total labor cost (activated operators) with temporal efficiency. Crucially, the inclusion of the makespan as a primary objective transforms the problem into a min-max formulation, which inherently demands optimal workload

balancing across the active fleet and is computationally more severe than standard sum-cost objectives.

2.2 Mathematical Formulation

The aforementioned operational logic needs to be transcribed into a deterministic algebraic representation, valid to build a MIP model to establish an exact objective-based evaluation for the subsequent neural solvers.

Sets and Indices

The system relies on the following primary sets to define the operational entities:

- J : The set of all available orders/missions. This set is expanded to $J' = J \cup \{0\}$, where node 0 represents the initial base depot.
- I_{max} : The total pool of available operators.

Parameters

The physical environment and operational limitations are modeled using the following parameters:

- P_{ij} : Processing time required for operator i to complete order j .
- T_{jk} : Travel time required to move from the location of order j to order k .
- H_{fixed} : Maximum allowed shift length (working minutes) per operator.
- M : A sufficiently large positive number (Big-M), used to linearize conditional constraints and substitute missing or undefined values.

Decision Variables

The model utilizes three primary types of decision variables to determine assignments, sequencing, and temporal state:

- Assignment and activation variables:
 - $x_{ij} \in \{0, 1\}$: binary variable equal to 1 if order j is assigned to operator i , and 0 otherwise.
 - $y_i \in \{0, 1\}$: binary variable equal to 1 if operator i is activated (assigned at least one task), and 0 otherwise.
- Routing variable:
 - $z_{ijk} \in \{0, 1\}$: binary variable equal to 1 if operator i travels directly from order j to order k , and 0 otherwise.
- Continuous temporal variables:
 - $S_j \geq 0, C_j \geq 0$: Start and completion times of order j .
 - $S_{i,\text{first}} \geq 0, C_{i,\text{last}} \geq 0$: Start time of the first task and completion time of the last task for operator i .
 - $Z \geq 0$: The global makespan (the total time required to complete all tasks).
 - $u_{ij} \geq 0$: Subtour elimination helper variable representing the sequence position of order j on the route of operator i .

Note that all time units should be standardized (shift time, operator speed that affect the processing time, ...). Also any distances are measured in "meter", and speeds are measured in meter/minute.

Objective Function

The problem is formulated as a multi-criteria optimization to balance time efficiency and labor costs. The objective minimizes a weighted sum of the global makespan and the total number of activated operators.

Hence, the multi-criteria objective function can be formalized as:

$$\text{Minimize } \alpha \cdot Z + \beta \cdot \sum_{i \in I_{max}} y_i \quad (2.1)$$

Where α and β are weighting factors defining the trade-off cost between minimizing execution time and minimizing staff utilization.

The Trade-off between Makespan and Operator Activation

The objective function is parameterized by α and β , which represent the relative weights assigned to competing goals: a higher α prioritizes the minimization of latency (execution time), whereas a higher β prioritizes resource conservation (staff utilization).

With more active operators: Missions are split across many routes, each route is shorter, so the overall makespan decreases. But $\sum y_i$ increases (more operators used).

With fewer active operators: Missions are concentrated on few routes, and these routes get close to the shift capacity H_{fixed} , so makespan increases. But $\sum y_i$ decreases (lower labor / resource cost).

Because of this obvious trade-off in multi-criteria optimization, the produced optimal schedule is defined *Pareto-optimal* schedule.

Pareto-optimal schedule: A schedule is called Pareto-optimal if it is not possible to decrease the value of one objective without increasing the value of the other.^[6]

Constraints

- Assignment: Each order j must be assigned to exactly one operator:

$$\sum_{i \in I_{max}} x_{ij} = 1 \quad \forall j \in J \quad (2.2)$$

- Routing and flow conservation:
 - The linking between order k and order j over the same operator i , iff operator i is assigned to these orders.

$$\sum_{k \in J', k \neq j} z_{ijk} = x_{ij} \quad \forall i \in I_{\max}, j \in J \quad (2.3)$$

- The number of flow-outs ($j \rightarrow k$) should be equal to the number flow-ins ($l \rightarrow j$) of the same operator i for any order j , including routes to the base node.

$$\sum_{k \in J', k \neq j} z_{ijk} = \sum_{l \in J', l \neq j} z_{ilj} \quad \forall i \in I_{\max}, j \in J \quad (2.4)$$

- Temporal sequencing: The start time of an order k must respect the completion time of the previous order j plus the travel time between them (using Big-M notation):

$$S_k \geq C_j + T_{jk} - M(1 - z_{ijk}) \quad \forall i \in I_{\max}, \forall j, k \in J, j \neq k \quad (2.5)$$

The completion time of an order is its start time plus the processing time required:

$$C_j = S_j + \sum_{i \in I_{\max}} (x_{ij} \cdot P_{ij}) \quad \forall j \in J \quad (2.6)$$

- Subtour elimination (MTZ): Miller-Tucker-Zemlin constraints ensure strict ordering and eliminate disconnected routing loops [9]:
 - Ordering rule

$$u_{ik} - u_{ij} + |J|z_{ijk} \leq |J| - 1 \quad \forall i \in I_{\max}, \forall j, k \in J \quad (2.7)$$

It forces strictly increasing order indices along used arcs, thereby eliminating subtours. Suppose that operator i goes from mission j to mission k , the rank of k must be at least 1 greater than the rank of j . This implies an effect to only move "forward" in the sequence along a used arc.

- Assignment rule

$$u_{ij} \leq |J|x_{ij} \quad \forall i \in I_{\max}, j \in J \quad (2.8)$$

It prevents "ghost" ordering values on missions that an operator does not actually visit, which keeps the MTZ logic correct and the relaxation tighter. Forcing u_{ij} to 0 when $x_{ij} = 0$.

- Base start rule

$$u_{ij} \geq z_{i0j} \quad \forall i \in I_{\max}, j \in J \quad (2.9)$$

It ensures that the first mission after the base has a positive rank, anchoring the MTZ ordering scale and helping rule out degenerate assignments where a mission appears to be visited but has rank 0.

- Activation and horizon limits:

- An order j can only be assigned to operator i iff the operator i is activated ($y_i = 1$).

$$x_{ij} \leq y_i \quad \forall i \in I_{\max}, j \in J \quad (2.10)$$

- The total number of trips that end at the base ($k=0$) must be equal to the number of activated operators ($y_i = 1$).

$$\sum_{j \in J} z_{ij0} = y_i \quad \forall i \in I_{\max} \quad (2.11)$$

- Return to base and shift completion

The correct representation implies that each operator i should return to the base depot (node 0) after completing all assigned missions. This is enforced using a Big-M formulation linked to the routing variable z_{ij0} , which equals 1 only for the final point of the route.

$$C_{i,\text{last}} \geq C_j + T_{j,0} - M(1 - z_{ij0}) \quad \forall i \in I_{\max}, j \in J \quad (2.12)$$

- The time elapsed between an operator's first assigned mission and his last assigned order must not exceed the fixed shift limit H_{fixed} .

$$C_{i,\text{last}} - S_{i,\text{first}} \leq H_{\text{fixed}} \cdot y_i \quad \forall i \in I_{\max} \quad (2.13)$$

Where

$C_{i,\text{last}}$: The completion time of last mission per operator i .

$S_{i,\text{first}}$: The start time of first mission per operator i .

- The start and completion bounds for each operator i are defined based on their initial and final movements regarding the base node:

* **Start time of the first mission:**

$$S_{i,\text{first}} = \begin{cases} S_k & \text{if } z_{i,0,k} = 1 \\ 0 & \text{otherwise} \end{cases}$$

* **Completion time of the last mission:**

$$C_{i,\text{last}} = \begin{cases} C_j + T_{j,0} & \text{if } z_{i,j,0} = 1 \\ 0 & \text{otherwise} \end{cases}$$

Note that: $P_{i,0} = 0, S_0 = 0, C_0 = 0$.

- Symmetry breaking: In order to reduce redundant equivalent solutions, operators must be activated in a sequential order:

$$y_i \leq y_{i-1} \quad \forall i \in I_{\max}, i \geq 1 \quad (2.14)$$

2.3 Additional Problem Formulation: Real-World Constraints

Operational Characteristics and Skill Compatibility

In the problem definition (sec. 2.1), logistic operations were defined as picking and unpicking tasks requiring specific processing times. In practice, this processing time is highly dependent on the physical characteristics of the material handling machine (e.g., forklifts) utilized by the assigned operator.

The primary movement unit in these logistic operations is the "pallet" which is categorized into various types based on its dimensions. Each pallet has predefined dimensions and weight, which physically constrain the loading, unloading, and travel speeds of the machine during its mission. Consequently, these physical properties directly influence the mission's required processing time.

Because a warehouse fleet typically consists of heterogeneous forklift models—each with differing fork dimensions and payload capacities—the mechanical mismatch between a machine's capabilities and a pallet's requirements must be minimized. To encapsulate this dynamic, the multi-criteria COP is extended to include an "**operator skill score**". This criterion aims to maximize the compatibility between the assigned operator and the requested pallet type. A higher skill score denotes a better physical fit, which implicitly minimizes the processing time.

Incorporating this criterion introduces the following parameters, constraints, and objective modifications:

Parameters:

- Q_{iu} (skill score): A numerical parameter representing how adequately forklift/operator i can handle a pallet of type u .
- O_{ju} (order type): A binary parameter that is 1 if order j is of pallet type u , and 0 otherwise.

Constraints: An order j can only be assigned to operator i if the operator possesses a skill score of at least 1 for that specific order type. This ensures strict physical compatibility:

$$\sum_{u \in U} O_{ju} \cdot Q_{iu} \geq x_{ij} \quad \forall i \in I_{max}, j \in J \quad (2.15)$$

Objective Function: To optimize the assignments, the overall system skill score must be maximized. Because the primary objective function is framed as a minimization problem, maximizing the skill score is mathematically equivalent to minimizing its negation (i.e., $\max f(x) \equiv \min -f(x)$).

By evaluating the total compatibility score of all active assignments $(\sum_{i,j} x_{ij} \sum_u O_{ju} Q_{iu})$, the new multi-objective function becomes:

$$\textbf{Minimize } \alpha \cdot Z + \beta \cdot \sum_{i \in I_{max}} y_i - \gamma \cdot \sum_{i \in I_{max}} \sum_{j \in J} \left(x_{ij} \sum_{u \in U} O_{ju} \cdot Q_{iu} \right) \quad (2.16)$$

Where α , β , and γ are normalization weights dictating the operational trade-off between time efficiency, labor cost, and machine-pallet compatibility.

For the sake of simplicity, this thesis maintains the objective function defined in (sec.2.1) and does not incorporate skill score optimization.

3. Mixed-Integer Programming Model

This chapter details the computational implementation of the mathematical framework established in the previous chapter. By instantiating the algebraic formulation as an exact mixed-integer programming (MIP) model, we construct a deterministic framework capable of exploring the combinatorial search space.

3.1 Optimality: Justification for Mixed-Integer Programming (MIP)

Mathematical optimization models are broadly categorized based on the domain of their decision variables. While a pure integer programming (IP) [10] model requires all variables to take discrete integer values ($x \in Z$), a MIP model accommodates a hybrid structure containing both discrete and continuous variables ($x \in Z, y \in R$).

For the spatially constrained operator-mission assignment problem, a MIP formulation is strictly required rather than a pure IP due to the fundamental nature of time. The operational decisions—such as whether to assign an operator to a mission (x_{ij}) or sequence one mission after another (z_{ijk})—are inherently binary discrete choices. However, the temporal variables—specifically the start times (S_j), completion times (C_j), and the global makespan (Z)—represent continuous durations.

Attempting to model this as a pure IP would require discretizing time into fixed intervals (e.g., a time-indexed formulation). Discretizing time is highly inefficient for scheduling problems with large horizons (H_{fixed}), as it triggers an exponential explosion in the number of variables (creating a binary variable for every possible second a task could start). By utilizing a MIP formulation, the model elegantly isolates the combinatorial complexity to the binary routing and assignment decisions, while allowing the exact temporal dynamics to be resolved in the continuous domain (R^+).

Solution Methodology: Branch-and-Bound Algorithm

To find the globally optimal schedule, modern exact solvers (such as Gurobi or CPLEX) rely on the **branch-and-bound (B&B)** algorithm [4], often augmented with cutting planes (Branch-and-Cut).

Because combinatorial problems are NP-hard, exploring every possible assignment sequence via brute force is computationally impossible. Instead, the solver uses a divide-and-conquer strategy that systematically partitions the search space while using mathematical bounds to ignore (prune) sub-optimal regions. The algorithm operates through the following core steps:

1. **Linear programming (LP) relaxation:** The solver begins at the root node by temporarily relaxing the binary constraints, allowing discrete variables to take fractional values (e.g., $x_{ij} = 0.5$). Solving this continuous LP is computationally fast and provides a strict *lower bound* (LB) on the minimization objective.
2. **Branching:** If the LP solution contains fractional values for variables that must be binary, the solver splits (branches) the problem into two mutually exclusive sub-problems. For example, it creates one branch where $x_{ij} = 0$ and another where $x_{ij} = 1$. This creates a search tree.
3. **Bounding and incumbents:** As the solver progresses down the tree, it occasionally finds a feasible solution where all integer constraints are naturally satisfied. The best integer solution found so far is called the *incumbent*, which serves as a strict *upper bound* (UB).

4. **Pruning:** If the solver evaluates a node and finds that its LP Lower Bound is strictly greater than the current incumbent upper bound, it immediately discards (prunes) that entire section of the tree.

Optimality is proven when the global lower bound of all unexplored nodes perfectly matches the upper bound of the incumbent solution (i.e., the optimality gap reaches 0%).

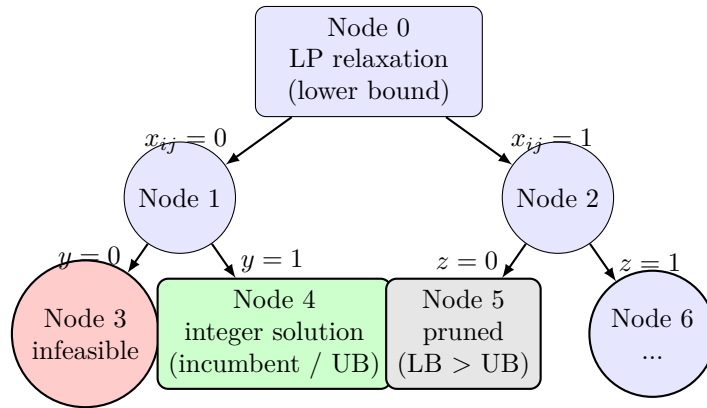


Figure 3.1: Simplified Illustration of the B&B Tree.

3.2 Computational Implementation

The mathematical formulation is translated into a computational MIP model using a state-of-the-art branch-and-bound optimization solver. The model explicitly defines the binary assignment variables, sequencing variables, and continuous temporal variables—defined in sec.2.2—to construct feasible routing and scheduling policies.

A critical modeling aspect of the proposed MIP formulation concerns the choice of the Big- M coefficients used to linearize the temporal sequencing constraints, clearly used in (2.4)& (2.11) in sec.2.2. The standard static value of Big- M is naturally $\geq 1e4$.

Although Big- M linearizations are standard in mixed-integer modeling, excessively large values lead to weak linear programming relaxations, poorer numerical conditioning, and slower branch-and-bound convergence—leading to severe computational inefficiencies.

For this reason, M should not be selected as a static arbitrary constant, but rather as the smallest valid upper bound implied by the structure of the problem instance. The Big- M or its equivalent values should be chosen as tight, problem-dependent bounds derived from the planning horizon and the maximum feasible travel and processing times. This modeling choice is consistent with the formulation principles discussed by Vielma[11], who emphasizes that strong mixed-integer linear formulations rely on tight bounds and carefully designed linearization constants rather than overly conservative coefficients.

Therefore, we choose to define M_{time} which is dynamically calculated for each instance. It is bounded tightly based on the maximum possible travel times (T_{jk}) and the shift capacity (H_{fixed}), ensuring numerical stability and accelerating the solver's convergence. The new M_{time} is replacing the standard Big- M used in eq. (2.5) & (2.12) of sec.2.2.

A naive approach to defining a Big- M alternative parameter for the temporal sequencing constraints is to calculate the absolute worst-case sequence in the entire warehouse. This implies an operator executing every single mission in the dataset using the slowest possible processing times and longest possible travel routes. The M_{time} constant is calculated to provide a valid upper bound on the total time horizon, ensuring the temporal constraints remain linear without being unnecessarily loose:

$$M_{time} = \sum_{j \in J} \left(\max_{k \in J_0} T_{jk} \right) + \sum_{j \in J} \left(\max_{i \in I_{max}} P_{ij} \right) + 1 \quad (3.1)$$

Where:

- $\sum_{j \in J} \left(\max_{k \in J_0} T_{jk} \right)$: The sum of the longest possible travel times departing from each order j to any destination k (including the base depot $\{0\}$).
- $\sum_{j \in J} \left(\max_{i \in I_{max}} P_{ij} \right)$: The sum of the worst-case processing times for each order j (i.e., the total duration if every order were assigned to its slowest capable operator).
- $+1$: A constant buffer to ensure the M_{time} remains strictly greater than any feasible path duration, maintaining numerical stability.

Solver Configurations and Hyperparameters

Given the NP-hard nature of the underlying heterogeneous vehicle routing and scheduling problem, exact solvers experience exponential time complexity as the problem size scales. To make the execution computationally affordable, specific solver configurations are enforced:

Time limits: A strict computational time limit is imposed on each problem instance (e.g., 15 minutes). If the solver cannot find the globally optimal solution within this window, the search is terminated, and the best-found feasible solution (the incumbent) is retained.

Relative MIP gap tolerance: The solver is configured to accept solutions that fall within a predefined optimality gap (e.g., 1% or 5%). Once the gap between the lower bound and the incumbent solution falls below this threshold, the solution is deemed sufficiently optimal for downstream use.

Multi-objective weighting: In the present business case, the primary objective is to minimize operator activation, with makespan minimization serving as a secondary goal. To enforce this hierarchy, the weight β must be sufficiently large relative to α . A naive approach to ensure that the solver prioritizes activation in all scenarios is to set $\beta = M$ and $\alpha = 1$. However, to maintain numerical stability and avoid excessively large coefficients, we only require β to exceed the maximum possible value of the makespan. Given that the makespan is strictly bounded by the shift duration limit H_{fixed} , we can effectively set $\beta = H_{\text{fixed}} + 1$ while maintaining $\alpha = 1$.

By setting $\beta = H_{\text{fixed}} + 1$, we guarantee that:

$$\beta \cdot y_i > \alpha \cdot \text{Makespan}$$

even if the makespan is at its absolute maximum. This effectively tells the solver: "saving one operator is always more valuable than saving any amount of time within a single shift."

3.3 Primary Solving Approach

While the theoretical mathematical formulation defines a single weighted-sum objective function in sec. 2.2 of the previous chapter, directly solving this aggregate function with an exact solver frequently introduces practical challenges. Standard weighted-sum optimization can obscure critical trade-offs due to scaling disparities between the two criteria (e.g., minutes vs. operator counts). Furthermore, it often leaves "routing noise"—inefficient, meandering routes for non-bottleneck operators—because the solver stops optimizing once the global makespan is locked to the upper bound.

To resolve this, traditional operations research (OR) often employs lexicographic optimization (LO) [12], which optimizes criteria sequentially. However, in this specific multi-criteria scheduling problem, the makespan and operator activation rate are intrinsically coupled. Optimizing one criterion inherently depends on the knowledge of the other: the optimal number of operators to activate depends on the target makespan to be achieved, and conversely, the optimal makespan cannot be determined without knowing the available active fleet size. A pure lexicographic approach forces sequential rigidity (e.g., locking the minimum number of operators before considering makespan), which frequently leads to severely sub-optimal global trade-offs and numerical infeasibility.

To retain the global trade-off visibility of the weighted-sum method while achieving the routing perfection typically associated with lexicographic optimization, the primary oracle used to generate high-quality schedules employs a custom **hybrid two-phase approach** enhanced by an initial capacity-based bounding mechanism.

Pre-Processing: The Bin Packing Cut

To accelerate the exact solver and tighten the mathematical formulation, a *bin packing cut* is applied prior to optimization. Because the branch-and-bound algorithm evaluates the optimal number of active operators ($\sum y_i$), exploring sub-trees with an impossibly small fleet size wastes significant computational time. The bin packing cut establishes a strict theoretical lower bound on the fleet size by aggregating the absolute minimum processing time ($\min P_{ij}$) and the minimum inbound travel time ($\min T_{kj}$) for every mission in the batch. By dividing this theoretical minimum total workload by the shift capacity (H_{fixed}) and applying a ceiling function, the model mathematically enforces a

minimum operator count constraint:

$$\sum_{i \in I_{max}} y_i \geq \left\lceil \frac{\text{Total Minimum Workload}}{H_{fixed}} \right\rceil \quad (3.2)$$

This valid inequality safely prunes massive infeasible sections of the exponential search tree without altering the optimal solution space, drastically speeding up initial solver convergence. The formal derivation of the *total minimum workload* is detailed in chapter 7, alongside other theoretical lower bound calculations.

Following this pre-processing step, the exact solver executes the two primary phases:

1. **Phase 1 - Global trade-off optimization:** The solver executes the standard weighted-sum objective ($\alpha Z + \beta \sum y_i$). This permits the branch-and-bound algorithm to dynamically evaluate the trade-off between adding an operator and reducing the makespan, finding the globally optimal combination dictated by the business weights without requiring capacity relaxations.
2. **Phase 2 - Noise elimination (total flow optimization):** Upon reaching optimality in phase 1, the algorithm locks both the optimal number of operators and the global makespan (incorporating a negligible $1e - 4$ numerical tolerance to prevent infeasibility). The objective function is then aggressively swapped to minimize the total flow time ($\sum C_j$) maintaining the assignments made in phase 1.

In summary, the hybrid approach guarantees that the generated schedules are both strategically optimal (perfect balance of cost and speed) and tactically flawless (no internal routing noise or wasteful operator paths), all while maintaining strict mathematical feasibility.

3.4 Limitations of the Exact MIP Solver in Practical Business Scenarios

While the MIP model guarantees mathematical optimality and provides an essential baseline for evaluation, its deployment as a standalone scheduling engine is fundamen-

tally incompatible with the requirements of a large-scale, real-world business environment. The inadequacies of the exact solver in practical logistics comes primarily from three operational limitations: computational intractability at scale, latency with respect to business-based throughput requirements, and deterministic rigidity.

Computational intractability at scale (the curse of dimensionality)

The spatially constrained operator-mission assignment problem inherently encompasses the NP-hard complexities of scheduling VRP. As the number of orders (J) and available operators (I_{max}) increases, the number of binary decision variables—specifically the routing assignments z_{ijk} —scales quadratically with respect to the orders, resulting in a model complexity of $O(I_{max} \cdot |J|^2)$. Because exact solvers rely on B&B algorithms to resolve these discrete choices, the theoretical size of the search tree grows exponentially, bounded by $O(2^{I_{max} \cdot |J|^2})$.

The curse of dimensionality: described by Bellman [13] as the exponential increase in computational difficulty, memory, or data requirements that occurs when moving from low-dimensional to high-dimensional spaces.

While the solver’s pruning mechanisms efficiently navigate this exponential space for small spatial clusters (e.g., mini-batches of 20 to 50 missions), modern warehouse operations routinely process batches ranging from 1,000 to 5,000 missions executed by hundreds of forklifts. At this industrial scale, the combinatorial explosion renders exact solvers computationally paralyzed; the time required to navigate the exponential search tree to find a feasible initial solution—let alone prove optimality—vastly exceeds practical operational limits.

Latency and throughput requirements

In industrial business cases, operational efficiency relies on high-throughput decision-making. Warehouse management systems must frequently generate and dispatch schedules within seconds (in case of continuous batching) to prevent operator idle time, meanwhile in our business case the batch is sent the day before and the schedule must be obtained within few hours. The MIP solver, relying on continuous LP relaxations and branch pruning, may require hours or days of intensive CPU computation to converge on a high-quality solution for large-scale batches. This high latency creates a critical bottleneck, where the mathematical optimization process itself becomes impractical.

Deterministic rigidity in dynamic environments

The MIP formulation is strictly deterministic, assuming a static and perfectly predictable

environment once execution begins. However, real-world logistics are highly stochastic: forklifts experience mechanical delays, travel paths become congested, and high-priority orders arrive dynamically. When such disruptions occur, a previously generated optimal MIP schedule becomes instantly invalid. Because exact solvers lack the ability to quickly extrapolate from previous solutions (short of complex heuristic warm-starts), adapting to a disrupted environment requires triggering a full re-computation from scratch, multiplying the severe latency issues discussed above.

Transition to neural combinatorial optimization

Consequently, while the MIP solver is an indispensable component for generating ground-truth data in offline, small-scale environments, its operational limitations necessitate a different approach for production-level execution. This structural gap motivates the adoption of a learning-based approach—specifically the graph neural network (GNN) solver. By shifting the heavy computational burden to an offline training phase, the neural solver learns to approximate the MIP’s exact logic imitating its optimality, enabling fast inference, and dynamic adaptability at runtime.

3.5 The Dual Role of the MIP Solver

In traditional OR, an exact solver is primarily used to directly solve the operational problem. In a neural combinatorial optimization framework, the MIP model assumes two distinct roles: an oracle for imitation learning and an exact benchmark for performance evaluation.

Ground Truth for Imitation Learning (Training Data Generation)

The neural model (e.g., GNN) requires a massive dataset of high-quality solved instances to learn effective assignment and sequencing policies. In this context, the MIP model acts as the “expert teacher” for the neural model.

Because the MIP solver cannot scale to compute whole warehouses, the global dataset is

preprocessed using spatial clustering to generate dense mini-batches. The MIP solver independently processes these mini-batches, generating ground-truth labels for the routing variables (z_{ijk}) and activation variables (y_i). By studying these optimal or near-optimal exact solutions, the neural model learns how to pack missions efficiently into shifts without violating the H_{fixed} constraints.

Exact Baseline for Evaluation

During the inference and evaluation phase, the MIP solver transitions into a benchmark against which the neural model’s performance is rigorously measured. By running both the trained neural model and the MIP solver on unseen test instances, we can quantitatively evaluate the learned model. The MIP provides the mathematical optimal baselines.

4. Graph Neural Networks and the One-Shot Architecture

This chapter details the architecture of the proposed neural combinatorial solver. To approximate the optimal scheduling policies generated by the MIP oracle, the heterogeneous graph of logistics problem is adapted to graph neural network (GNN). The proposed architecture leverages the message passing neural network (MPNN) framework, specifically employing heterogeneous graph attention network v2 (HeteroConv + GATv2) layers to capture complex spatial and capacity constraints, followed by multi-layer perceptrons (MLPs) to decode node embeddings into assignable routing probabilities.

4.1 Imitation Learning: The Scope of the Neural Solver

In the context of combinatorial optimization, machine learning models are typically trained using either reinforcement learning (RL) or imitation learning (IL). Imitation learning [14] is the problem of emulating an expert policy for decision making by learning from demonstrations. Compared to RL, the learner in IL does not observe reward-based feedback, and must imitate the expert's behavior based on demonstrations alone; their objective is to achieve performance close to that of the expert on an unobserved reward function.

To structure this training process, it is necessary to contextualize the approach within the primary subfields of imitation learning:

1. **Inverse Reinforcement Learning (IRL)**: In this paradigm, the agent observes the expert to deduce the underlying reward function driving the expert’s behavior. This is unnecessary for our scheduling problem, as the objective function (minimizing makespan and activation) is already explicitly defined mathematically.
2. **Interactive Imitation Learning (IIL)**: Algorithms like dataset aggregation (Dagger) train the model interactively. The neural network generates a partial schedule, and if it makes a sub-optimal routing decision, the system pauses and queries the exact MIP solver to calculate the optimal recovery path from that specific new state. While highly robust against cascading errors, Interactive IL requires running the computationally heavy branch-and-bound MIP solver repeatedly *inside* the neural network’s training loop, rendering it prohibitively slow for large datasets.
3. **Offline Behavioral Cloning (BC)**[15]: In this purely supervised approach, a fixed dataset of optimal expert solutions is generated entirely prior to training. The model is then trained offline to predict the expert’s discrete actions given the initial state, utilizing standard classification loss functions.

Due to the extreme computational latency of the MIP oracle, this thesis explicitly adopts *imitation learning* framework—specifically, an **offline behavioral cloning** approach. The fundamental scope of the neural solver is not to discover new operational rules from scratch, but to imitate the mathematical optimal schedules generated by the exact MIP model. The scheduling problem is framed as a supervised multi-task classification pipeline:

- **Input**: The physical layout of the warehouse, operator capacities, and unassigned missions are provided to the network as a graph.
- **Expert’s action (ground-truth labels)**: The offline MIP solver dictates the optimal discrete actions for that static state—specifically, which operators to activate (y_i), which assignments to make (x_{ij}), and which routes to sequence (z_{ijk}).
- **Learning objective**: The neural solver optimizes its internal weights to minimize the loss between its own predictions and the MIP model’s binary decisions.

At inference time, the neural solver is completely decoupled from the exact solver, allowing it to predict near-optimal schedules in seconds. However, translating the warehouse input into a format that the neural network can process requires an architecture capable of understanding relational spatial topologies, which introduces the necessity of graph neural networks.

4.2 Introduction to Graph Neural Networks (GNNs)

Deep learning models, such as multi-layer perceptrons (MLPs) and convolutional neural networks (CNNs), are traditionally designed to process data in structured, euclidean formats, such as fixed-size grid arrays (images) or sequential vectors (text). However, many real-world operational problems—including supply chain logistics, routing, and scheduling—cannot be naturally forced into these rigid structures without losing critical relational and spatial information.

Graphs provide a natural, non-euclidean representation of such problems, where physical entities are represented as nodes and their relationships or spatial distances are represented as edges. GNNs are a specialized class of neural architectures designed to operate directly on this graph-structured data. By applying iterative, localized mathematical transformations (neural message passing) across the nodes and edges, GNNs learn continuous vector representations (embeddings) that capture both the individual features of a node and the structural topology of its surrounding neighborhood.

Core GNN Architectures

The foundational logic of modern GNNs is unified under the message passing neural network (MPNN) framework (see next subsection). GNNs are broadly categorized into several primary architectures:

Graph Convolutional Network (GCN): GCNs approximate spectral graph filtering by isotropically aggregating information from all neighbors. In a GCN, edge weights

are generally static and defined purely by the topological structure of the graph (e.g., normalized by node degrees). While highly efficient for simple node classification, GCNs treat all neighbors equally and cannot natively incorporate explicit, variable edge features (such as continuous travel times) without complex architectural modifications.

GraphSAGE (Sample and Aggregate): Designed specifically for scalability on massive graphs (e.g., millions of nodes), GraphSAGE samples a fixed-size neighborhood for each node during training rather than processing the entire graph. While it allows for inductive learning and handles large-scale data well, its standard mean- or max-pooling aggregators lack the fine-grained, pairwise interaction necessary to solve precise combinatorial matching tasks.

Graph Isomorphism Network (GIN): GINs are designed to maximize the discriminative power over entire graph structures, making them exceptionally powerful for graph-level classification tasks (e.g., predicting if a specific molecule is toxic). However, they rely on simple sum aggregators that can be numerically unstable when applied to optimization tasks requiring fine-grained node-level or edge-level routing predictions.

Graph Attention Network (GAT): GATs introduce an anisotropic aggregation mechanism. Rather than treating all neighbors equally, GATs compute dynamic attention scores (α_{ij}) between connected nodes, allowing the network to selectively prioritize specific, important neighbors.

Overview of Message Passing Framework (MPNN)

Let $G = (V, E)$ denote a graph, where V is the set of nodes and E is the set of edges. For each node $u \in V$, let x_u denote its input feature vector, and let e_{uv} denote the feature vector associated with edge $(u, v) \in E$.

In a message-passing GNN, each node embedding is updated iteratively by aggregating information from its neighborhood $\mathcal{N}(u)$. At layer k , the embedding of node u is computed as

$$h_u^{(k+1)} = \text{UPDATE}^{(k)}\left(h_u^{(k)}, \text{AGGREGATE}^{(k)}\left(\{h_v^{(k)} : v \in \mathcal{N}(u)\}\right)\right) \quad (4.1)$$

$$= \text{UPDATE}^{(k)}\left(h_u^{(k)}, m_{\mathcal{N}(u)}^{(k)}\right), \quad (4.2)$$

where AGGREGATE and UPDATE are differentiable functions. The initial node representations are given by

$$h_u^{(0)} = x_u, \quad \forall u \in V. \quad (4.3)$$

After K message-passing steps, the final embedding of node u is

$$z_u = h_u^{(K)}, \quad \forall u \in V. \quad (4.4)$$

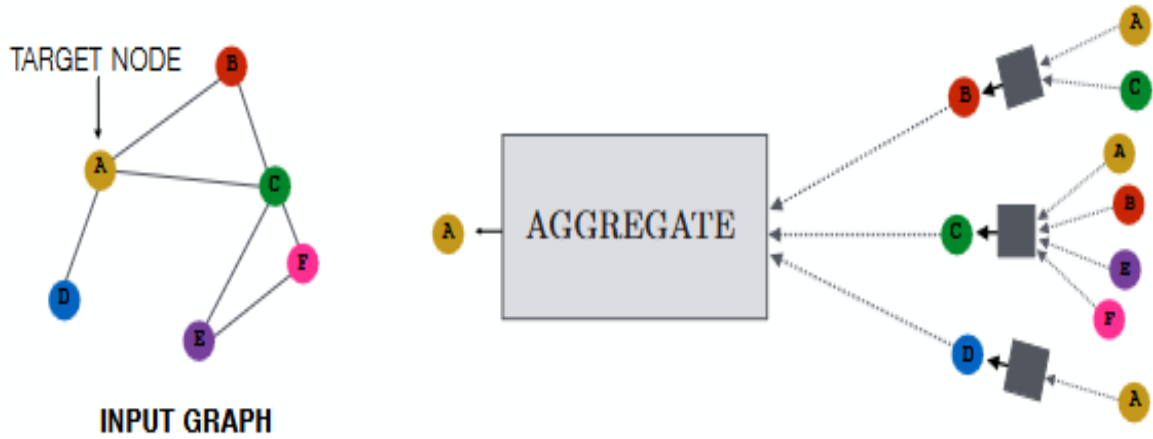


Figure 4.1: GNN: Single Node Message Aggregation from Its Local Neighborhood

The three fundamental aggregation operators, often referred to as pooling methods, are:

- Mean aggregation (averaging): It computes the element-wise arithmetic average of the incoming message vectors from all neighbors. Commonly utilized in standard GCNs. Mean aggregation is highly effective at capturing the general distribution or macroscopic trend of a neighborhood.

While it ensures numerical stability during training, it washes out distinct, individual signals. In a routing context, if an operator is surrounded by five high-cost missions and one highly optimal, low-cost mission, mean aggregation will blend these values, potentially obscuring the existence of the optimal choice.

- Sum aggregation: It computes the element-wise sum of all incoming neighbor feature vectors. Frequently utilized in architectures like GINs and GraphSAGE variants. Unlike mean pooling, sum aggregation preserves information about the graph's size and node degree (i.e., a node with 10 neighbors will yield a larger aggregated vector than a node with 2 neighbors). It is mathematically proven to

be the most expressive aggregator for distinguishing different graph structures. Sum aggregation can be numerically unstable during optimization. In combinatorial problems where feature magnitudes must be explicitly minimized (e.g., minimizing travel times), a high degree node will accumulate massive numerical values, causing gradients to explode during backpropagation.

- **Max aggregation (max pooling):** It performs an element-wise maximum operation across the neighborhood, retaining only the highest scalar value for each feature dimension. Max pooling acts as a strong feature detector, isolating the most dominant or salient attribute within a neighborhood regardless of how many other neighbors exist.

It completely ignores the distribution and magnitude of the rest of the neighborhood. An operator node would only recognize the single worst-case (or best-case) travel time, rendering it blind to the density of the surrounding workload.

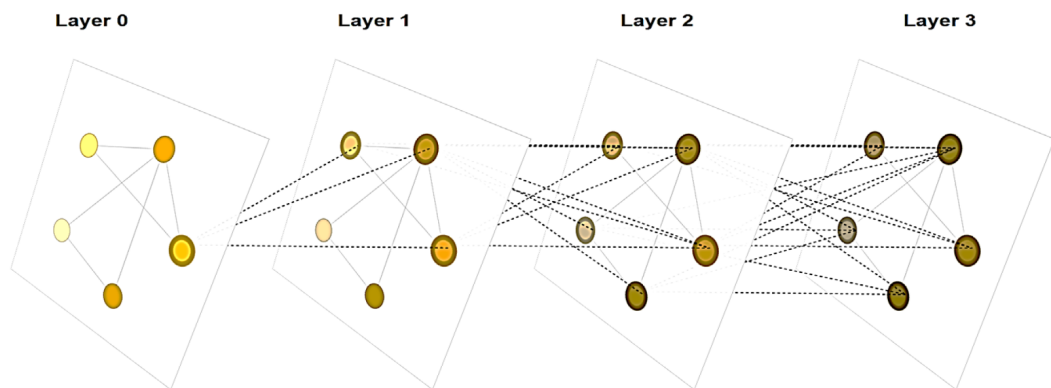


Figure 4.2: GNN: The Process of Updating Node Embedding through Multiple Layers. [16].

Neighborhood Determination and Scalable Message Passing

A fundamental challenge in applying GNN to industrial scheduling is computational scalability. In a naive representation of the routing problem, the graph would be fully connected: every operator would be linked to every order, and every order would have a sequence edge to every other order. While this works for small academic instances,

scaling to real-world environments—which may contain hundreds of operators and 1,000 to 10,000+ orders—results in a dense adjacency matrix with quadratic $\mathcal{O}(|V|^2)$ edge growth.

During the message-passing phase, aggregating feature vectors across a fully connected graph of this magnitude triggers the “neighborhood explosion” phenomenon [17]. The exponential growth of message generation across multiple GNN layers quickly exhausts GPU memory limits, rendering full-batch training impossible.

Dynamic Sampling vs. Static Spatial Pruning

To mitigate neighborhood explosion, the standard approach in modern large-scale GNN literature relies on dynamic mini-batch sampling techniques. Methods such as standard neighbor sampling (NS) or the more advanced layer-neighbor sampling (LABOR) [17] dynamically restrict the receptive field by randomly sampling a fixed number of neighbors (fanout) for each target node during training. LABOR specifically improves upon NS by overlapping sampled neighborhoods across layers, significantly reducing the memory footprint without sacrificing gradient variance.

However, while dynamic sampling is highly effective for general-purpose networks, applying random stochastic sampling to routing and scheduling graphs risks breaking critical physical topologies. Randomly dropping an adjacent sequence edge might artificially sever the optimal shortest path, blinding the network to the true global optimum.

Therefore, rather than relying on stochastic dynamic sampling during training, this architecture utilizes strict *static spatial and temporal pruning* prior to message passing. We bound the receptive field of each node by pruning the adjacency matrix using deterministic heuristics:

- **Sequence neighborhoods (order-to-order):** For any given order j , potential successor edges ($j \rightarrow k$) are restricted strictly to its K -nearest physical neighbors. Orders located at opposite ends of the facility are structurally disconnected, as transitioning between them is physically suboptimal.
- **Assignment neighborhoods (operator-to-order):** Edges connecting operators to orders are pruned based on operational zoning and physical constraints (e.g., matching operator capacity to specific pallet dimensions).

Impact on the Attention Mechanism

By explicitly defining these sparse, physically bounded neighborhoods, the message-passing complexity is reduced from $\mathcal{O}(|V|^2)$ to $\mathcal{O}(|V| + |E_{sparse}|)$. This deterministic sparsification is particularly critical for the GATv2 layers. Restricting the neighborhood ensures that the attention mechanism does not weaken its weights over thousands of irrelevant, distant nodes. Unlike dynamic sampling methods, which introduce variance across epochs, our static pruning guarantees that the GNN consistently evaluates the true competitive local paths. This allows the architecture to scale seamlessly to tens of thousands of orders while maintaining highly discriminative spatial embeddings and strictly respecting the physical reality of the warehouse environment.

In our *real-world experiments*, Since the batch of missions to schedule is not exceeding 1,000 missions; and for the sake of simplicity, we decided to keep the *full connectivity* between nodes.

4.3 Message Passing Bi-Directionality in Warehouse Heterogeneous Graphs

In a GNN layer, every node embedding h_u is updated by aggregating messages from its neighbors. To capture the complex dependencies in a warehouse environment, information must flow bi-directionally. An order node must perceive its assigned operator to update its temporal status, while an operator node must aggregate features from its assigned orders to update its capacity and state embeddings.

To facilitate this, we re-formalize our data object to include bi-directional edges, ensuring operator nodes can receive messages from order nodes and vice versa. Consequently, the graph representation originally presented in sec. 2.1 of ch. 2 is extended to enable operator nodes to receive order's messages as illustrated in fig. 4.3:

Operator embeddings ($h_{i \in I}$): These are updated by aggregating messages from orders through reversed assignment edges ($j \xrightarrow{\text{rev}} i$), allowing the operator to "sense" the total workload of its assigned missions through the new arc "rev_assignment".

Order Embeddings ($h_{j \in J}$): These are updated by aggregating messages from both

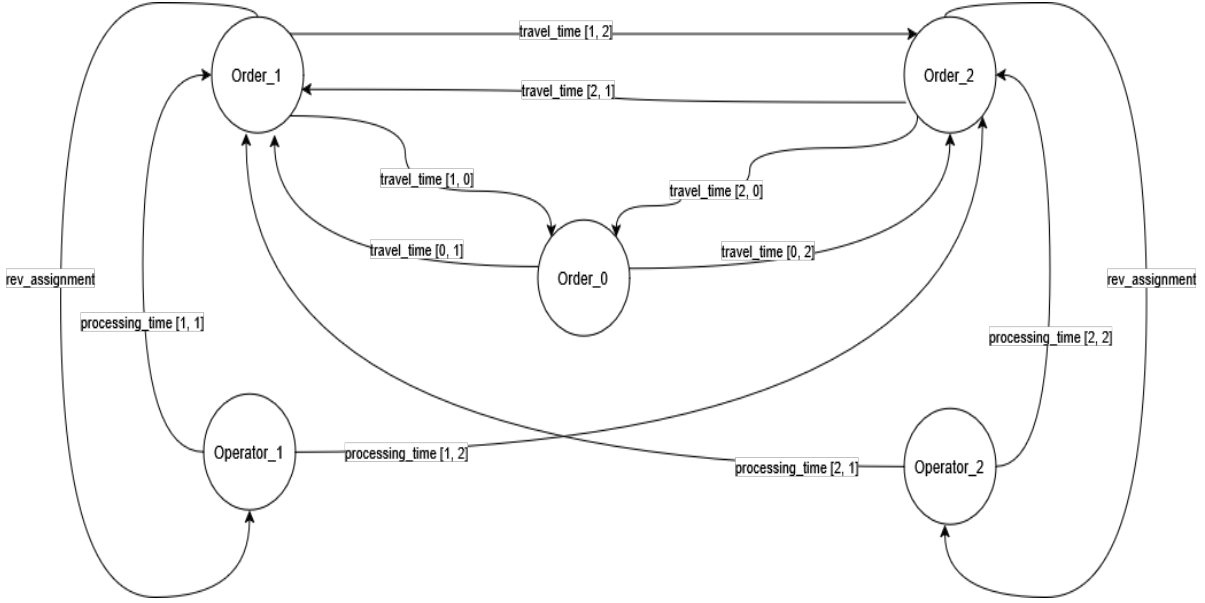


Figure 4.3: Graph Representation for Message Passing.

intra-set sequencing edges ($j' \rightarrow j$) and assignment edges ($i \rightarrow j$) from the operator set. We don't have to add the reverse arc between each order pair since we already have asymmetric E_T edges.

Justification for GATv2 in Warehouse Routing

For the heterogeneous operator-mission scheduling problem studied in this thesis, the routing decisions depend heavily on precise spatial distances. Therefore, standard isotropic GCNs and GraphSAGE architectures are inadequate, as averaging neighborhood features inherently washes out critical spatial routing signals.

The GAT architecture [18] was selected because its attention mechanism inherently supports the integration of edge features (i.e., travel time T_{ij}) directly into the weighting logic. If a travel time penalty is exceptionally high, the GAT layer can dynamically assign a near-zero attention score to that edge, effectively recognizing it as an operational "wall" during message passing.

Specifically, this thesis implements the **GATv2** variation [19]. The original GAT suffers from a known *static attention flaw* where the non-linear activation is applied after the parameter projection, limiting its expressiveness. To resolve this, the proposed archi-

itecture employs the GATv2 mechanism, which achieves universal “dynamic attention” by altering the order of operations. By concatenating the node and edge features, applying the shared linear transformation, and passing the result through a **LeakyReLU** activation *before* the final vector projection, the attention function can uniquely rank neighbors for every specific query node.

Formally, the unnormalized attention score between node i and neighbor j is computed as:

$$score_{ij} = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{W} \cdot [\mathbf{h}_i || \mathbf{h}_j || \mathbf{e}_{ij}]) \quad (4.5)$$

Where $\mathbf{h}_i$ and $\mathbf{h}_j$ are the respective node embeddings, $\mathbf{e}_{ij}$ is the explicit edge feature vector (e.g., travel time T_{ij} and processing time P_{ij}), $||$ denotes concatenation, $\mathbf{W}$ is the shared learnable weight matrix, and $\mathbf{a}$ is the learnable attention parameter vector.

The final attention coefficients are then normalized across the neighborhood $\mathcal{N}(i)$ using the softmax function:

$$\alpha_{ij} = \text{softmax}_j(e(h_i, h_j)) = \frac{\exp(score_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(score_{ik})} \quad (4.6)$$

These dynamic coefficients (α_{ij}) dictate the exact weight with which neighbor j 's features are aggregated into node i 's updated embedding during the message-passing step.

4.4 Multi-Layer Perceptrons (MLP) in Graph Neural Networks

Before detailing the specific multi-criteria neural architectures, it is necessary to establish the foundational role of the MLP within the GNN framework. The MLP is one of the most fundamental architectures in deep learning, consisting of multiple layers of nodes (neurons) in a directed graph, with each layer fully connected to the next [20].

A standard MLP (see fig.4.4) consists of an input layer, one or more hidden layers, and an output layer. The transformation of a feature vector x from layer l to layer $l + 1$ is mathematically defined as:

$$h^{(l+1)} = \sigma(w^{(l)}h^{(l)} + b^{(l)}) \quad (4.7)$$

where $w^{(l)}$ is the learned weight matrix, $b^{(l)}$ is the bias vector, and $\sigma(\cdot)$ represents a non-linear activation function (e.g., ReLU or sigmoid). By stacking these non-linear transformations, MLPs act as universal function approximators capable of learning highly complex, non-linear mappings from input to output spaces [21].

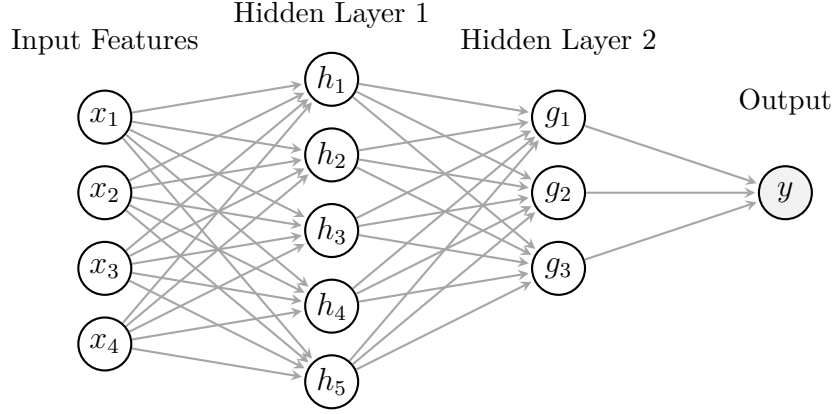


Figure 4.4: Example of a Multi-Layer Perceptron (MLP) Architecture with Two Hidden Layers.

In the context of GNNs, the architectural paradigm heavily relies on decoupling structural message-passing from local feature transformation [22, 16]. While convolutional operators (such as GATv2) are responsible for aggregating relational information and graph topology, MLPs are strictly responsible for processing the resultant latent embeddings.

Specifically, MLPs in GNNs serve two primary functions:

1. **Latent feature projection:** MLPs are applied independently and identically across all nodes (or edges) to project raw input features into higher-dimensional latent spaces before message-passing begins, ensuring the convolutional layers operate on expressively rich embeddings [16].
2. **Task-specific decoding (heads):** Once the message-passing layers have updated the node representations with neighborhood context, MLPs are used as terminal decoding heads. Because the topological context is already embedded within the features by the GNN layers, the MLP can act locally on individual nodes or node-pairs to output final predictions, such as classification logits or regression values [23, 24].

In the GNN architectures we are proposing in this chapter and the next chapter, we leverage this paradigm by employing shallow MLPs as parallel task-specific decoding **heads**. By doing so, the model isolates the spatial routing logic within the GATv2 layers, while the MLPs independently map the dynamically updated operator and order embeddings into distinct probabilistic distributions.

4.5 Proposed Architecture: One-Shot GNN and Coupled Multi-Task MLP Heads

After completing the message-passing iterations within the GATv2 encoder, the GNN yields final, context-aware continuous embeddings for every order ($\mathbf{z}_j$) and operator ($\mathbf{z}_i$). Because this architecture is trained via offline behavioral cloning to directly imitate the discrete choices of the MIP oracle, these embeddings must be translated into explicit probability scores.

This translation is framed as a supervised multi-task learning problem. Rather than designing entirely separate networks for each decision, the architecture leverages a single shared encoder (the GATv2 layers) acting as a “trunk”, which subsequently splits into three parallel MLP decoding heads. Each head consists of three fully connected linear layers, utilizing ReLU activations for the hidden layers, and concluding with a sigmoid activation (σ) to ensure outputs strictly represent bounded probabilities ($\hat{P} \in [0, 1]$).

Global Context Vector

To ensure that all discrete routing and assignment decisions remain mathematically bounded by the parameters defining the specific scheduling instance, a global context vector ($\mathbf{u}$) is universally applied across the decoder. The vector is defined as:

$$\mathbf{u} = [\alpha, \beta, H_{fixed}] \quad (4.8)$$

This vector contains the critical problem parameters: the makespan weight (α), the operator activation weight (β), and the physical shift capacity (H_{fixed}). To prevent the neural network from relying on implicit memorization of constraints, $\mathbf{u}$ is explicitly concatenated to the input tensors of all three MLP heads, forcing every decision layer to evaluate spatial features through the lens of the global optimization bounds.

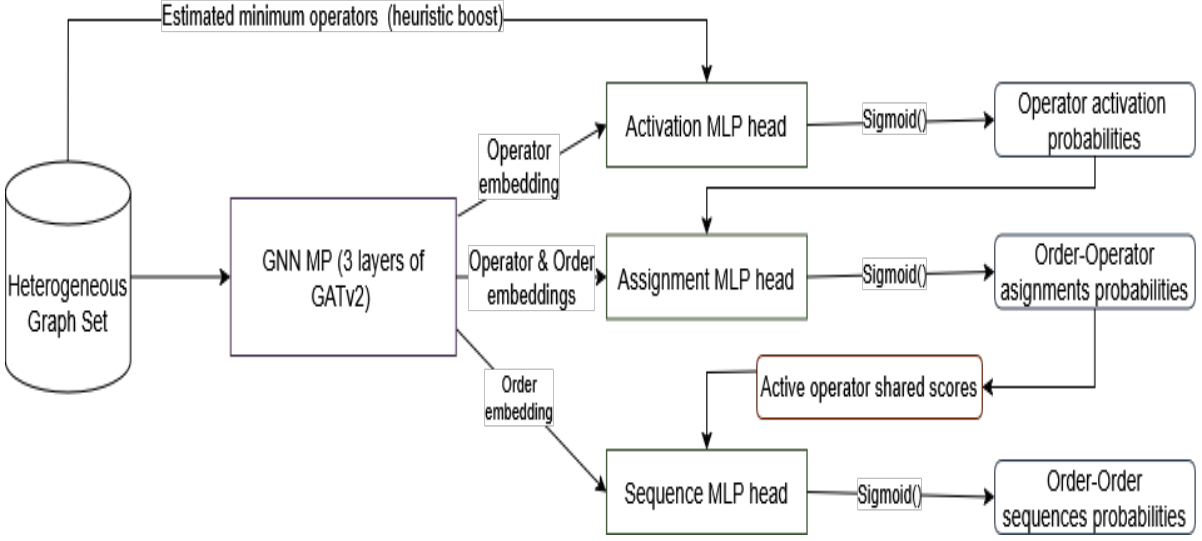


Figure 4.5: One-Shot Architecture of the GNN-MLP Model.

Global Parameter Normalization

To ensure numerical stability across gradients, both α, β need to be normalized to the same scale $\in [0, 1]$ before passing them in u :

$$\alpha = \frac{\alpha}{\alpha + \beta}, \quad \beta = \frac{\beta}{\alpha + \beta} \quad (4.9)$$

For the same reason, H_{fixed} is normalized as the following:

$$H_{fixed} = \frac{H_{fixed}}{H_{fixed_{max}}} \quad (4.10)$$

1. Operator Activation Head

The first head evaluates the *activation policy*. It operates exclusively on the operator node embeddings ($\mathbf{z}_i$) to determine whether activating a specific operator is mathematically necessary to fulfill the workload without violating the H_{fixed} shift limit.

To assist the network in establishing a baseline activation requirement, a heuristic scalar I_{min} (calculated by eq.3.2 in ch.3) is dynamically calculated based on the total processing workload divided by the normalized shift capacity. This feature, alongside the global parameter vector ($\mathbf{u}$), is concatenated to the operator embedding:

$$\hat{P}_{active}^{(i)} = \sigma \left(\text{MLP}_{act} \left([\mathbf{z}_i || \mathbf{u} || I_{min}^{(i)}] \right) \right) \quad (4.11)$$

2. Order Assignment Head

The second head predicts the *assignment policy*, representing the likelihood that order j should be allocated to operator i . This is structured as an edge-classification task. Crucially, the MLP does not simply process the node embeddings; it concatenates the source operator embedding ($\mathbf{z}_i$), the destination order embedding ($\mathbf{z}_j$), the global parameters ($\mathbf{u}$), and the explicit edge features $\mathbf{e}_{ij}$ (travel time and processing time).

To enforce hierarchical logic, this head explicitly receives the decoupled output from the activation head as an input feature:

$$\hat{P}_{assign}^{(ij)} = \sigma \left(\text{MLP}_{assign} \left([\mathbf{z}_i || \mathbf{z}_j || \mathbf{u} || \mathbf{e}_{ij} || \hat{P}_{active}^{(i)} || S_{affinity}^{(ij)}] \right) \right) \quad (4.12)$$

To ensure the assignment logic respects routing geometry and minimizes overlapping routes, an explicit spatial affinity score $S_{affinity}^{(ij)}$ is calculated. First, the cartesian coordinates of order j are normalized and converted into an angular trajectory $A_j \in [0, 1]$ using the four-quadrant inverse tangent:

$$A_j = \frac{\text{atan2}(Y_j/Y_{max}, X_j/X_{max}) + \pi}{2\pi} \quad (4.13)$$

The affinity score is then defined as the absolute difference between this angular trajectory and the operator's normalized monotonic ID:

$$S_{affinity}^{(ij)} = |id_{mono}^{(i)} - A_j| \quad (4.14)$$

This geometrically bounds the network, explicitly biasing the MLP to cluster geographically proximal orders to specific operators.

3. Temporal Sequence Head

The final head predicts the *sequence policy*, determining the temporal routing transitions (i.e., whether order k should immediately follow order j in a sequence). This edge-classification task utilizes the order embeddings ($\mathbf{z}_j, \mathbf{z}_k$), the global parameters ($\mathbf{u}$), and the order-to-order spatial travel time ($\mathbf{e}_{jk}$).

To guarantee that the sequence logic respects the global assignment layout, this head explicitly receives the *active operator shared score* (S_{jk}^{active}), ensuring sequences are only

formed between orders assigned to the same active operator:

$$\hat{P}_{seq}^{(jk)} = \sigma \left(\text{MLP}_{seq} \left([\mathbf{z}_j || \mathbf{z}_k || \mathbf{u} || \mathbf{e}_{jk} || S_{jk}^{active}] \right) \right) \quad (4.15)$$

Addressing Gradient Domination and Multi-Task MLP Conflicts via Explicit Logical Coupling

While a standard multi-task GNN architecture evaluates these parallel heads independently, applying this methodology to a highly constrained scheduling problem introduces a severe phenomenon known as *Gradient Domination*.

During backpropagation, the sequence head evaluates exponentially more edges ($O(|J|^2)$) than the activation head ($O(|I_{max}|)$). Because all three heads backpropagate their losses into the shared GNN trunk, the massive volume of gradients from the sequence and assignment losses overwhelm the activation gradients. Consequently, the shared embeddings optimize exclusively for spatial proximity, entirely ignoring the logical constraints governing operator activation.

Furthermore, evaluating the prediction heads independently ignores the strict hierarchical logic of the scheduling problem: an order can only be assigned to an active operator, and two orders can only be sequenced together if they share the same assigned operator. Joint evaluation is crucial to prevent logical conflicts, such as assigning tasks to inactive operators or, more critically, predicting high-probability sequence routes between orders assigned to different operators.

To resolve this gradient interference, the proposed architecture abandons parallel independence in favor of **explicit logical coupling**.

- **Activation-to-assignment link:** The predicted probability of the activation head ($\hat{P}_{active}^{(i)}$) is explicitly detached from the activation gradient graph and concatenated as an additional input feature into the assignment head. This forces the assignment logic to respect the activation state.
- **Assignment-to-sequence link:** An "active-operator shared probability" score is derived computationally by performing a dot-product between the assignment probability vectors of order j and order k .

For a sequence edge $(j \rightarrow k)$ to be valid, both orders must be assigned to the same operator i . Furthermore, operator i must be classified as active. We formulate this shared active score S_{jk}^{active} as the sum of joint probabilities across all available operators I_{max} :

$$S_{jk}^{active} = \sum_{i \in I_{max}} \left(\hat{P}_{assign}^{(ij)} \cdot \hat{P}_{assign}^{(ik)} \cdot \hat{P}_{active}^{(i)} \right) \quad (4.16)$$

This scalar score is fed directly into the sequence head, ensuring the sequence logic is strictly constrained by the assignment predictions.

By explicitly conditioning the subsequent MLPs on the predictions of the prior heads, the network learns to align the operational rules hierarchically in these heads via shared intermediate representations and joint loss penalties. More importantly, this establishes a direct mathematical pathway for the sequence and assignment losses to flow backward and directly tune the weights of the preceding heads, entirely bypassing the gradient interference within the GNN trunk.

4.6 Multi-Objective Loss Function and Constraint Penalties

Training the multi-head GNN architecture requires a loss formulation capable of balancing three distinct classification objectives—activation, assignment, and sequencing—while strictly enforcing operational feasibility bounds. We utilize a dynamically weighted loss function augmented with a continuous log-barrier capacity penalty.

Multi-Objective Loss Function

To ensure the model respects the global makespan and activation objectives alongside the operational time horizon constraint H_{fixed} , we need to introduce a specialized multi-component loss function. A naive approach to multi-objective learning would be to scale the specific prediction heads by the scheduling objective weights α (makespan) and β (operator activations). However, empirical testing revealed that multiplying individual loss components by these variables causes severe gradient vanishing for the heads associated with the lower weight. To maintain stable training dynamics, the raw losses are

summed with uniform weights, alongside a weighted capacity penalty (e.g., $\lambda_{cap} = 0.01$):

$$\mathcal{L}_{total} = \mathcal{L}_{act} + \mathcal{L}_{assign} + \mathcal{L}_{seq} + \lambda_{cap} \cdot \mathcal{L}_{cap} \quad (4.17)$$

During backpropagation, each individual loss component directly drives the parameter updates for its respective MLP classification head. Meanwhile, the aggregated total loss $\mathcal{L}_{total}$ provides a unified gradient that updates the shared GNN “trunk” (the message-passing layers), ensuring that the underlying spatial node embeddings are globally optimized to satisfy all criteria simultaneously.

Binary Cross-Entropy (BCE) for Decision Probabilities

Because the scheduling problem is mapped to a heterogeneous graph, every routing and assignment step is formulated as a binary node or edge classification task (e.g., 1 if an assignment edge exists, 0 otherwise). The output layers of the three MLP heads utilize sigmoid activations σ to produce continuous probabilities $\hat{y}_n \in [0, 1]$. To evaluate the divergence between these predictions and the discrete ground-truth MIP schedules $y_n \in \{0, 1\}$, we employ BCE.

However, routing graphs inherently suffer from severe *class imbalances*[\[25\]](#)—particularly in the sequence head, where positive edges often account for less than 1% of the target tensor. A standard BCE would cause the model to collapse into a trivial local minimum, predicting an empty sequence matrix of all zeros. To counteract this, we compute a dynamic positive class weight w_{pos} per batch:

$$w_{pos} = \frac{N^{neg}}{\max(N^{pos}, 1)} \quad (4.18)$$

Therefore, we calculate the weight per each example w_n :

$$w_n = \begin{cases} w_{pos} & \text{if } y_n = 1 \\ 1.0 & \text{if } y_n = 0 \end{cases}$$

The dynamically weighted BCE loss for N predictions is formulated as:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{n=1}^N w_n \cdot \left[y_n \log(\hat{y}_n) + (1 - y_n) \log(1 - \hat{y}_n) \right] \quad (4.19)$$

By explicitly scaling the gradient of the minority positive class by w_{pos} , the loss function heavily penalizes missed routing connections and unassigned orders, forcing the network

to actively learn the scheduling topology. This base formulation is applied to compute the individual losses for the activation ($\mathcal{L}_{act}$), assignment ($\mathcal{L}_{assign}$), and sequence ($\mathcal{L}_{seq}$) heads.

Log-Barrier Capacity Penalty For Time Horizon

While the dataset inherently represents feasible schedules, the continuous nature of neural gradients allows intermediate predictions to violate the physical shift constraint (H_{fixed}). To penalize these violations, we estimate the expected continuous workload W_i for each operator i as the dot product of the predicted assignment probabilities and the physical processing times p_j , scaled by a dimensionless heuristic factor γ (e.g., 1.15) to account for expected travel time overhead:

$$W_i = \sum_{j \in J} \left(\hat{P}_{assign}^{(ij)} \cdot p_j \cdot \gamma \right) \quad (4.20)$$

To push the network away from the boundary before an operator exceeds the shift limit, we apply a continuous log-barrier function using the softplus activation over a 5% buffered margin:

$$\mathcal{L}_{cap} = \frac{1}{|I|} \sum_{i \in I} \text{Softplus} \left(W_i - (0.95 \cdot H_{fixed}^{(i)}) \right) \quad (4.21)$$

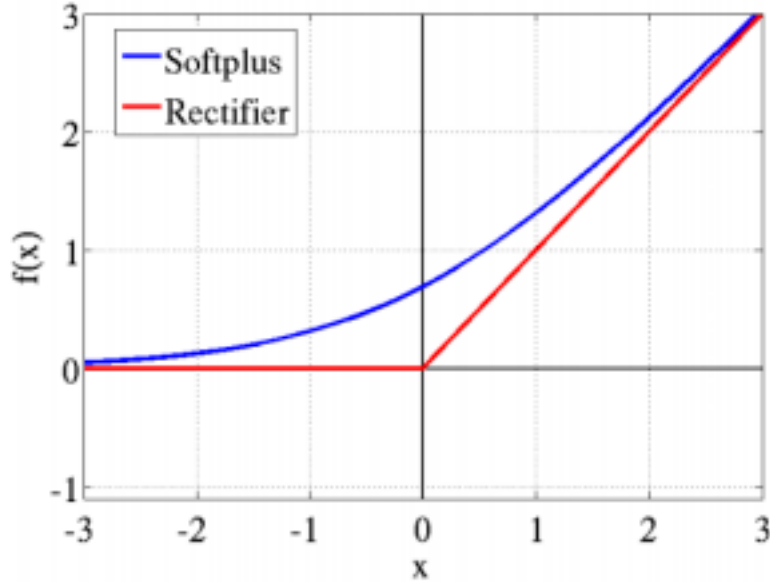


Figure 4.6: ReLU and Softplus Curves.

The choice of softplus over the standard rectified linear unit (ReLU) is deliberate and mathematically necessary for stable optimization (see fig.4.6). A standard ReLU penalty ($\max(0, x)$) evaluates to exactly zero until the $0.95 \cdot H_{fixed}$ boundary is strictly violated,

at which point the gradient becomes abruptly active. This discontinuous gradient flow causes unstable, sudden weight updates during backpropagation. In contrast, softplus ($\ln(1 + e^x)$) is infinitely differentiable and provides a smooth approximation of ReLU[26]. By producing a small, exponentially decaying non-zero gradient even when the workload is slightly below the threshold, softplus acts as a predictive log-barrier. It gently pushes the assignment probabilities away from overloaded operators *before* they violate the hard constraint, leading to smoother convergence and highly robust, capacity-aware routing distributions.

Limitations and Transition to Recurrent Modeling

The proposed one-shot GNN architecture provides a coherent multi-task framework for jointly predicting operator activation, assignment, and sequencing decisions within a single forward pass. This design enables efficient parallel inference (short inference time) and allows the model to learn shared spatial representations across the three decision heads.

Nevertheless, the scheduling problem remains inherently sequential. Each assignment decision modifies the residual state of the system by reducing the remaining operator capacity, altering the set of feasible future assignments, and constraining the admissible sequence structure. In the proposed one-shot architecture, penalizing operator-order assignments is based on the estimated time horizon violations which doesn't guarantee the full awareness of time horizon as it would be in a step-by-step approach.

As a result, a purely one-shot formulation may struggle to fully capture the evolving dependency between current choices and future feasible actions.

This limitation raises several important research questions. Can a single forward pass adequately represent the dynamic evolution of operator workloads and route feasibility? Is the simultaneous prediction of assignment and sequencing sufficient to guarantee structural consistency in more complex instances? Can static node embeddings faithfully encode a decision process in which each action changes the future state of the problem? More fundamentally, would a recurrent formulation better model the iterative nature of schedule construction under fixed time-horizon constraints?

These questions motivate the introduction of a recurrent graph-based architecture in the

next chapter. Rather than treating the schedule as a static object predicted in one step, the recurrent formulation models it as a sequence of interdependent decisions, where the graph state is progressively updated after each partial action.

5. Graph-to-Sequence Neural Network Architecture

As established in the previous chapter, a fundamental limitation of the one-shot GNN architecture is its inability to dynamically update spatial representations in response to sequential decision-making. In physical routing and scheduling environments, every assignment made by the solver irreversibly alters the future state of the problem: remaining operator capacity decreases, geographic coordinates shift, and the pool of available orders shrinks.

In this chapter, we introduce how the graph-to-sequence (Graph2Seq) neural network which takes into account these observations by having a step-by-step context awareness, in order to imitate each single decision taken by the MIP solver.

5.1 Proposed Architecture: Node-Aware Graph2Seq with Coupled Multi-Task MLP Heads

To explicitly model the evolving decision landscape, we transition to an autoregressive *Graph2Seq* encoder-decoder architecture. In modern neural combinatorial optimization, Graph2Seq models are explicitly designed to map complex, non-Euclidean graph structures into ordered output sequences [27]. This paradigm has proven highly effective in

VRP, where static graph embeddings are dynamically decoded into sequential routes using recurrent mechanisms [28].

By adopting a Graph2Seq formulation, we decompose the scheduling problem into an sequential series of localized state updates. Rather than predicting the entire routing graph at once, the network incrementally builds the schedule step-by-step. To achieve this efficiently, the proposed architecture (see fig. 5.1) is logically separated into two distinct components: a *static spatial encoder* based on GATv2 that processes the baseline warehouse layout just once, and a *dynamic recurrent decoder* powered by gated recurrent unit (GRU) that iteratively updates operator states and predicts the next optimal assignment.

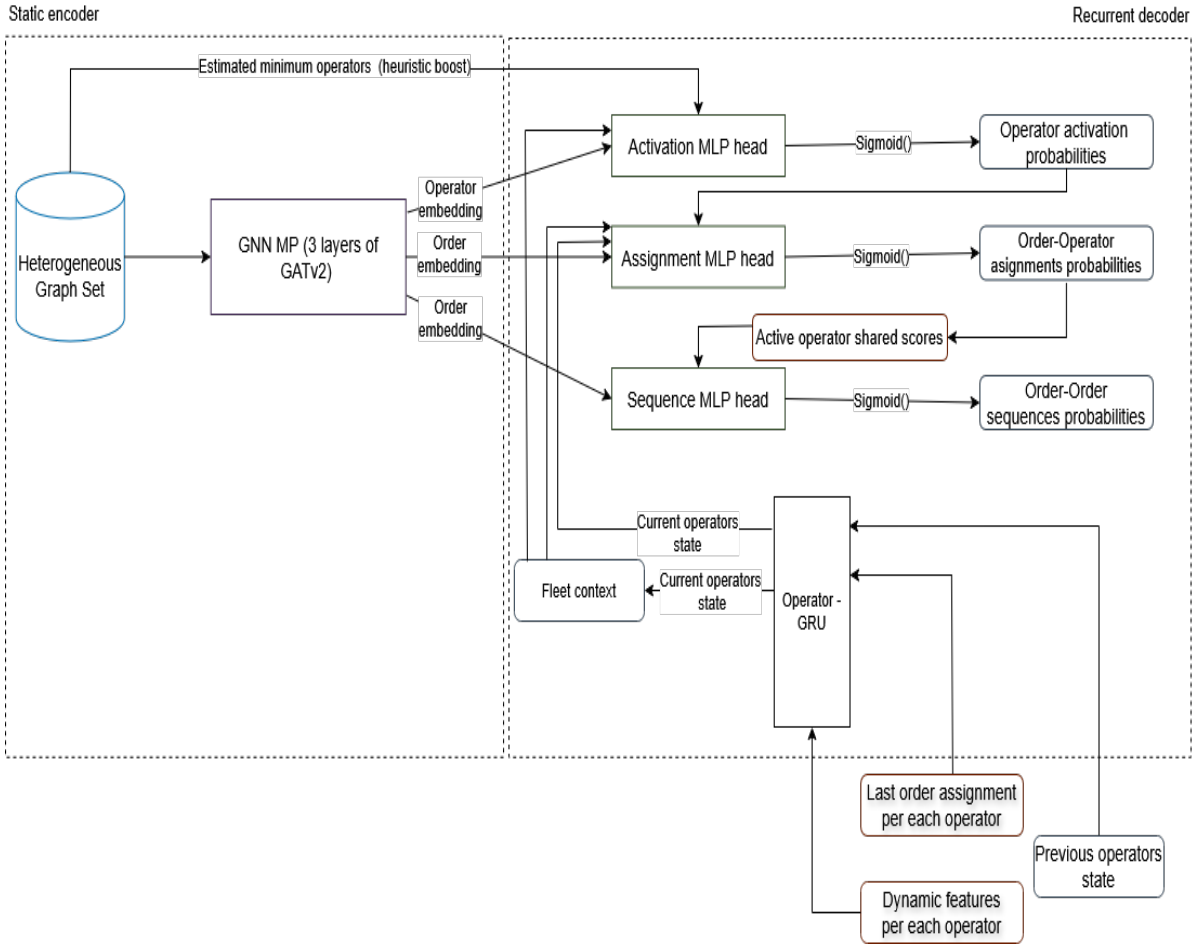


Figure 5.1: Recurrent Architecture of the Graph2Seq Model.

Unlike traditional Graph2Seq models that rely on a global graph pooling bottleneck, our architecture employs a **node-aware** approach. We preserve the structural identity of the nodes by passing their GATv2 embeddings directly to a recurrent decoder, allowing

the model to make sequence decisions based on local node information rather than a global summary.

Operator State Update via GRU

At the beginning of each step t , the hidden state of operator i ($\mathbf{h}_i^{(t)}$) must be updated to reflect the completion of its previous assignment. To model this state transition, we utilize a gated recurrent unit (GRU) cell [29] (see fig. 5.2) rather than a standard recurrent neural network (RNN) or a long short-term memory (LSTM) cell.

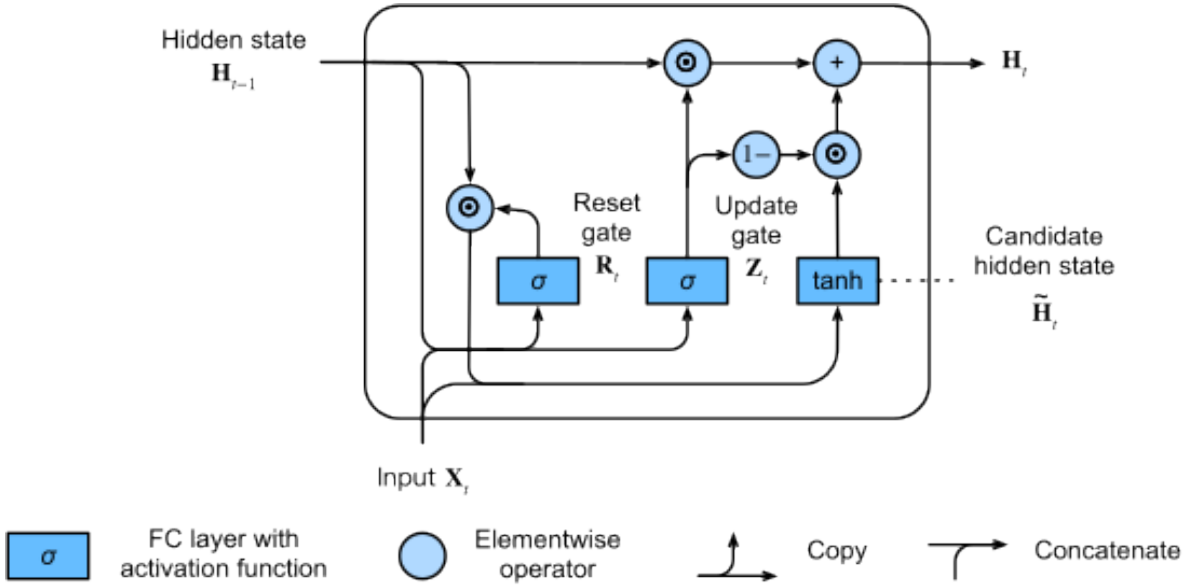


Figure 5.2: Gated Recurrent Units (GRU) Architecture.

The choice of the GRU is driven by both computational efficiency and architectural compatibility with combinatorial routing graphs:

- **Mitigation of vanishing gradients:** Standard RNNs struggle to propagate gradients across long sequences, making them incapable of tracking an operator’s capacity across a long routing episode. Like LSTMs, GRUs utilize a gating mechanism to selectively preserve or discard information, preventing gradient decay over time.
- **Computational efficiency and parameter reduction:** While LSTMs rely on three distinct gates (input, forget, and output) and maintain a separate internal

cell state, GRUs merge the cell and hidden states and rely on only two gates (update and reset) [30]. This streamlined architecture reduces the number of tensor operations and trainable parameters by approximately 25%, significantly accelerating training and inference without sacrificing expressive power [31].

- **Direct state exposure:** Because the GRU does not possess a separate, gated internal cell memory like an LSTM, its entire hidden state is fully exposed at every time step t . This is highly advantageous for our specific routing formulation, as the downstream prediction heads (activation, assignment, sequence) might require direct, unobstructed access to the operator’s exact current state ($\mathbf{h}_i^{(t)}$) to make immediate spatial routing decisions.

The input to the GRU cell consists of the static embedding of the last completed order ($\mathbf{z}_{j_i}^{static}$) concatenated with the operator’s current dynamic features $\mathbf{d}_i^{(t)}$ (e.g., remaining capacity, current X/Y coordinates):

$$\mathbf{h}_i^{(t)} = \text{GRUCell}\left([\mathbf{z}_{j_i}^{static} \parallel \mathbf{d}_i^{(t)}], \mathbf{h}_i^{(t-1)}\right) \quad (5.1)$$

For $t = 0$, the hidden state is initialized using the static operator embedding generated by the GATv2 encoder ($\mathbf{h}_i^{(0)} = \mathbf{z}_i^{static}$), and the last order embedding is initialized as a zero vector.

5.2 The Static Spatial Encoder

The encoder is executed exactly once at the beginning of the scheduling episode ($t = 0$). Its purpose is to process the physical characteristics of the warehouse—such as absolute coordinates, dimensions, and baseline travel distances—and map them into a rich, static continuous embedding space.

Let the initial physical features of the operators and orders be denoted as $\mathbf{x}_i^{(0)}$ and $\mathbf{x}_j^{(0)}$ respectively. These features are projected through initial linear layers and passed into a multi-layer GATv2 augmented with heterogeneous convolutions. The output of the encoder is a set of static structural embeddings for the operators ($\mathbf{z}_i^{static}$) and the orders ($\mathbf{z}_j^{static}$):

$$[\mathbf{z}_i^{static}, \mathbf{z}_j^{static}] = \text{Encoder_GNN}(\mathbf{x}_i^{(0)}, \mathbf{x}_j^{(0)}, E) \quad (5.2)$$

By caching these static embeddings, the model drastically reduces computational overhead. The computationally expensive message-passing layers are not re-evaluated during the recurrent loop; rather, the dynamic state updates are handled exclusively by lightweight, localized recurrent cells.

5.3 The Dynamic Recurrent Decoder

The core of the Graph2Seq is the dynamic recurrent decoder, which operates sequentially at each decoding step t . To track the evolving status of each operator—specifically their current location, accumulated workload, and remaining shift capacity (H_{fixed})—we introduce a localized GRU cell for the operator nodes.

Operator State Update via GRU

At the beginning of each step t , the hidden state of operator i ($\mathbf{h}_i^{(t)}$) must be updated to reflect the completion of its previous assignment. The input to the GRU cell consists of the static embedding of the last completed order ($\mathbf{z}_{j_{last}}^{static}$) concatenated with the operator's current dynamic features $\mathbf{d}_i^{(t)}$ (e.g., remaining capacity per operator i , current

X/Y coordinates):

$$\mathbf{h}_i^{(t)} = \text{GRUCell}\left([\mathbf{z}_{iast}^{static} \parallel \mathbf{d}_i^{(t)}], \mathbf{h}_i^{(t-1)}\right) \quad (5.3)$$

For $t = 0$, the hidden state is initialized using the static operator embedding $\mathbf{h}_i^{(0)} = \mathbf{z}_i^{static}$, and the last order embedding is initialized as a zero vector.

Global Fleet Context Integration

Because scheduling requires heavy coordination between operators to optimize the make-span and balance the total flow time, individual operators cannot make assignment decisions in total isolation. To provide global situational awareness, the updated hidden states of all available operators are aggregated using a mean-pooling operation to form a *global fleet context* vector $\mathbf{c}_{fleet}^{(t)}$:

$$\mathbf{c}_{fleet}^{(t)} = \frac{1}{|I_{max}|} \sum_{i \in I_{max}} \mathbf{h}_i^{(t)} \quad (5.4)$$

This context vector is broadcasted back and concatenated into the input of the downstream prediction heads, allowing the network to evaluate whether assigning an order to operator i is globally optimal relative to the current status of the rest of the fleet.

5.4 Iterative Multi-Task Prediction Heads

Using the updated recurrent states and the global fleet context, the decoder utilizes three specialized MLP heads to evaluate the next optimal scheduling action based on the current context.

1. Activation Head

The activation head determines whether an operator i should remain active or be idled. The prediction relies on the global fleet context $\mathbf{c}_{fleet}^{(t)}$ and the current operator embedding $\mathbf{h}_i^{(t)}$ in addition to the aforementioned global parameters (see. eq.4.11 of ch.4):

$$\hat{P}_{active}^{(i,t)} = \sigma\left(\text{MLP}_{act}\left([\mathbf{h}_i^{(t)} \parallel \mathbf{c}_{fleet}^{(t)} \parallel \mathbf{u} \parallel I_{min}^{(i)}]\right)\right) \quad (5.5)$$

Architectural Alternatives for Activation Tracking

From an architectural standpoint, an alternative approach would be to explicitly track the operator's activation state by introducing a dynamic binary feature (e.g., $is_active_t = 1.0$ if the operator has been assigned at least one order in prior steps, and 0.0 otherwise) directly into the operator's dynamic state vector $\mathbf{d}_i^{(t)}$.

Providing this explicit flag would relieve the MLP from repeatedly re-evaluating the activation necessity from scratch, effectively saving computational effort by locking in the "active" status once an assignment is made. However, we deliberately maintain the continuous prediction mechanism at every recurrent step for two critical reasons:

- **Architectural coherency and head coupling:** Preserving the continuous activation head ensures structural consistency with the baseline one-shot architecture (see fig. 4.5 in ch. 4). In both models, the prediction heads are explicitly coupled: the continuous confidence score from the activation head ($\hat{P}_{active}^{(i,t)}$) is concatenated directly into the input of the assignment head. Replacing this differentiable probability with a rigid binary flag would sever this neural coupling, complicating direct performance comparisons between the two architectures.
- **Auxiliary gradient signal:** Forcing the network to continuously re-evaluate the activation state acts as a powerful auxiliary objective during training. It ensures that the recurrent hidden state $\mathbf{h}_i^{(t)}$ receives a dense, persistent gradient signal regarding the balance between remaining fleet capacity and unassigned workload, leading to a much richer spatial representation.

2. Assignment Head

The assignment head evaluates the compatibility between the dynamic operator state $\mathbf{h}_i^{(t)}$ and the static order embedding $\mathbf{z}_j^{static}$ in addition to other known parameters (see eq. 4.12 in ch. 4):

$$\hat{P}_{assign}^{(ij,t)} = \sigma \left(\text{MLP}_{assign} \left([\mathbf{h}_i^{(t)} \parallel \mathbf{c}_{fleet}^{(t)} \parallel \mathbf{z}_j^{static} \parallel \mathbf{u} \parallel \hat{P}_{active}^{(i,t)} \parallel S_{affinity}^{(ij)}] \right) \right) \quad (5.6)$$

3. Sequence Head

The sequence head evaluates the transition probability from the previously assigned order j to the newly considered order k regarding their embeddings: $\mathbf{z}_j^{static}, \mathbf{z}_k^{static}$, in addition to other known parameters (see eq.4.15 in ch.4).

$$\hat{P}_{seq}^{(jk,t)} = \sigma \left(\text{MLP}_{seq} \left([\mathbf{z}_j^{static} \parallel \mathbf{z}_k^{static} \parallel \mathbf{u} \parallel S_{jk}^{active}] \right) \right) \quad (5.7)$$

Dynamic Masking of Completed Actions

Because the model evaluates sequentially, it must not re-assign orders that have already been completed in previous steps. Before the output probabilities are returned, a deterministic boolean masking operation (see fig. 5.3) is applied. If an order j is flagged as *assigned*, its corresponding assignment and sequence probabilities are forced to 0.0, strictly pruning the decision space as the routing episode progresses toward completion.

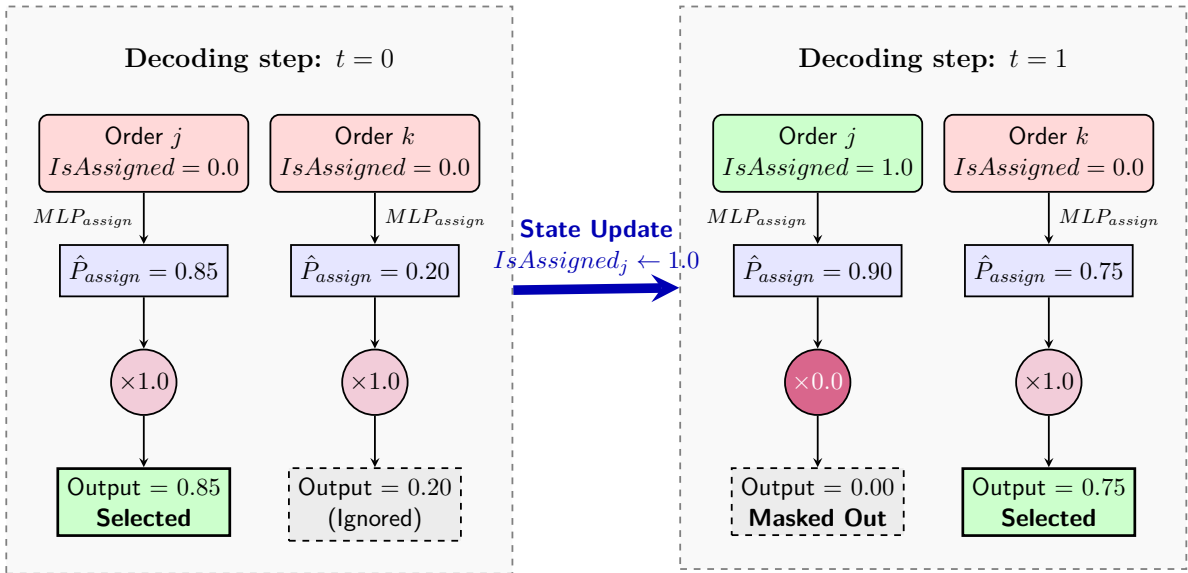


Figure 5.3: Representation of the Dynamic Masking Mechanism across Consecutive Decoding Steps.

Implicit Memory and Route Trajectory

It is important to note that at any given step t , the input to the GRU explicitly contains only the feature embedding of the *most recently* assigned order, rather than a cumulative list of all past assignments.

Despite this localized input, the network is fully capable of tracking the operator’s entire historical route. This is achieved through the fundamental mechanics of the GRU’s hidden state ($\mathbf{h}_i^{(t)}$). Rather than relying on an explicit external memory bank, the GRU’s internal update gate systematically blends the new spatial information from step t into the continuous hidden state carried over from step $t - 1$. As a result, the hidden state acts as a compressed, continuous topological footprint. By step $t = k$, $\mathbf{h}_i^{(k)}$ implicitly encodes the geometric trajectory and accumulated workload of all k previously assigned orders, maintaining a strict $\mathcal{O}(1)$ memory footprint per operator regardless of the length of the scheduling sequence.

5.5 Autoregressive Training via Teacher Forcing and BPTT

Training an autoregressive Graph2Seq model requires a fundamentally different gradient calculation strategy than a one-shot architecture. Because the routing predictions are generated sequentially, the hidden state of the GRU at step t mathematically depends on the hidden state from step $t - 1$. To optimize the network parameters across these sequential dependencies, we employ backpropagation through time (BPTT) [32].

In standard BPTT, the recurrent network is conceptually “unrolled” across all time steps (fig. 5.4), and gradients are propagated backward from the final step to the first. However, allowing an untrained model to autoregressively decode an entire warehouse schedule from scratch results in severe exposure bias: early assignment errors cascade, pushing the GRU hidden states completely off the optimal trajectory, rendering the final sequence gradients meaningless.

To stabilize training, we implement **teacher forcing** training. Rather than allowing the model to freely predict and sequence all assignments simultaneously—which could lead to cascading errors and infeasible states—the true optimal assignment sequences derived

from the exact MIP solver are broken down into non-overlapping ”chunks” (explained in sec.6.2 of the next chapter). During training, the model is evaluated sequentially across these chunks. This mechanism forces the network to **imitate** the exact step-by-step resolution path taken by the expert solver, incrementally updating the dynamic features at each sequential step.

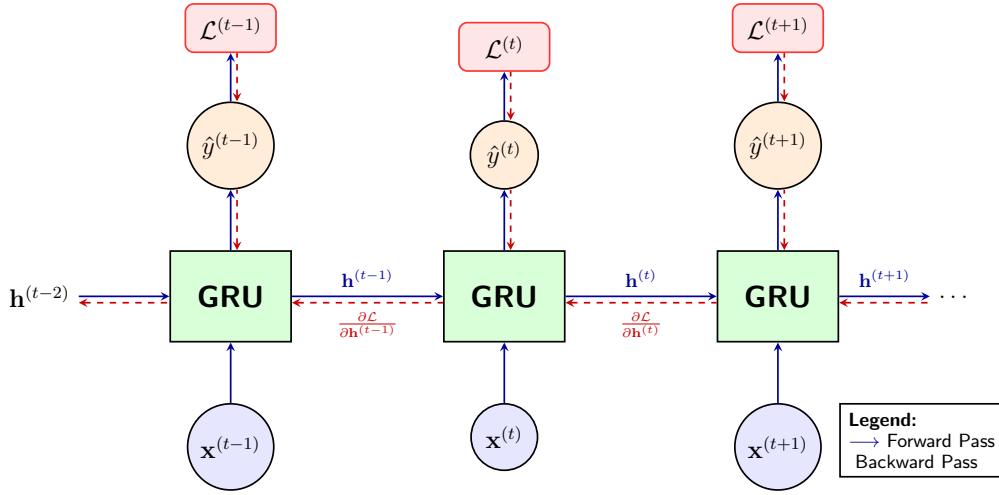


Figure 5.4: Unrolled Representation of a GRU over Time to Illustrate BPTT.

Step-wise Loss Evaluation and Target Masking

At each decoding step t , the recurrent decoder produces raw probabilities across the three prediction heads ($\hat{P}_{active}^{(t)}$, $\hat{P}_{assign}^{(t)}$, $\hat{P}_{seq}^{(t)}$). We maintain the exact dynamically weighted loss formulation (see eq.4.17) introduced in the previous chapter, preserving the capacity penalty and the global parameter normalizations.

However, because the sequence is evaluated iteratively, the global ground truth targets ($y \in \{0, 1\}$) must be dynamically masked to reflect the temporal state. If the BCE loss were calculated against the full, static ground-truth matrix at every step t , the network would be heavily penalized for failing to predict future assignments that it is not yet supposed to make.

To rectify this, we dynamically mask the ground truth targets at each step. For a given time step t , if the optimal MIP schedule assigns operator i to order j , the specific target

$\mathbf{y}_{assign}^{(ij,t)}$ is unmasked (1.0), while other ground-truth assignments are zeroed out:

$$\mathbf{y}_{assign,masked}^{(ij,t)} = \begin{cases} \mathbf{y}_{assign}^{(ij)} & \text{if edge } (i,j) \text{ is the active target at step } t \\ 0 & \text{otherwise} \end{cases} \quad (5.8)$$

This exact masking logic is mirrored for the sequence targets ($\mathbf{y}_{seq}^{(t)}$). The total step-wise loss $\mathcal{L}^{(t)}$ is then calculated as the weighted BCE between the unmasked targets and the current neural predictions.

By accumulating $\mathcal{L}^{(t)}$ across all chunks within the routing episode and triggering BPTT, the network learns to accurately predict the immediate next action while correctly ignoring previously resolved tasks and those scheduled for future steps.

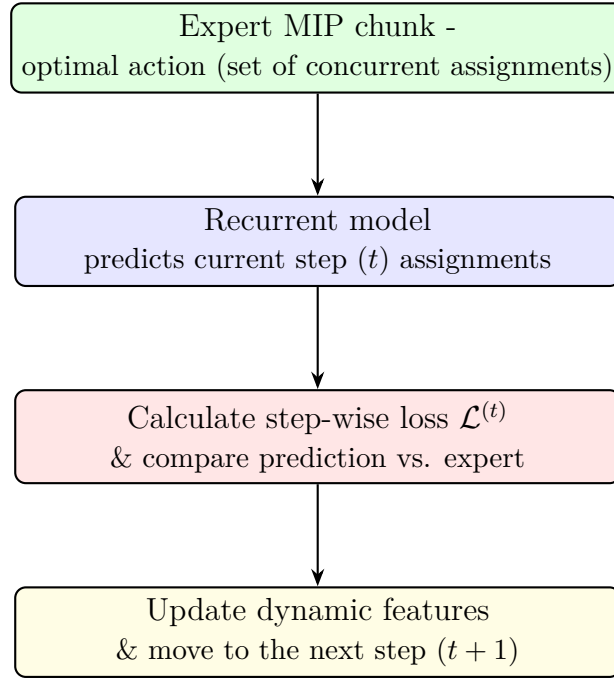


Figure 5.5: Teacher Forcing Pipeline.

6. Inference and Scheduling Methods

In neural combinatorial optimization, solving a routing problem extends beyond predicting accurate continuous probabilities. The core challenge lies in decoding the multi-dimensional output tensors ($\hat{P}_{act}, \hat{P}_{assign}, \hat{P}_{seq}$) generated by the neural network into a discrete, mathematically valid scheduling matrix ($\mathbf{y} \in \{0, 1\}$). This decoding process must strictly adhere to the physical constraints of the warehouse environment, ensuring that no operator exceeds their maximum shift capacity (h_{fixed}) and that no order is assigned to multiple operators simultaneously.

This chapter details the exact decoding algorithms required to transition from neural predictions to feasible physical schedules. We first analyze the one-shot GNN model (presented in ch.4) and the post-processing methods needed to decode the output probabilities into a valid schedule. We then formalize the autoregressive inference algorithm governing the Graph2Seq architecture (presented in ch.5). Finally, we establish the algorithmic baselines required to benchmark the neural network’s generalization capabilities on large-scale warehouse topologies where exact optimization bounds are computationally intractable.

6.1 One-Shot GNN Inference and Neural Decoding

As established in chapter 4, the one-shot GNN baseline processes the entire heterogeneous graph simultaneously, predicting the total scheduling matrix in a single forward pass. Without an internal sequential memory mechanism, this approach requires specialized post-hoc decoding to select each operator’s assignments enforcing constraint validity, in order to have a final valid schedule.

One-Shot GNN Model and Greedy Decoder

To transition from static probabilities to a valid schedule, the greedy decoder cascades the decisions through a rigid, three-stage hierarchy (fig.6.1):

1. **Operator activation:** The decoder first determines the active fleet. We have two possible strategies that can be used to estimate the pool of active operators:
 - Calculate the estimated number of operators to activate based on an activation threshold ($T_{activation}$) in which any operator i that has $\hat{P}_{active}^{(i)} > T_{activation}$ is considered an active operator.
The threshold $T_{activation}$ can be tuned through the precision-recall curve (see sec.8.5 in ch.8) considering "active" and "inactive" as positive and negative classes respectively.
 - Calculate the theoretical minimum number of operators (k) required to process all orders (see. eq.3.2 in ch.3). The k operators with the highest activation probabilities ($\hat{P}_{active}$) are flagged as active, while the rest remain inactive. Then increment k carefully if the current k is considered "unfeasible".

Thresholding can be an advantage if we want to have a confidence in the estimated number of operators by the activation head, it might be faster than calculating the theoretical minimum k then incrementing it whenever needed. However, since

we're trying to extremely minimize the activation rate, we choose the active pool of operators I to start with the *theoretical minimum number* of operators (k).

2. **Global assignment:** With the active fleet established, the algorithm groups the assignment probabilities ($\hat{P}_{assign}$) into quantized bins (rounded to 3 decimals) to handle tied predictions fairly. It iterates through all unassigned orders, greedily assigning each order to the active operator with the highest assignment probability. To prevent severe imbalances, the algorithm utilizes a cascading tie-breaker: if probabilities are equal, it prefers the operator with the highest current load to promote dense, localized spatial clusters.
3. **Local sequence routing:** Once the orders are clustered to specific operators, the decoder relies on the sequence probabilities ($\hat{P}_{seq}$) to establish the temporal routes. For a given operator, the decoder starts at the base and recursively selects the unvisited order with the highest sequence probability from the current node. Before finalizing a step, the physical constraints are rigidly checked: the accumulated travel time and processing time must not exceed h_{fixed} . If a sequence edge violates the shift horizon, it is discarded, and the decoder falls back to the next highest probable edge.

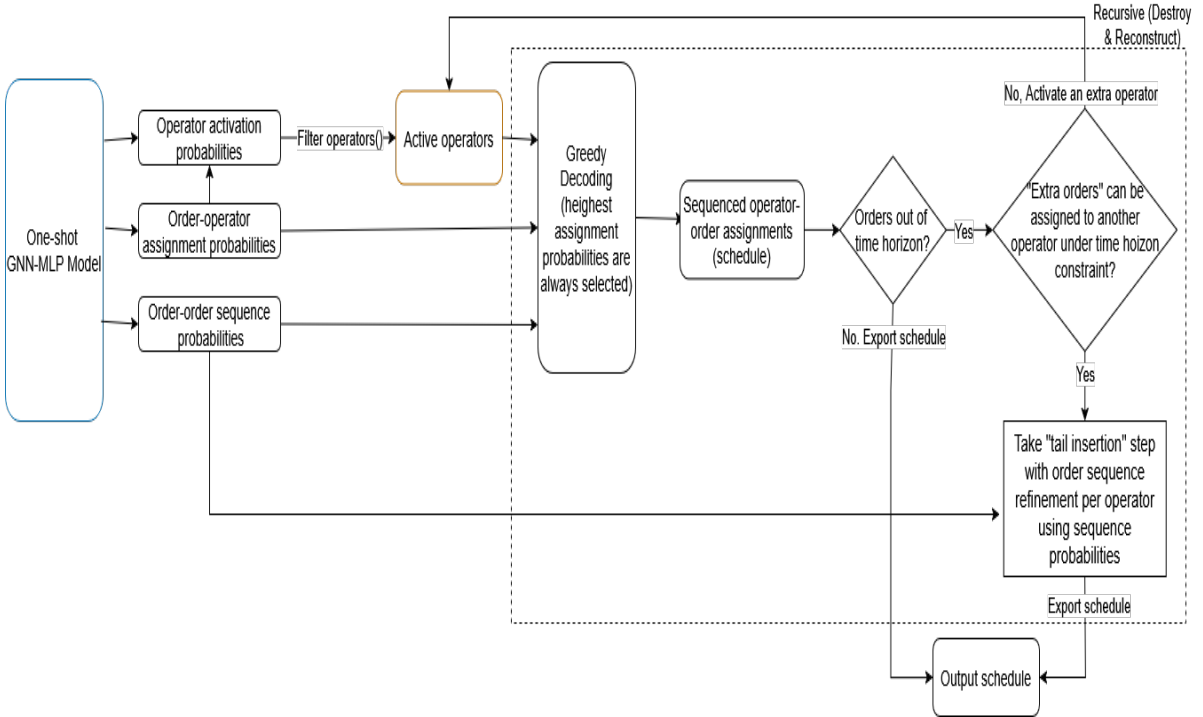


Figure 6.1: One-Shot GNN Model and Greedy Decoding Procedure

Limitations and Tail-Insertion Pass: A core limitation of this static greedy decoding approach is that if an operator’s capacity is unexpectedly exhausted during stage 3 (sequence routing), any remaining orders assigned to them during stage 2 are ”rejected”. To ensure schedule completeness, the decoder incorporates a post-hoc *tail-insertion* pass. Rejected orders are returned to a global pool and greedily appended to the routes of other active operators, provided they possess remaining temporal capacity. These modified routes are subsequently reshuffled using the sequence probabilities associated with the corresponding operator. If all active operators are operating at full capacity, the algorithm activates an *additional* standby operator from the inactive pool and completely reconstructs the schedule.

The **trade-off** in this strategy is that while the neural training and inference times are extremely short, the subsequent post-processing decoding step incurs a high computational overhead. Because the model outputs all assignment probabilities simultaneously without enforcing sequential dependencies, the decoder must evaluate the entire probability matrix at once to resolve constraints and construct a globally feasible schedule.

loa 1: Simplified pseudocode of one-shot GNN greedy decoding

Require: Probabilities $\hat{P}_{act}, \hat{P}_{assign}, \hat{P}_{seq}$, Operators I , Orders J , Time capacity h_{fixed}

Ensure: Routing schedule S

- 1: // **Stage 1: Operator activation**
- 2: $k \leftarrow$ Minimum operators needed to process all J , or a custom required number.
- 3: $I_{active} \leftarrow$ Top k operators $i \in I$ with highest $\hat{P}_{act}(i)$
- 4: // **Stage 2: Global assignment**
- 5: Initialize clusters $C_i \leftarrow \emptyset$ for all $i \in I_{active}$
- 6: **for** each order $j \in J$ **do**
- 7: $i^* \leftarrow \operatorname{argmax}_{i \in I_{active}} \hat{P}_{assign}(i, j)$
- 8: $C_{i^*} \text{.add}(j)$
- 9: **end for**
- 10: // **Stage 3: Local sequence routing**
- 11: $J_{rejected} \leftarrow \emptyset$
- 12: **for** each operator $i \in I_{active}$ **do**
- 13: $curr \leftarrow \text{Base}, \quad time \leftarrow 0$
- 14: **while** C_i is not empty **do**

```

15:    $j^* \leftarrow \operatorname{argmax}_{j \in C_i} \hat{P}_{seq}(curr, j)$ 
16:    $finish \leftarrow time + T_{travel}(curr, j^*) + T_{proc}(j^*)$ 
17:   if  $finish \leq h_{fixed}$  then
18:      $S_i.append(j^*), \quad time \leftarrow finish$ 
19:      $curr \leftarrow j^*, \quad C_i.remove(j^*)$ 
20:   else
21:      $J_{rejected}.add(C_i)$  {full capacity, reject remaining}
22:     break
23:   end if
24: end while
25: end for
26: // Stage 4: Post-hoc tail-insertion with sequence refinement
27: Greedily append  $j \in J_{rejected}$  to routes  $S_i$  with spare capacity and try to insert/distribute them in the tail of sequence of any active operator  $i$ , with a sequence refinement step based on  $\hat{P}_{i_{seq}}$ .
28: if unsatisfied constraints then
29:   Restart from stage 1 with  $k \leftarrow k + 1$ 
30: end if
31: return  $S$ 

```

6.2 Autoregressive Graph2Seq Inference

Unlike the one-shot model, the proposed Graph2Seq architecture relies on dynamic, state-aware autoregressive decoding. By updating the internal hidden state of the fleet after routing decisions, the recurrent neural network continuously adapts to the evolving physical topology of the warehouse [33], recalculating the assignment probabilities ($\hat{P}_{assign}$) dynamically

Iterative Greedy Sequence Decoder with Non-Overlapped Chunking

The inference algorithm governing the recurrent model is designed to heavily prioritize *minimal operator activation* while maintaining strict capacity feasibility. To overcome the high computational latency traditionally associated with autoregressive decoding (where the network must execute a full forward pass for every single order), we developed a highly optimized *non-overlapped chunked decoding* heuristic.

The algorithm operates through the following strict hierarchy (fig.6.2):

1. **Activation first:** Similarly to stage 1 of the one-shot GNN greedy decoder, the decoder calculates the absolute theoretical minimum of operators required to process the batch (K). The GNN’s activation head ($\hat{P}_{active}$) is used strictly to rank and lock the active fleet to these top K operators. All other operators are entirely masked from the solver space.
2. **Temperature-scaled chunking assignment:** Instead of processing assignments sequentially, the decoder pulls forward a ”chunk” of highly confident assignments simultaneously. However, on large-scale warehouse graphs, the GNN often becomes overconfident, saturating the probabilities heavily toward 0.0 or 1.0. This saturation destroys the continuous gradient needed to sort and rank the best edges accurately.

To resolve this, the algorithm dynamically calculates a temperature parameter $\tau = 1.0 + (\alpha \cdot |J|)$ to calibrate the network’s confidence [34]. Because the model natively outputs normalized sigmoid probabilities ($\hat{P}_{assign}$) for stable gradient balancing across multiple heads, the decoder safely approximates the pre-activation logits via an inverse sigmoid transform:

$$Z = \ln(\hat{P}_{assign}/(1 - \hat{P}_{assign})). \quad (6.1)$$

The logits are then scaled and re-activated: $\hat{P}_{scaled} = \sigma(Z/\tau)$.

This scaling softens the distribution, allowing the decoder to correctly rank the most confident edges. The top assignments are then filtered to form a strictly *non-overlapping* chunk, ensuring that within a single forward-pass iteration, each operator receives at most one new order.

3. **Non-overlapped chunking:** At each decoding iteration, the assignment candidates are sorted by their predicted probabilities and filtered into a chunk such that

no operator and no order appear more than once in the same chunk. Formally, if (i, j) denotes an operator–order pair, then a candidate is accepted only if $i \notin I_{\text{seen}}$ and $j \notin J_{\text{seen}}$, where I_{seen} and J_{seen} track the operators and orders already selected in the current iteration. This greedy filtering strategy follows the general principle of conflict-free scheduling and conflict-based planning, where simultaneous assignments are accepted only if they do not introduce contention among agents or tasks [35], before the recurrent state is updated.

4. **Recurrent feature and hidden state update:** Once the non-overlapping chunk of orders is physically verified against the time horizon (h_{fixed}) and assigned, the temporal features of the graph are updated [36]. The selected operators' remaining capacities are decreased according to the incurred travel and processing times:

$$h_{\text{rem}}^{(t+1)} = h_{\text{rem}}^{(t)} - T_{\text{travel}} - T_{\text{proc}} \quad (6.2)$$

and their coordinates are moved to the locations of the completed orders. These updated features are then passed, together with the previous hidden state $h_i^{(t)}$ and the embedding of the last completed order $z_j^{(t-1)}$, into the recurrent module. The GRU computes a new hidden embedding for the operators (see. eq.5.3 in ch.5), which is returned to the decoder and reused in the next autoregressive iteration to repredict the assignment probabilities for the remaining unassigned orders.

5. **Recursive active operators' pool expansion:** The algorithm attempts to schedule the entire batch using strictly K operators. If an iteration yields no valid assignments (i.e., it is mathematically impossible to fit the remaining orders within the capacities of the locked pool), the decoder discards the current schedule and recursively restarts the entire inference loop, incrementing the active pool by one ($K + 1$).

By processing mutually exclusive routing decisions in parallel chunks, this algorithm preserves the dynamic state-awareness of an autoregressive GRU while drastically accelerating inference time on large-scale datasets.

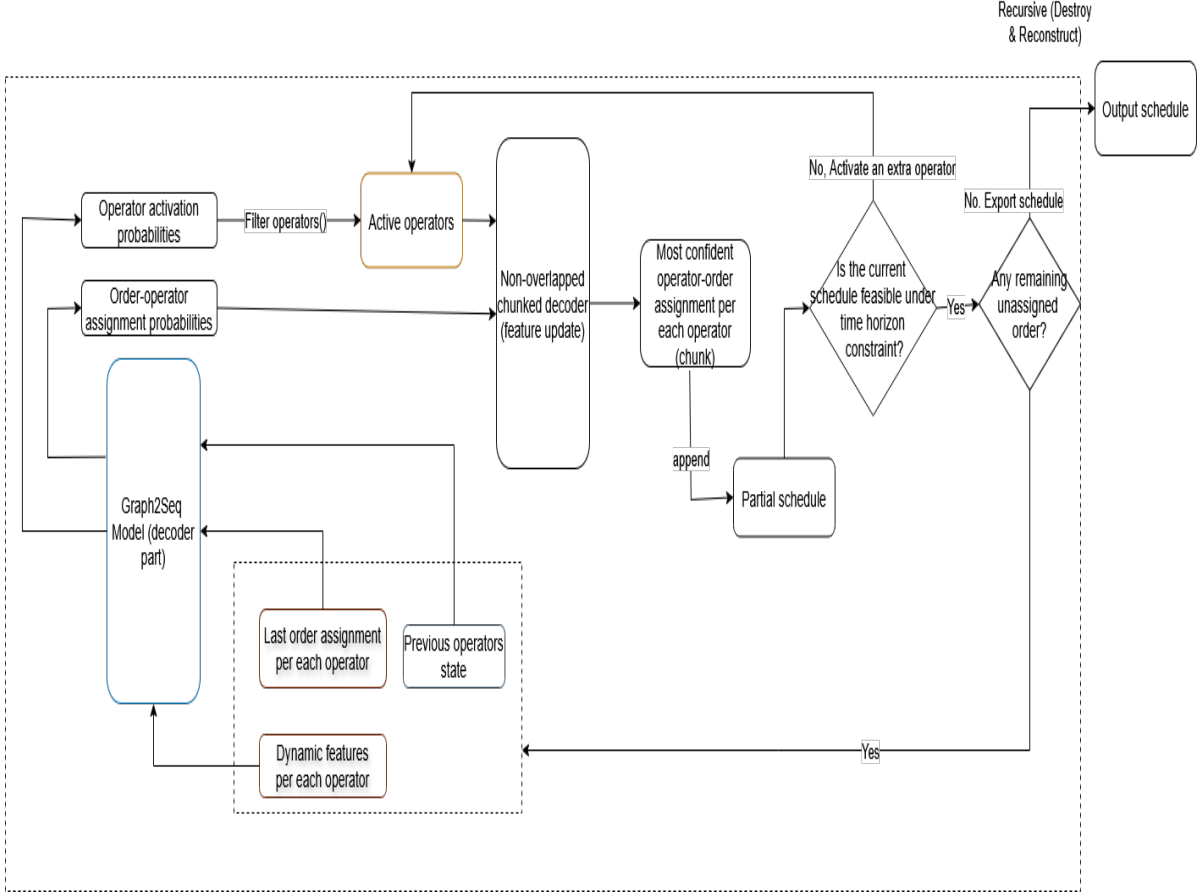


Figure 6.2: Graph2Seq Model and Iterative Greedy Sequence Decoding Procedure

loa 2: Simplified pseudocode of Graph2Seq iterative greedy sequence decoding (non-overlapped chunking)

Require: Graph G , Operators I , Orders J , Time capacity h_{fixed} , Temperature τ

Ensure: Routing schedule S

- 1: $k \leftarrow$ minimum number of operators required, or a custom target
- 2: $I_{active} \leftarrow$ top k operators $i \in I$ with highest $\hat{P}_{act}(i)$
- 3: $J_{unassigned} \leftarrow J$
- 4: $S \leftarrow$ empty routes for all $i \in I_{active}$
- 5: **while** $J_{unassigned} \neq \emptyset$ **do**
- 6: $\hat{P}_{assign} \leftarrow$ ForwardPass(node embeddings, hidden state, masked graph)
- 7: $\hat{P}_{scaled} \leftarrow$ ApplyTemperatureScaling($\hat{P}_{assign}, \tau$)
- 8: // **Extract non-overlapped chunk**

```

9:    $Edges \leftarrow \text{sort } \hat{P}_{scaled}(i, j) \text{ in descending order for } i \in I_{active}, j \in J_{unassigned}$ 
10:   $Chunk \leftarrow \emptyset, I_{seen} \leftarrow \emptyset, J_{seen} \leftarrow \emptyset$ 
11:  for each  $(i, j)$  in  $Edges$  do
12:    if  $i \notin I_{seen}$  and  $j \notin J_{seen}$  then
13:      if  $\text{Feasible}(i, j, S, h_{fixed})$  then
14:         $Chunk \leftarrow Chunk \cup \{(i, j)\}$ 
15:         $I_{seen} \leftarrow I_{seen} \cup \{i\}$ 
16:         $J_{seen} \leftarrow J_{seen} \cup \{j\}$ 
17:      end if
18:    end if
19:  end for
20:  if  $Chunk = \emptyset$  then
21:    return  $\text{AutoregressiveDecoder}(G, I, J, k + 1)$ 
22:  end if
23:  // Route and update state
24:  for each  $(i, j) \in Chunk$  do
25:     $S_i \leftarrow S_i \cup \{j\}$ 
26:     $J_{unassigned} \leftarrow J_{unassigned} \setminus \{j\}$ 
27:    Update operator  $i$  capacity and location
28:    Mask order  $j$ 
29:  end for
30: end while
31: return  $S$ 

```

6.3 Algorithmic Baselines for Large-Scale Benchmarking

To validate the generalization of the proposed neural models, its decoded schedules must be benchmarked against reliable mathematical bounds.

The Computational Limits of Exact Solvers

For small batch evaluations, the neural predictions are strictly compared against the optimal solutions generated by the MIP solver. However, our CO problem is deeply considered NP-hard. As the size of the warehouse graphs scales to real-world industrial dimensions, the exact MIP solver experiences combinatorial explosion, routinely timing out without yielding an optimal bound. This necessitates the introduction of a fast algorithmic baseline to evaluate the neural model on large-scale datasets where true optimality cannot be computed.

The Standalone Heuristic Greedy Solver

As a baseline for comparison with the learned decoders, we also implement a standalone heuristic greedy solver. It iteratively constructs a feasible schedule directly from the raw batch without using any neural predictions. The main objective here is to greedily minimize the overall makespan by adding each order to the operator that offers the *best completion time* with respect to its current local workload, reflecting our original **objective function** (see sec.2.1 in ch.2).

A compact mathematical way to express that greedy objective is:

$$\min C_{\max} \quad \text{with} \quad C_{\max} = \max_{i \in I} C_i \quad (6.3)$$

where C_i is the completion time of operator i .

For the greedy construction, each new order j is assigned to the operator i that minimizes the resulting completion time with respect to the current completion time $C_i(t)$:

$$i^*(j) = \arg \min_{i \in I} \{C_i(t) + T_{\text{travel}}(i, j) + T_{\text{proc}}(j)\} \quad (6.4)$$

Where $i^*(j)$ is the best candidate operator who fits for order j , and then the operator completion time $C_{i^*(j)}(t+1)$ is updated as:

$$C_{i^*(j)}(t+1) = C_{i^*(j)}(t) + T_{\text{travel}}(i^*(j), j) + T_{\text{proc}}(j) \quad (6.5)$$

Also here, we decide to maintain the definition of the active pool of operators I as the theoretical minimum number of operators (k) required to process all orders (see. eq.3.2 in ch.3), as we used it for neural output decoding in ch.6.

The scheduling logic is purely greedy. Missions are processed in a fixed order, and for each mission the solver scans the currently active operators and selects the one that yields the smallest completion time, computed as the operator's current accumulated time plus the travel time from the previous mission and the processing time of the candidate mission. If the resulting completion time exceeds the horizon h_{fixed} , the current operator pool is deemed insufficient and the solver increases the number of active operators. This continues until a feasible schedule is found or all operators have been activated without violating the shift time.

Unlike the recurrent GNN decoder, this heuristic baseline does not learn assignment preferences from data; instead, it relies entirely on deterministic greedy selection and horizon-based feasibility checks.

This baseline is useful because it provides a transparent and computationally simple reference point. It allows us to compare the learned neural model decoders against a conventional handcrafted solver that is easier to interpret but less adaptive to the combinatorial and spatial structure of the warehouse scheduling problem.

loa 3: Pseudocode of the standalone heuristic greedy solver

Require: Mission set J , active operator set I_{act} , processing times T_{proc} , travel times T_{travel} , time horizon h_{fixed}

- 1: Initialize completion times $C_i \leftarrow 0$ for all $i \in I_{act}$
- 2: Initialize routes $R_i \leftarrow \emptyset$ for all $i \in I_{act}$
- 3: **for** each order $j \in J$ **do**
- 4: $i^* \leftarrow \emptyset$
- 5: $C^* \leftarrow +\infty$
- 6: **for** each operator $i \in I_{act}$ **do**
- 7: $C_i^{cand} \leftarrow C_i + T_{travel}(i, j) + T_{proc}(j)$
- 8: **if** $C_i^{cand} < C^*$ **then**
- 9: $C^* \leftarrow C_i^{cand}$
- 10: $i^* \leftarrow i$
- 11: **end if**
- 12: **end for**
- 13: **if** $C^* \leq h_{fixed}$ **then**

```

14:   Assign order  $j$  to operator  $i^*$ 
15:    $R_{i^*} \leftarrow R_{i^*} \cup \{j\}$ 
16:    $C_{i^*} \leftarrow C^*$ 
17: else
18:   Activate an additional operator if available
19:   Recompute the best feasible operator for order  $j$ 
20:   Assign order  $j$  to the feasible operator with minimum completion time
21: end if
22: end for
23: return  $\{R_i\}_{i \in I_{act}}$ 

```

7. Key Performance Indicators and Lower Bound Calculation for Benchmarking

This chapter establishes the quantitative framework used to evaluate the proposed neural solvers. To rigorously assess both the optimality and scalability of the learned heuristics, the benchmarking protocol is structured across two dimensions: small-scale mini-batches and large-scale full problems. We define a comprehensive set of key performance indicators (KPIs) and formalize the computation of theoretical lower bounds (LBs). By analyzing the optimality gaps relative to exact solvers (MIP), heuristic baselines (Greedy), and theoretical lower bounds, we demonstrate the model’s capacity to generalize learned routing policies to computationally intractable scales.

7.1 Key Performance Indicators (KPIs)

The evaluation relies on five distinct KPIs, designed to capture both the global efficiency of the schedule and the fair utilization of the operator fleet. Let I denote the set of all available operators and $I_{active} \subseteq I$ the subset of activated operators. For each operator $i \in I_{active}$, let C_i represent the completion time C of its assigned route.

Makespan (C_{max})

The makespan (defined in sec.2.1 of ch.2) measures the global completion time of the schedule, identifying the bottleneck route. It is defined as the maximum completion time C across all active operators:

$$Z = C_{max} = \max_{i \in I_{active}} C_i \quad (7.1)$$

Total Flow Time (TFT)

Total flow time [37] measures the cumulative completion times of all scheduled orders $j \in J$, where C_j is the time order j is completed. TFT evaluates the temporal **efficiency** of the schedule:

$$TFT = \sum_{j \in J} C_j \quad (7.2)$$

Activation Score

The activation score directly quantifies the fleet utilization cost by counting the absolute number of operators deployed to execute the schedule. Let $I \subseteq I_{max}$ be the subset of operators assigned to at least one order. The activation score, denoted as A , is simply the cardinality of this subset:

$$A = |I| \quad (7.3)$$

Minimizing this score is critical in operational contexts where deploying an additional vehicle or human operator incurs a substantial fixed cost.

Weighted Combination Score

Because warehouse scheduling is intrinsically a multi-objective problem, we define a weighted combination score that jointly penalizes the temporal makespan and the absolute fleet activation cost. This trade-off is governed by the relative weights α and β :

$$W = \alpha \cdot Z + \beta \cdot A \quad (7.4)$$

This combined metric directly mirrors the **objective function** minimized by the multi-criteria MIP formulation, allowing us to evaluate the neural solver's ability to balance time efficiency against resource utilization.

Coefficient of Variation (CV)

To assess the workload balance among the active fleet, we calculate the coefficient of variation of the route completion times. Let μ_C and σ_C be the mean and standard deviation of C_i for $i \in I$. The CV [38] is given by:

$$CV = \frac{\sigma_C}{\mu_C} = \frac{\sqrt{\frac{1}{|I|} \sum_{i=1}^I (C_i - \mu_C)^2}}{\mu_C} \quad (7.5)$$

where μ_C is calculated as the following:

$$\mu_C = \frac{1}{|I|} \sum_{i=1}^I C_i$$

A lower CV implies a highly balanced schedule. CV evaluates the workload **unfairness** over active operators of the schedule.

We state that makespan, activation, and weighted combination scores are considered "**primary objective**" KPIs, while the TFT and CV scores are considered "**efficiency**" and "**fairness**" KPIs, respectively.

7.2 Theoretical Lower Bound (LB) Calculation

To evaluate absolute performance independently of the heuristic and exact baselines, a theoretical lower-bound vector is defined for the main key performance indicators. Let J denote the set of orders and I the set of operators. For each order $j \in J$, let:

$$\tilde{p}_j = \min_{i \in I} P_{i,j} + \min_{k \in J', k \neq j} T_{k,j}$$

where $J' = J \cup \{0\}$ (see sec.2.2 of ch.2), $P_{i,j}$ is the processing time of order j on operator i , and $T_{i,j}$ is an estimate of the minimum travel contribution needed to serve order j by operator i . This quantity serves as an instance-dependent effective workload approximation.

TFT lower bound (TFT_{LB}): A conservative lower bound on the total workload is obtained by aggregating the effective minimum service times across all orders:

$$TFT_{LB} = \sum_{j \in J} \tilde{p}_j - T_{\text{pruned}}. \quad (7.6)$$

The correction term T_{pruned} accounts for the fact that the workload is distributed across multiple operators rather than executed as a single long route. If the batch is split over K operators, the removed inter-order travel links can be approximated as:

$$T_{\text{pruned}} \approx (K - 1) T_{\text{avg}}, \quad (7.7)$$

where T_{avg} is the average travel time in the batch.

Activation lower bound (A_{LB}): Given the strict shift horizon constraint h_{fixed} , the minimum number of operators required to process the estimated workload is

$$A_{LB} = \left\lceil \frac{TFT_{LB}}{h_{\text{fixed}}} \right\rceil. \quad (7.8)$$

This bound reflects the minimum fleet size needed to complete all tasks without violating the operator capacity constraint.

Makespan lower bound (Z_{LB}): The makespan is bounded below by both the largest individual task and the average workload per activated operator:

$$Z_{LB} = \frac{TFT_{LB}}{A_{LB}}. \quad (7.9)$$

This expression follows the classical parallel-machine lower-bound logic, where the makespan cannot be smaller than the heaviest job and cannot be smaller than the average load assigned to the active fleet.

CV lower bound (CV_{LB}): The theoretical minimum coefficient of variation is:

$$CV_{LB} = 0. \quad (7.10)$$

This value corresponds to a perfectly balanced workload distribution across operators.

Weighted combination lower bound (W_{LB}): Using the same objective weights as in the multi-criteria model, the lower bound on the weighted combination score is

$$W_{LB} = \alpha \cdot C_{\text{max},LB} + \beta \cdot A_{LB}. \quad (7.11)$$

In summary, the lower-bound benchmark vector is

$$KPI_{LB} = \{Z_{LB}, A_{LB}, W_{LB}, TFT_{LB}, CV_{LB}\}. \quad (7.12)$$

The lower-bound vector provides an additional reference to quantify how far the obtained schedule remain from the theoretical ideal without considering the strict feasibility constraints.

7.3 Benchmarking Protocol and Gap Analysis

The core of our evaluation methodology remains on analyzing relative performance gaps across different problem scales. For any given KPI, the relative gap of the neural solver's prediction (KPI_{pred}) with respect to a reference value (KPI_{ref}) is defined as:

$$Gap_{ref}(KPI) = \frac{KPI_{pred} - KPI_{ref}}{KPI_{ref}} \times 100\% \quad (7.13)$$

Mini-Batch Evaluation: MIP Optimality Imitation

For small-scale mini-batches, the MIP solver can reliably find the optimal or near-optimal solution (based on the used tolerance gap). By setting $KPI_{ref} = KPI_{optimality}$, we calculate the *neural solver optimality gap* " $Gap_{optimality}$ " with respect to the MIP solver optimality (e.g., $Gap_{optimality} \leq 5\%$ for excellent imitation, $5\% < Gap_{optimality} \leq 10\%$ for acceptable imitation)). This measures the neural model's ability to imitate the exact solver's decisions on makespan, activation, weighted combination, TFT, CV.

Large-Scale Evaluation: Heuristic Dominance

For full-batch problems, the MIP solver faces combinatorial explosion and becomes *intractable* in real-world scenario. Therefore, the reference standard shifts to the standalone greedy heuristic detailed in the previous chapter. We compute the *greedy heuristic baseline gap* using $KPI_{ref} = KPI_{heuristic}$. A negative gap indicates that the neural solver outperforms the deterministic baseline on the large-scale topology.

Generalization Proof via Lower Bound Gaps

Because we cannot compute the MIP optimality gap on full-scale problems, we must prove that the neural solver’s strong performance on mini-batches generalizes robustly to large graphs. To demonstrate this, we benchmark the model against the scale-invariant theoretical lower bounds (KPI_{LB}).

First, we calculate the LB gap for the mini-batches (Gap_{LB}^{mini}). Second, we compute the LB gap for the full-scale batches (Gap_{LB}^{full}). Finally, the generalization capability is proven by evaluating the divergence between these two gaps:

$$\Delta Gap_{LB} = Gap_{LB}^{full} - Gap_{LB}^{mini} \quad (7.14)$$

If ΔGap_{LB} remains small (e.g., $\Delta Gap_{LB} \leq 5\%$ for excellent generalization, $5\% < \Delta Gap_{LB} \leq 10\%$ for acceptable generalization), it mathematically demonstrates that the neural solver successfully scales its learned combinatorial policies, maintaining a consistent distance from theoretical optimality regardless of the graph’s size, effectively bridging the gap between exact small-scale imitation and large-scale practical execution.

Detailed Per-Horizon Evaluation

To assess the robustness of the proposed scheduling models across different temporal regimes, the evaluation is carried out not only at the aggregate instance level but also at the level of individual imposed horizon settings. This *per-horizon* analysis is particularly important because the difficulty of the scheduling problem is strongly influenced by the available time budget, since tighter horizons increase congestion, reduce routing flexibility, and intensify the trade-off between operator activation and completion time. For the mini-batch dataset, each instance is evaluated against its corresponding exact MIP solution whenever the problem size remains tractable. In this regime, the per-horizon analysis measures how closely the proposed neural solver reproduces the optimal behavior under different horizon lengths. For each horizon value, and with respect to the aforementioned KPIs, the optimality gap is computed with respect to the MIP reference, allowing the identification of horizons where the learned policy is near-optimal and horizons where the combinatorial structure becomes more challenging, balancing workload across operators without unnecessarily increasing the number of active resources.

For the full-batch setting, exact optimization is no longer computationally feasible, and the evaluation is therefore performed against the greedy baseline and lower

bound values. The same KPI set is reported per horizon, but now the corresponding gap is measured relative to the greedy schedule and lower bound values. This allows a direct assessment of how the proposed solver behaves as the horizon becomes more restrictive or more relaxed in large-scale instances. In particular, shorter horizons typically increase the pressure on activation efficiency and route compactness, while longer horizons expose the solver’s ability to balance workload across operators without unnecessarily increasing the number of active resources.

A further advantage of the per-horizon evaluation is that it enables a consistent comparison between mini-batches and full-batches under (un)matched temporal constraints. By analyzing the KPI gaps horizon by horizon, it becomes possible to determine whether the model preserves its relative performance when scaling from small instances, where optimal solutions are known, to large instances, where only heuristic references are available. This comparison is essential for demonstrating **generalization**, since a model that performs well only on easy horizons but degrades sharply under tighter constraints would not be suitable for deployment in realistic warehouse environments.

The per-horizon results are therefore interpreted as a structured diagnostic tool. They reveal not only the average quality of the schedules, but also the sensitivity of each method to the operational time budget. In this way, the evaluation provides a finer-grained view of model performance than aggregate summaries alone and supports a more rigorous comparison between exact, heuristic, and learned solvers.

8. Experimental Setup and Training Strategies

In this chapter, we describe the dataset preparation pipeline, the construction of training and evaluation instances, and the computational environment used for the experiments on the proposed architectures in ch.4 and ch.5. The goal is to ensure that the reported results are reproducible and that the learning procedure is evaluated under controlled and well-defined conditions.

8.1 Dataset Structure and Cleaning

The raw warehouse optimization data were stored in tabular form and loaded into `pandas` dataframes for preprocessing. Each record corresponded to an order and contained, among other attributes, spatial coordinates describing the source and destination positions of the task.

Each order is characterized by physical and geometric features (`weight`, `height`, `width`, `length`, `from_x`, `from_y`, `from_z`, `to_x`, `to_y`, `to_z`) and has a unique order identifier. The distance between locations is calculated by an external tool that uses the A* algorithm [39]. Each operator is characterized by mechanical and kinematic features (`speed`, `up_speed`, `up_speed_with_load`, `down_speed`, `down_speed_with_load`, `fork_width`, `fork_length`) and has a unique operator identifier as well.

In particular, the model relies on the source "*from*" and destination "*to*" coordinates, which encode the 3D geometry of each order.

The data were cleaned before training to improve consistency and reduce noise. Exact duplicate rows with the same identifier were removed, and all coordinate fields were converted to numerical format. Missing values and invalid entries, such as NaN or infinite

values, were handled explicitly during preprocessing. In addition, the records were sorted *chronologically* when a temporal ordering was required, so as to preserve the operational structure of the original data as much as possible.

Heterogeneous Graph Generation

To feed this structured data into the neural model, the preprocessed tabular records are mapped into a heterogeneous graph using the `PyTorch Geometric` library. The graph contains two distinct types of nodes: **order** nodes and **operator** nodes (see sec.2.1 in ch.2).

The **order** node features consist of the normalized physical dimensions and coordinate attributes described above. The **operator** node features combine the scaled mechanical attributes with an injected 8-dimensional *sinusoidal positional* encoding. This strong positional encoding allows the neural network to clearly distinguish between different operators, ensuring that the model can learn to assign distinct tasks to specific operators and output varying activation probabilities.

The relationships between these entities are modeled using two types of edges:

- **order-to-order** edges: These represent the routing sequence between tasks. The edge attribute is the normalized travel time between the source and destination points.
- **operator-to-order** edges: These represent the potential assignment of a task to a worker. The edge attribute is a two-dimensional vector containing the normalized processing time of the task (based on the operator’s speed and the task’s vertical coordinates) and the initial travel time from the operator’s base location to the task’s starting point.

Finally, to allow the network to condition its predictions on the changing business context and time constraints, the problem parameters α (makespan weight), β (activation weight), and H (the time horizon) are normalized and injected into the graph as a global state vector. All temporal edge features and global horizon parameters are scaled uniformly relative to a standard 8-hour shift factor (480 minutes) to prevent numerical instability within the attention mechanisms of the deep GNN layers (as mentioned in sec.4.5 of ch.4) .

8.2 Ground Truth and Mini-Batch Generation

To make the learning problem tractable and to obtain ground-truth schedules, the large-scale instances were partitioned into smaller mini-batches. Each mini-batch was constructed by spatial clustering on the order coordinates, using only the source and destination locations. This procedure produced compact groups of order that were sufficiently similar in space while still maintaining diversity across the overall dataset.

In real-world warehouse routing operations, it is common to encounter multiple missions with identical source and destination locations, reflecting high-frequency material flows. To preserve this realistic operational characteristic while preventing the neural network from overfitting to a few dominant routes, a controlled sample-based deduplication strategy was applied prior to clustering. Specifically, a bounded percentage of these spatial duplicates was intentionally maintained in the dataset. Each mini-batch was then constructed by applying **spatial clustering** to these filtered mission coordinates, using only the pickup and delivery locations. This procedure produced compact groups of missions that were sufficiently similar in space while still maintaining structural diversity across the overall dataset.

Each mini-batch was then solved exactly with a MIP model under a randomly selected time-horizon constraint. This design choice was adopted to avoid biasing the neural model toward a single fixed horizon, since training under only one horizon can narrow the learned decision policy and reduce generalization across unseen temporal settings [40, 41]. To promote horizon-invariant learning, we sampled the constraint from the set $\{60, 90, 120, 480\}$ measured in minutes, assigning approximately 25% of the mini-batches to each horizon. The resulting optimal schedules provided supervised targets for training the GNN model. In this way, the MIP solver acted as the oracle that generated ground-truth labels for operator activation, task assignment, and sequencing decisions. This strategy allowed the model to learn from optimal examples while remaining robust across different horizon lengths.

loa 4: Simplified Pseudocode for Spatial mini-batch generation with controlled duplication

Require: Input order dataset, target mini-batch size, duplicate retention limit

Ensure: Set of spatially clustered mini-batches

- 1: Load the mission dataframe and remove exact mission duplicates
 - 2: Clean coordinate columns and convert them to numeric values
 - 3: Keep at most a fixed number of spatial duplicates (e.g., 20–40%) for each route
 - 4: Standardize the spatial coordinates
 - 5: Compute the number of clusters from the dataset size and target min-batch size (e.g., 15–50 orders per mini-batch)
 - 6: Apply constrained clustering on the standardized coordinates
 - 7: **for** each cluster **do**
 - 8: Save the corresponding order to a separate file
 - 9:
 - 10: **return** the generated mini-batches
-

Isomorphic Spatial Augmentation

Because only a very limited number of large *test* batches were available, evaluating the model on the original instances alone would not be sufficient to assess robustness. To address this limitation, we generated additional test instances through *isomorphic spatial augmentation*, namely geometric transformations that preserve the pairwise euclidean distances between order locations. In routing and graph-based optimization, such transformations are particularly useful because they produce new instances that are spatially different in appearance while remaining equivalent from the optimization point of view.

The key idea is that the augmented batches represent exactly the same physical scheduling problem, but embedded differently in the coordinate space. Therefore, the relative structure of the order-operator graph is unchanged: distances between source and destination locations remain identical, travel times remain unchanged, and the optimal schedule for the original batch is still optimal for the transformed batch. As a result, these augmentations allow us to test whether the GNN has

learned true spatial and topological invariances rather than memorizing absolute coordinate patterns [42].

The augmentation strategy was also used during *training* to increase the effective size of the dataset. Since the number of real-world batches was limited, the number of exact solutions available from the MIP solver was limited as well. Spatial augmentation was therefore applied by generating additional mini-batch instances through geometric and spatially consistent transformations of the original data. This procedure enriched the diversity of the training sample (with ~ 2600 examples in total), reduced overfitting to specific coordinate configurations, and exposed the model to a wider range of order layouts while preserving the structural characteristics of the original optimization problem [43, 44].

The implemented augmentation scheme includes nine distance-preserving transformations, which can be grouped into three categories of geometric transformations.

Category 1: Reflections (flips). These transformations simulate mirrored warehouse layouts:

- **Flip X-axis:** all X coordinates are multiplied by -1 , producing a *left-right* mirror of the layout.
- **Flip Y-axis:** all Y coordinates are multiplied by -1 , producing a *top-bottom* mirror of the layout.

Category 2: Transposition. This transformation swaps the spatial axes:

- **Swap X and Y axes:** each point (x, y) is *mapped* to (y, x) . Geometrically, this corresponds to a reflection across the diagonal line $y = x$, and can also be interpreted as a rotated and mirrored version of the original layout.

Category 3: Translations (shifts). These transformations move the entire instance to another region of the coordinate plane while preserving all relative distances:

- **Shift $(+20, +20)$:** small diagonal translation *upward* and to the *right*.
- **Shift $(-20, -20)$:** small diagonal translation *downward* and to the *left*.
- **Shift $(+50, -50)$:** medium translation to the *right* and *downward*.
- **Shift $(-50, +50)$:** medium translation to the *left* and *upward*.

- **Shift** (+200, 0): extreme translation along the *X-axis*.
- **Shift** (0, +200): extreme translation along the *Y-axis*.

The shift magnitudes were chosen to expose the model to both moderate and extreme changes in absolute coordinate values. This is important because a robust GNN should base its decisions on relational structure and travel geometry, not on the absolute numerical range of the coordinates [42].

These augmented batches were used during testing to evaluate invariance and generalization. If the model has truly learned the routing problem from graph topology and relative spatial relations, its performance should remain stable across all transformed instances. In contrast, a model that has partially memorized absolute spatial patterns may fail under reflections or extreme translations. For example, performance degradation on flipped layouts would indicate sensitivity to directional bias, while degradation on large shifts would suggest dependence on absolute coordinate magnitudes rather than invariant spatial relationships.

8.3 Learning Rates and Optimization Strategy

From an optimization perspective, the learning rate is the scalar step size that controls how far the model parameters move along the negative gradient direction at each training iteration. Let w denote the set of trainable weights and let $\mathcal{J}(w)$ be the training objective (e.g., $\mathcal{J}(w) = \mathcal{L}(w)$, where $\mathcal{L}$ is the loss function). Under gradient-based optimization, the parameter update at training iteration t is given by:

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w \mathcal{J}(w^{(t)}), \quad (8.1)$$

where $\eta > 0$ denotes the learning rate. Therefore, η directly regulates the trade-off between convergence speed and optimization stability: if it is too large, the updates may overshoot good minima and lead to unstable training or divergence; if it is too small, convergence may become excessively slow and the optimizer may struggle to make sufficient progress. In other words, η determines the **step size** at each iteration of an optimization algorithm (e.g., gradient descent) as it moves toward minimizing the loss function.

In deep neural networks, the role of the learning rate is particularly critical because the optimization landscape is highly non-convex and different subsets of parameters may exhibit markedly different gradient magnitudes and convergence behaviors. For this reason, learning-rate selection is commonly treated as a core optimization *hyperparameter*, since it strongly influences training dynamics, final solution quality, and generalization performance.

Our model was trained with a multi-head architecture, in which a shared GNN trunk is followed by three task-specific MLP prediction heads. Because the different heads are associated with different learning dynamics, experiments were conducted both with a single global learning rate and with distinct learning rates for different parts of the network. The latter strategy was adopted to better balance the optimization of shared and task-specific parameters.

Using different learning rates can be advantageous when one head learns faster than the others or when the gradients associated with different tasks have very different magnitudes [45, 46]. In multi-task optimization, a uniform learning rate may cause unstable convergence or bias the shared representation toward the dominant task, a phenomenon often related to gradient conflict or loss imbalance [47, 48]. By decoupling the learning rates, the updates can be scaled according to the complexity and convergence behavior of each prediction head. For this reason, learning-rate selection and task-specific optimizer initialization were treated as critical *hyperparameters* of the training procedure.

We define the learning-rate set as the collection of step sizes assigned to the shared trunk and to each of the three task-specific heads. Hence, in our multi-optimizer setting, a formal mathematical definition is

$$\mathcal{H}_\eta = \{\eta_{\text{trunk}}, \eta_{\text{active}}, \eta_{\text{assign}}, \eta_{\text{seq}}\}, \quad (8.2)$$

where η_{trunk} denotes the learning rate of the shared GNN trunk, while η_{active} , η_{assign} , and η_{seq} denote the learning rates associated with the activation, assignment, and sequence heads, respectively.

8.4 Model Quantitative Evaluation Metrics

In supervised classification, there are several quantitative metrics used to evaluate the model in order to deduce if the model learned correctly the latent patterns from the given training datasets, the most common metric is the *accuracy*. Relying only on classification accuracy to evaluate predictive performance is often misleading, particularly in the presence of **class imbalance**. This vulnerability is formally known as the *accuracy paradox*[\[49\]](#)—a counter-intuitive phenomenon in machine learning where a model achieves exceptionally high accuracy despite possessing zero true predictive power. This paradox occurs almost exclusively in highly imbalanced datasets, where the class distribution is considered imbalanced in such a dataset (e.g., the negative class "majority" substantially outnumbers the positive class "minority").

In our warehouse scheduling context, the modeled graph is inherently imbalanced. Out of all possible combinatorial edges, only a small fraction represent valid sequencing or assignment connections. Similarly, only a limited subset of available operator nodes will be activated during a given shift, leaving the rest inactive.

To illustrate the danger of the accuracy paradox, consider a hypothetical dataset containing 1,000 potential scheduling routes, where 990 are invalid (label 0) and only 10 are valid (label 1).

- **Scenario A (naive predictor):** A model completely ignores the input features and trivially predicts "invalid" (0) for every route. It correctly identifies all 990 negative instances but misses all 10 positive ones. Consequently, the model achieves an accuracy of $990/1000 = 99\%$. Despite this high score, the model is practically useless for scheduling because it fails to identify a single valid route.
- **Scenario B (useful predictor):** A capable model correctly identifies 8 of the 10 valid routes (true positives), misses 2 valid routes (false negatives), and incorrectly classifies 20 invalid routes as valid (false positives). The resulting accuracy is $(970 + 8)/1000 = 97.8\%$.

Even though scenario B represents a highly functional model and scenario A is entirely useless, scenario A yields a higher accuracy. This discrepancy demonstrates how raw accuracy can provide a dangerously false sense of model performance.

Precision, Recall, and the F1-Score

To overcome the limitations of standard accuracy, performance evaluation must rely on metrics that explicitly measure the correct identification of the positive class. Let true positives (TP) be the correctly predicted positive instances, false positives (FP) be the negative instances incorrectly predicted as positive, and false negatives (FN) be the positive instances incorrectly predicted as negative.

Precision: It quantifies the exactness of the model. Out of all instances the model predicted as positive, precision measures the fraction that were actually positive. A high precision indicates a low rate of false positives:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (8.3)$$

Recall: It quantifies the completeness of the model. Out of all actual positive instances in the ground truth, recall measures the fraction that the model successfully identified. A high recall indicates a low rate of false negatives:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (8.4)$$

F1-Score: It is the harmonic mean of precision and recall. It provides a single, robust metric that balances the trade-off between minimizing false positives and false negatives. Because it explicitly focuses on the positive class and severely penalizes models where either precision or recall is low, the f1-score is highly resistant to being overwhelmed by a majority negative class:

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8.5)$$

Metric Selection for Task-Specific Heads

Because our GNN architecture tackles a multi-objective scheduling problem encompassing operator activation, order assignment, and sequence ordering, metric selection must reflect the specific structural imbalances of each task.

1. **Activation and assignment heads (binary classification):** These tasks are formulated as highly sparse binary classification problems. In a typical scheduling graph, over 99% of all possible operator-to-order assignment edges

are mathematically invalid (label 0). A model predicting "No" for all edges would achieve near-perfect accuracy while failing to assign any work. Therefore, the **f1-score** is the primary evaluation metric for these heads. It forces the network to strike a strict balance: achieving high coverage of the required orders (recall) without predicting spurious or invalid assignment links (precision).

2. **Sequence head (pairwise ordering):** The sequencing task relies on relative pairwise ordering, where a directed edge $A \rightarrow B$ is classified as positive only if order A strictly precedes order B . While the class distribution for valid sequence pairs is typically more balanced (meaning standard accuracy would theoretically be acceptable), the **f1-score** is retained for this head as well. This ensures a consistent and unified quantitative evaluation framework across all components of the multi-head architecture.

8.5 Hyperparameter Tuning

The primary objective of machine learning is to train models that generalize well to unseen data. During training, a model balances two primary sources of error: *underfitting* and *overfitting* [50, 51]. Underfitting occurs when a model lacks the structural capacity or the optimization runway to capture the underlying patterns in the training data, resulting in poor performance on both the training and test sets. Conversely, overfitting occurs when a model possesses excessive capacity or is trained for too long, causing it to memorize stochastic noise and specific training configurations rather than learning the true underlying data distribution. This results in an artificially low training error, but a severely degraded performance on unseen test data.

Hyperparameter tuning was performed to navigate this trade-off, identifying a model configuration that provided stable convergence and strong generalization performance. The tuning process covered the main architectural and optimization parameters, including batch size, learning rates, architectural complexity (e.g., hidden dimensions and the number of attention heads), and task-specific decision thresholds. By systematically adjusting these parameters, we regulate the representational capacity of the network and control the learning dynamics, thereby mitigating the risks of both underfitting and overfitting.

To prevent the reported test performance from being biased by repeated inspection of the test data (a phenomenon known as *data leakage*), hyperparameters were evaluated exclusively using cross-validation on the training set. The selected configuration was then fixed for the final testing phase, ensuring that hyperparameter tuning served strictly as an unbiased model selection tool.

Although several methods exist for hyperparameter optimization, **grid search** combined with **K -fold cross-validation** [51] is the most widely adopted technique. The grid represents all possible combinations of predefined hyperparameter values. For each combination, K -fold cross-validation is executed to record the validation scores (e.g., f1-scores), and the combination yielding the highest average score is selected. The K -fold cross-validation process operates as follows:

1. **Dataset partitioning:** The training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ of N independent and identically distributed samples is divided into K disjoint subsets (folds) $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_K$ of approximately equal size $M = N/K$, such that:

$$\bigcup_{k=1}^K \mathcal{D}_k = \mathcal{D} \quad \text{and} \quad \mathcal{D}_i \cap \mathcal{D}_j = \emptyset \quad \forall i \neq j \quad (8.6)$$

For each fold $k \in \{1, 2, \dots, K\}$, we define the validation set as $V_k = \mathcal{D}_k$ and the training set as $T_k = \mathcal{D} \setminus \mathcal{D}_k$.

2. **Parameter learning (inner optimization):** For each hyperparameter configuration $\lambda \in \Lambda$, we train a model $f(x; \theta, \lambda)$ on the training set T_k . The optimized model parameters $\hat{\theta}_k(\lambda)$ are found by minimizing the loss function $\mathcal{L}$:

$$\hat{\theta}_k(\lambda) = \arg \min_{\theta} \mathcal{L}(f(T_k; \theta, \lambda)) \quad (8.7)$$

Since the model is trained independently for each hyperparameter configuration across all folds, this step requires K distinct training runs per configuration.

3. **Validation scoring:** We evaluate the learned parameters $\hat{\theta}_k(\lambda)$ on the held-out validation set V_k using a validation metric $\mathcal{L}_{\text{val}}$ (e.g., f1-Score). The validation error for fold k is:

$$E_k(\lambda) = \frac{1}{|V_k|} \sum_{(x,y) \in V_k} \mathcal{L}_{\text{val}}(y, f(x; \hat{\theta}_k(\lambda), \lambda)) \quad (8.8)$$

4. **Cross-validation estimator:** The estimator for the hyperparameter configuration λ is the average of the validation errors across all K folds:

$$CV(\lambda) = \frac{1}{K} \sum_{k=1}^K E_k(\lambda) \quad (8.9)$$

This estimator serves as a proxy for the expected generalization error on unseen data.

5. **Hyperparameter optimization (outer optimization):** The tuning process solves the optimization problem over the discrete grid space Λ to find the configuration λ^* that minimizes the cross-validation error:

$$\lambda^* = \arg \min_{\lambda \in \Lambda} CV(\lambda) \quad (8.10)$$

6. **Final refit:** Once λ^* is identified, the final model parameters θ^* are learned using the entire dataset $\mathcal{D}$ to maximize information utilization:

$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in \mathcal{D}} \mathcal{L}(y, f(x; \theta, \lambda^*)) \quad (8.11)$$

The final model used for inference is $f(x; \theta^*, \lambda^*)$.

 loa 5: Simplified Pseudocode of Grid Search with K -Fold Cross-Validation

Require: Training set $\mathcal{D}$, hyperparameter grid Λ , number of folds K , evaluation metric M

Ensure: Best hyperparameter configuration λ^*

```

1:  $\lambda^* \leftarrow \emptyset$ 
2:  $s^* \leftarrow -\infty$ 
3: for each hyperparameter configuration  $\lambda \in \Lambda$  do
4:   Split  $\mathcal{D}$  into  $K$  folds:  $\mathcal{D}_1, \dots, \mathcal{D}_K$ 
5:   Initialize score list  $S_\lambda \leftarrow \emptyset$ 
6:   for each fold  $k = 1, \dots, K$  do
7:      $V_k \leftarrow \mathcal{D}_k$ 
8:      $T_k \leftarrow \mathcal{D} \setminus \mathcal{D}_k$ 
9:     Train model  $f(\cdot; \theta, \lambda)$  on  $T_k$ 
10:    Evaluate on  $V_k$ :  $s_k \leftarrow M(f, V_k)$ 
11:    Append  $s_k$  to  $S_\lambda$ 
12:  end for
13:   $\bar{s}_\lambda \leftarrow \frac{1}{K} \sum_{k=1}^K s_k$ 
14:  if  $\bar{s}_\lambda > s^*$  then
15:     $s^* \leftarrow \bar{s}_\lambda$ 
16:     $\lambda^* \leftarrow \lambda$ 
17:  end if
18: end for
19:
20: return  $\lambda^*$ 

```

Hyperparameters to Be Tuned

The proposed multi-head GNN models was trained after tuning a set of architectural, optimization, and evaluation hyperparameters. These parameters were selected because they directly influence convergence stability, representational capacity, regularization, and final predictive performance.

Batch Size. The batch size determines the number of graph instances processed before each optimization and backpropagation step. Using a batch size greater than one is generally preferred for two main reasons. First, it improves gradient stability: updating the model using a single graph can produce noisy and highly variable gradients because individual scheduling instances may differ significantly in difficulty. In contrast, averaging the loss over multiple graphs yields a smoother and more reliable gradient signal, which improves generalization. Second, it increases hardware efficiency, since GPUs are designed for parallel computation and are underutilized when processing one graph at a time.

In the `Pytorch Geometric` framework, batching does not create a tensor of shape (B, N, F) —where B is the number of graphs processed together “batch size”, N is number of nodes per graph and F is number of features per node. Instead, it constructs one large disjoint graph from all graphs in the mini-batch. For example, a batch of 32 graphs with 500 nodes each results in a single graph with 16,000 nodes. If the batch size is too large, this can lead to out-of-memory (OOM) errors. Since our graphs are relatively small (mini-batches), with fewer than 500 nodes, batch sizes of 32 or 64 were computationally feasible. For this reason, batch size was treated as a formal hyperparameter and optimized through grid search.

Learning rate. The learning rate controls the step size (per head as mentioned in sec.8.3) used to update the model weights during training. If the learning rate is too large, the optimization process may become unstable and the loss may diverge. If it is too small, training may become excessively slow or may stagnate in poor local minima. Typical values include 10^{-5} , 10^{-4} , 10^{-3} , and 10^{-2} .

Hidden dimension. The hidden dimension defines the size of the learned node embeddings propagated through the network. Common values include 64, 128, and 256. If the hidden dimension is too small, the model may not be expressive enough to capture the complex relationships between operators, orders, and constraints, leading to underfitting. If it is too large, training and inference become slower, and the model may overfit.

Number of attention heads. The number of attention heads in `GATv2Conv` controls how many independent attention mechanisms operate in parallel. Using more heads enables the model to attend to different aspects of the graph neighborhood simultaneously. For example, one head may focus on spatial proximity, another on capability or equipment compatibility, another on temporal alignment, and another on capacity-

related constraints. In our experiments, we considered four attention heads as a balanced choice for capturing these complementary signals.

Number of GNN layers. The number of GNN layers determines how many message-passing rounds are performed. With few layers, each node only aggregates information from immediate neighbors. With more layers, information can propagate farther through the graph, but too many layers may cause *oversmoothing*, where node representations become indistinguishable. In practice, two or three layers are usually sufficient for most graph-routing problems, and were therefore treated as the main candidates for tuning.

Dropout. Dropout is a regularization mechanism used to reduce *overfitting* by randomly deactivating a fraction of hidden units during training. It improves generalization by preventing the model from relying too heavily on a small subset of features. A low dropout rate may be insufficient to control memorization, while an excessively high rate may remove too much signal and harm learning. Accordingly, dropout was included among the tunable regularization parameters. Typical values in $[0, 0.2]$.

Should We Tune the Number of Training Iterations (Epochs)?

The number of epochs dictates how many complete iterations the optimization algorithm makes over the entire training dataset to update the model weights. The required number of epochs to reach an optimal decision boundary is inherently coupled with the learning rate. If the chosen learning rate is high, each gradient step is large, allowing the optimizer to traverse the loss landscape rapidly and potentially reach the convergence to a local minimum in fewer iterations. Conversely, a lower learning rate yields smaller, more stable weight updates, but requires a proportionally higher number of epochs to achieve convergence.

Due to this strong interdependence, simultaneously tuning both the learning rate and the number of epochs expands the hyperparameter search space exponentially, resulting in severe computational inefficiency. A more practical and standard strategy is to fix the number of epochs at a sufficient empirical upper bound (e.g., 50 epochs) and restrict the grid search to tuning the task-specific learning rates. This ensures that the optimizer has an adequate runway to converge without needlessly repeating hyperparameter combinations that represent the same effective optimization distance (i.e., step size multiplied by the number of steps).

Furthermore, treating the number of epochs as an unbounded hyperparameter can be actively harmful. Once the model converges, executing additional training epochs provides no meaningful reduction in the loss and can instead degrade the learned representations. Iterating beyond convergence exposes the model to overfitting, where the network begins to memorize noise in the training set rather than generalizing. Additionally, prolonged exposure to stochastic gradient noise in the late stages of training can induce parameter drift, causing the weights to slowly deviate from a previously discovered optimal point. Consequently, fixing the epoch count and optimizing the learning rate provides a more stable, efficient, and computationally viable tuning framework.

Tuning Task-Specific Decision Thresholds

Unlike standard hyperparameters, the classification thresholds used to evaluate quantitative metrics (e.g., f1-score) do not affect the internal weights or the prediction quality of the model, meaning the network does not need to be retrained for different threshold values. However, applying a single threshold across all tasks is suboptimal. Because our model handles three fundamentally different tasks—activation, assignment, and sequence—the optimal decision boundaries differ significantly due to varying task difficulties and imbalanced class distributions. Consequently, an independent threshold was tuned using “precision-recall curve” [52] for each specific task head to maximize its respective quantitative score.

Let the set of task-specific thresholds be defined as

$$\mathcal{T} = \{\tau_{\text{activation}}, \tau_{\text{assignment}}, \tau_{\text{sequence}}\}, \quad (8.12)$$

where $\tau_{\text{activation}}$, $\tau_{\text{assignment}}$, and τ_{sequence} denote the decision thresholds for the activation, assignment, and sequence heads, respectively.

For a given task head $h \in \{\text{activation}, \text{assignment}, \text{sequence}\}$, let $p_i^{(h)} \in [0, 1]$ be the predicted probability for sample i . The binary prediction induced by a threshold τ_h is defined as

$$\hat{y}_i^{(h)}(\tau_h) = \begin{cases} 1, & \text{if } p_i^{(h)} \geq \tau_h, \\ 0, & \text{otherwise.} \end{cases} \quad (8.13)$$

To tune τ_h , the threshold is swept over a range of candidate values in $[0, 1]$. For each candidate threshold, precision and recall (mentioned in sec.8.4) are computed on the validation set with respect to τ_h .

The precision-recall curve is then obtained by plotting $\text{Precision}_h(\tau_h)$ against $\text{Recall}_h(\tau_h)$ as τ_h varies (see fig.8.1). Intuitively, increasing the threshold makes the classifier more conservative, which often increases precision but decreases recall, whereas lowering the threshold has the opposite effect.

In order to select the operating point for each head, we maximize a **target validation metric**, typically the f1-score $F1_h(\tau_h)$:

$$\tau_h^* = \arg \max_{\tau_h \in [0,1]} F1_h(\tau_h). \quad (8.14)$$

Accordingly, the optimal threshold is chosen independently for each prediction head:

$$\mathcal{T}^* = \{\tau_{\text{activation}}^*, \tau_{\text{assignment}}^*, \tau_{\text{sequence}}^*\}. \quad (8.15)$$

This post-training tuning strategy allows each task to operate at its own most suitable decision boundary, thereby improving quantitative evaluation without altering the learned model parameters.

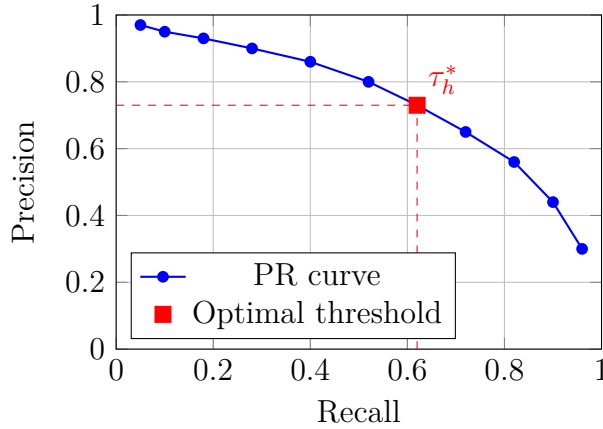


Figure 8.1: Precision-recall curve with an example optimal decision threshold.

8.6 Training Curves and Monitoring

Training was monitored using loss and f1-score curves (mentioned in sec.8.4). These plots were used to verify that the model converged properly and to identify possible signs of overfitting, underfitting, or unstable gradients. In particular, the evolution of the training and validation losses provided a direct indication of how well the model generalized across unseen instances.

The **relationship** between training and validation curves provides a standard diagnostic

for model capacity. A widening gap between them, with low training loss but substantially higher validation loss, suggests *overfitting*, whereas similarly high losses on both splits indicate *underfitting*.

The plots were therefore an essential part of the experimental analysis, complementing the quantitative metrics reported in the model evaluation section.

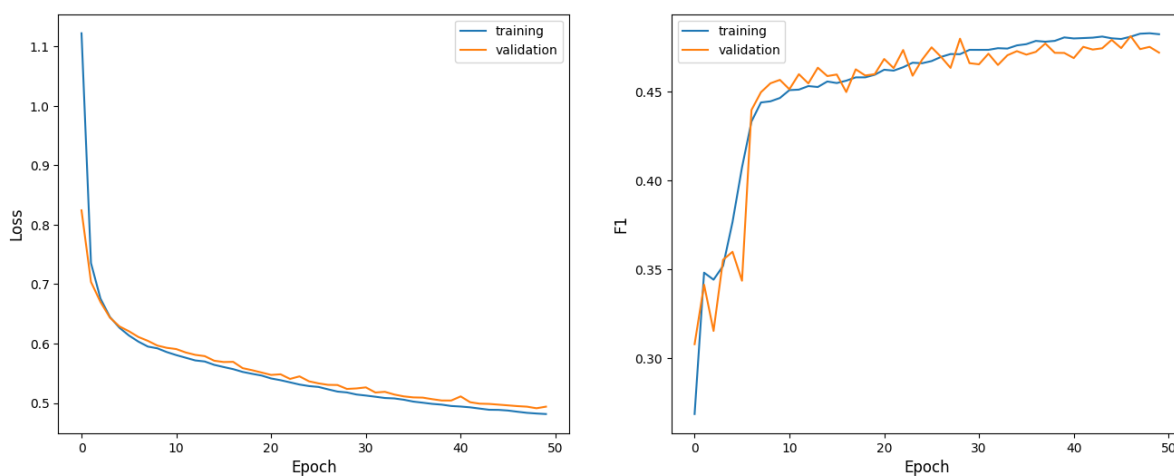
Model-Based Training and Validation Curves Analysis

The learning curves were an essential part of the experimental analysis, complementing the quantitative metrics reported in the model evaluation section. The training and validation trajectories for the loss and f1-score provide direct insight into convergence stability, learning dynamics, and the models' ability to generalize without overfitting. We compared the one-shot GNN model (proposed in ch.4) against the Graph2Seq recurrent model (proposed in ch.5) across the overall objective and the three task-specific heads (activation, assignment, and sequence).

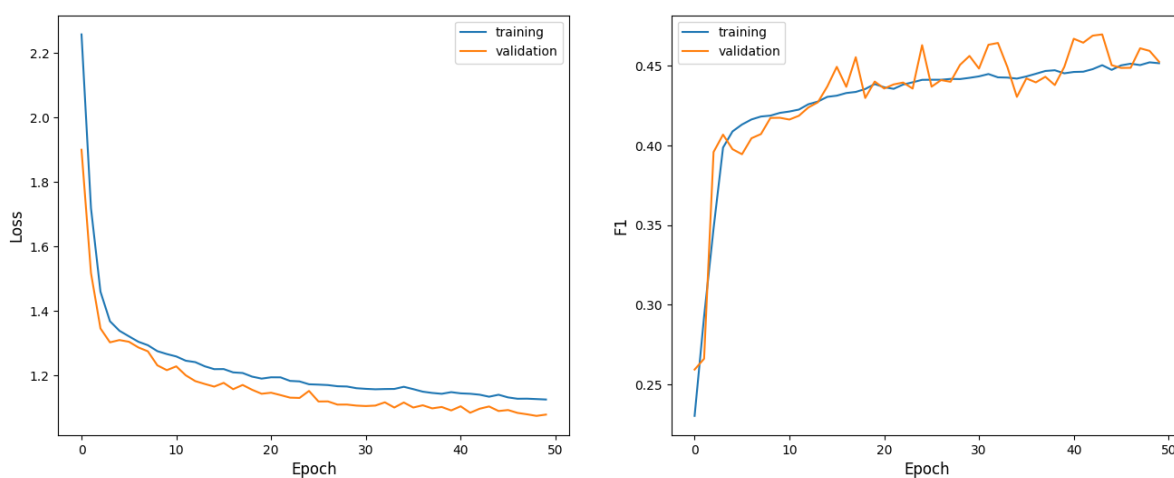
Overall Model Performance

The overall training loss curves for both the one-shot and recurrent models demonstrate a steep initial descent followed by a gradual stabilization, indicating effective global optimization. However, a structural difference is visible in the validation behavior. The one-shot model (fig. 8.2a) achieves a relatively smooth validation f1-score that plateaued steadily around 0.45–0.48. In contrast, the recurrent model (fig. 8.2b) exhibits a slightly more volatile validation trajectory during the initial epochs before stabilizing at a comparable f1-score. This increased early-stage variance in the recurrent model is expected, as the sequence-to-sequence unrolling mechanism must adapt to BPTT, making the joint loss landscape more complex to traverse.

Furthermore, while the training and validation loss curves successfully converge for both models, the recurrent model exhibits a distinct gap: its training loss stabilizes at a higher value (≈ 1.2) compared to its validation loss. This behavior is a direct consequence of the BPTT optimization and the strict teacher-forcing applied during training, which rigorously accumulates penalty gradients across the fully unrolled temporal chunks, naturally resulting in a higher stabilized loss during the training phase.

Overall Model Performance (Multi-Optimizers)

(a) Overall Model Performance (One-Shot GNN Model)

Overall Model Performance (Recurrent-Multi-Optimizers)

(b) Overall Model Performance (Graph2Seq Recurrent Model)

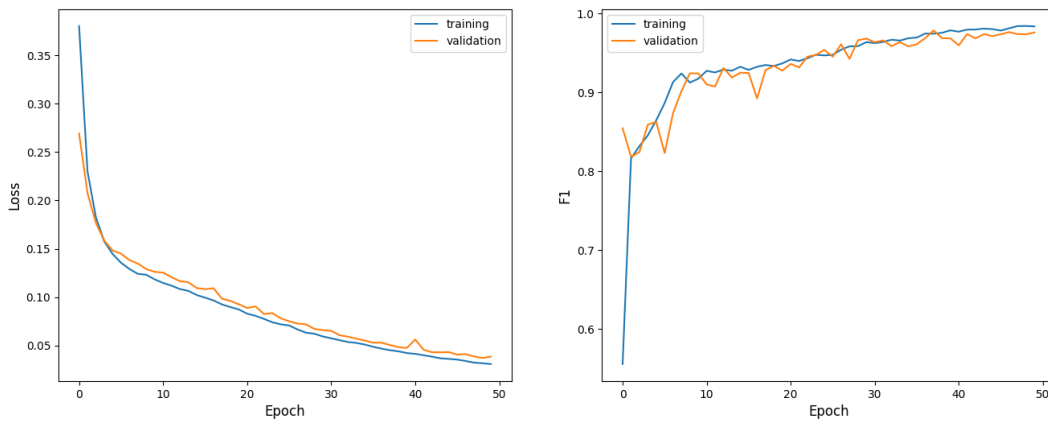
Figure 8.2: Overall Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

Activation Head Performance

The activation head represents the easiest classification task, as indicated by the rapid convergence of the f1-score in both architectures. For the one-shot model (fig. 8.3a), the validation f1-score climbs quickly and tightly tracks the training score, eventually

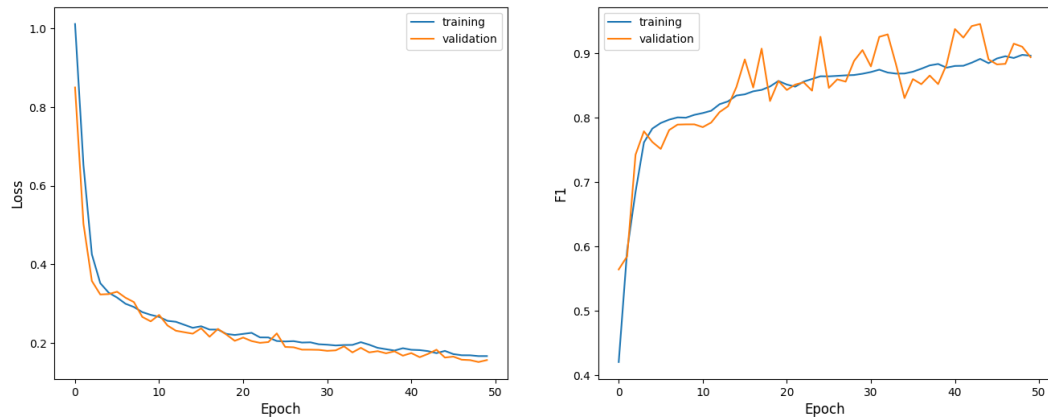
stabilizing just below 1.0. The recurrent model (fig. 8.3b) shows an equally impressive terminal f1-score (around 0.90), but the validation curve exhibits minor oscillatory behavior. This suggests that while the recurrent model successfully learns which operators to activate, the sequential decoding introduces minor gradient noise to the activation head caused by back-propagation signals compared to the direct predictions of the one-shot approach. Both loss curves converge for both models but at different values (≈ 0.05 for the one-shot model, ≈ 0.20 for the recurrent model).

Activation Head Performance (Separate-Optimizer)



(a) Activation Head Performance (One-Shot GNN Model)

Activation Head Performance (Recurrent-Separate-Optimizer)

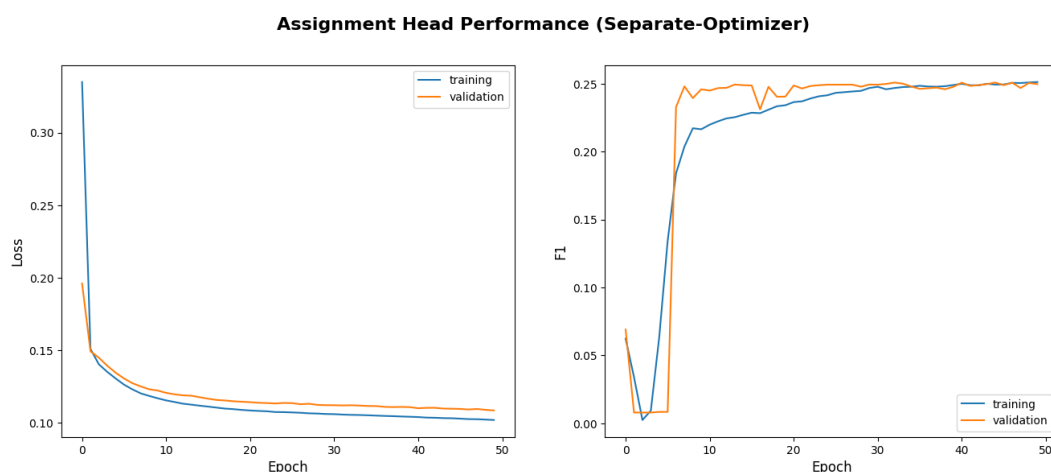


(b) Activation Head Performance (Graph2Seq Recurrent Model)

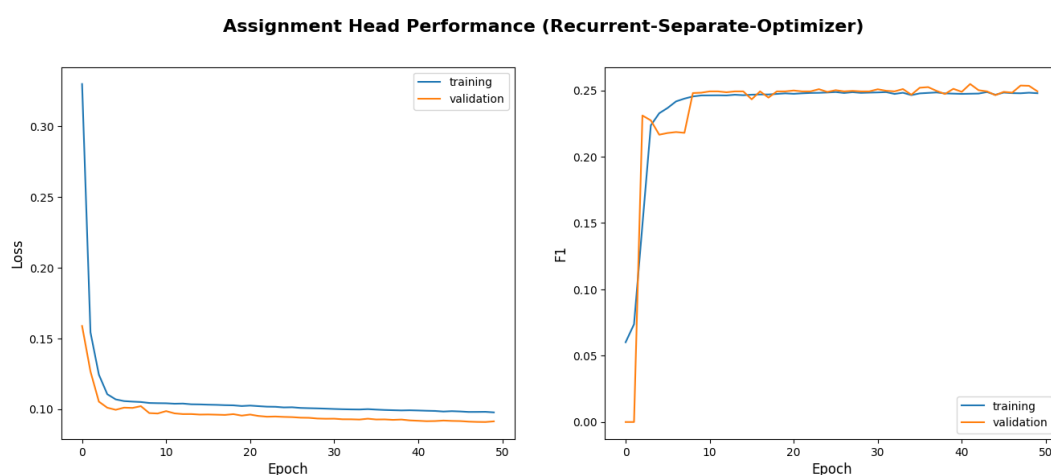
Figure 8.3: Activation Head Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

Assignment Head Performance

The assignment head performance highlights a significant learning bottleneck in the early epochs. For both the one-shot (fig. 8.4a) and recurrent (fig. 8.4b) models, the f1-score remains close to zero for the first 5 epochs before experiencing a sharp, step-like increase, ultimately stabilizing near 0.25. Both loss curves converge for both models at the same value (≈ 0.10).



(a) Assignment Head Performance (One-Shot GNN Model)



(b) Assignment Head Performance (Graph2Seq Recurrent Model)

Figure 8.4: Assignment Head Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

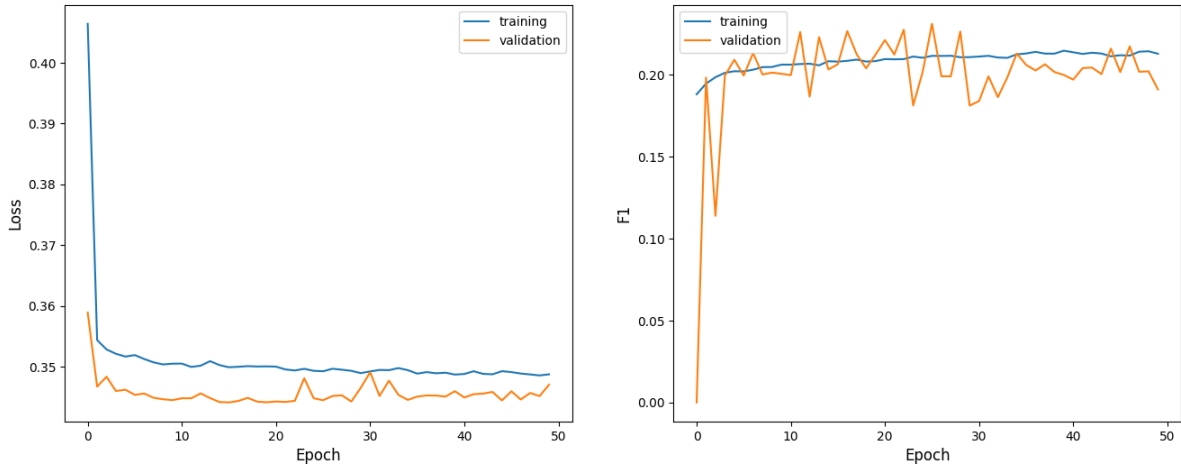
The distinct phase-transition behavior implies that the models initially struggled with the extreme sparsity of the assignment edges (predicting "0" for almost all combinations to minimize binary cross-entropy loss). Once the GNN layers generated sufficiently distinct node embeddings, the task-specific optimizer successfully broke the symmetry, enabling the assignment head to accurately identify valid edges. Notably, the recurrent model's validation curve tracks the training curve with a slight over-performance, suggesting strong regularization and robust generalization for the assignment task.

Sequence Head Performance

The sequence head proved to be the most challenging predictive task, resulting in distinct behaviors between the two architectures. In the one-shot model (fig. 8.5a), the training loss decreases steadily, but the f1-score stabilizes at a lower absolute value (≈ 0.20), accompanied by high variance in the validation curve. This indicates that predicting all sequential steps simultaneously without a temporal autoregressive mechanism leads to high *uncertainty* and suboptimal sequencing. Conversely, the recurrent model (fig. 8.5b) manages this task differently. Although the validation f1-score experiences a sharp drop near epoch 3, it recovers quickly and maintains a more stable validation curve. While the absolute f1-score remains low due to the strict pairwise ordering nature, the recurrent model's sequential decoding structure demonstrates a higher capacity to handle the step-by-step nature of the sequencing task, mitigating some of the validation instability seen in the one-shot model.

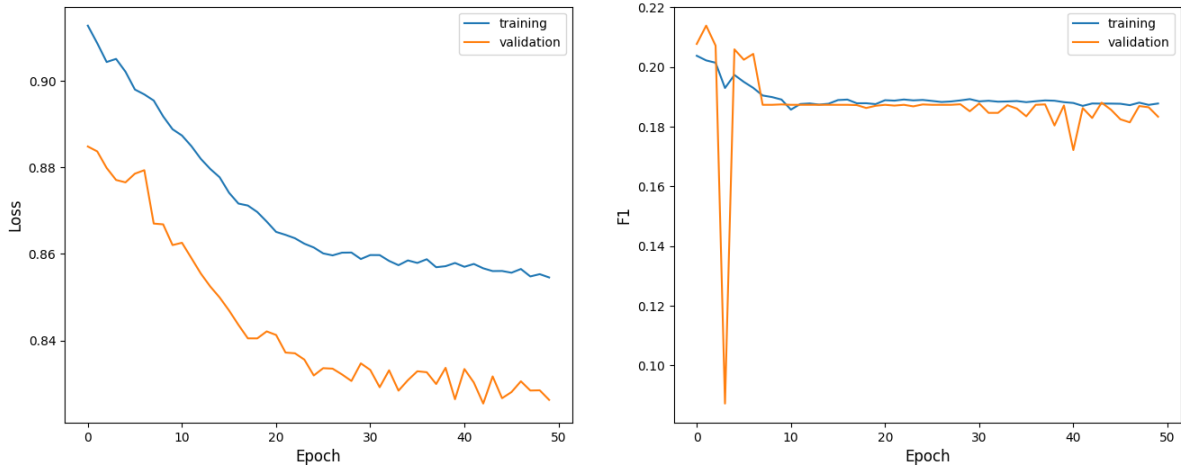
An important observation is the numerical discrepancy between the loss magnitudes: the one-shot model's sequence loss converges near 0.35, whereas the recurrent model's sequence loss stabilizes near 0.86. This difference does not imply that the recurrent model performs worse; rather, it reflects a fundamental difference in the evaluation scope. The one-shot model evaluates sequence edges as an independent pairwise binary classification over the entire graph simultaneously. Although we apply positive class weighting to counteract the extreme sparsity of the sequence paths, the vast majority of target edges are still zero. Consequently, the massive volume of "easy" true negatives dominates the denominator of the mean BCE loss. This dilution artificially deflates the numerical loss value down to 0.35, masking the model's actual struggle to correctly identify the rare positive sequences, as reflected by its poor f1-score.

Sequencing Head Performance (Separate-Optimizer)



(a) Sequence Head Performance (One-Shot GNN Model)

Sequencing Head Performance (Recurrent-Separate-Optimizer)



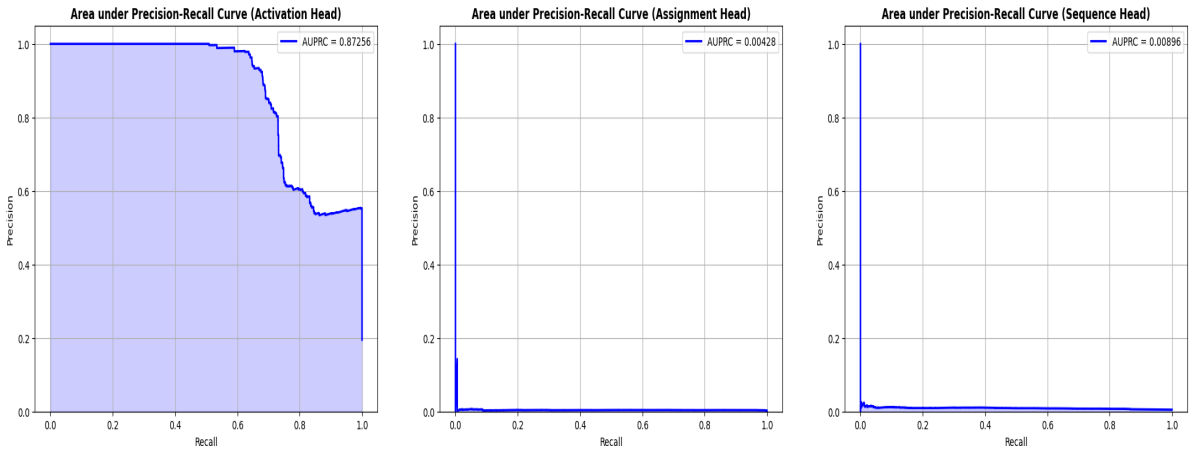
(b) Sequence Head Performance (Graph2Seq Recurrent Model)

Figure 8.5: Sequence Head Performance Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

Thresholding and Task-Specific Head F1-score

The stabilization of the assignment f1-score at approximately 0.25 is a direct consequence of the extreme *class imbalance* inherent in the graph-routing formulation. Because the vast majority of possible operator-to-order assignment edges are invalid (label 0), the

positive class is exceptionally sparse. In this highly imbalanced environment, even a small number of false positive predictions drastically reduces precision, while missing valid edges penalizes recall. Because the f1-score is the harmonic mean of precision and recall, it remains constrained; the model successfully learns to identify a portion of the correct assignments (modest recall), but the vast combinatorial space of potential edges leads to unavoidable false positive predictions (low precision). The same justification can be given to the sequence f1-score which is stabilized at approximately 0.20.



(a) Activation head (AUPRC ~ 0.873). (b) Assignment head (AUPRC ~ 0.004). (c) Sequence head (AUPRC ~ 0.009).

Figure 8.6: Area under Precision-Recall Curve (PRC) for different model heads.

Despite the low absolute value, the fact that the recurrent model's validation curve tracks the training curve closely indicates that the network is learning the underlying assignment and sequencing decisions rather than simply memorizing the training data. In support of this interpretation, the area under the precision-recall curve (AUPRC) is highly asymmetric across heads (fig. 8.6): activation reaches a strong value of 0.8725, whereas assignment and sequence remain extremely low at 0.0042 and 0.0089, respectively. Such low AUPRC values make it difficult to identify a meaningful tuned threshold for those heads, so a modest f1-score can still be acceptable for a specific operating point.

Furthermore, threshold-based evaluation is not used in the actual schedule-building procedure, where we select the top order-operator assignments and order-order sequences under the horizon constraint rather than applying a binary cutoff "**thresholding**". For

this reason, we retain the f1-score plots as an auxiliary classification-style diagnostic, complementary to the loss curves, rather than as the main criterion for schedule construction.

For all heads, the loss for both models successfully converged, since we applied dynamic class weighting (defined in sec. 4.6 of ch. 4).

Summary of the Pipeline

In summary, the experimental pipeline followed four main steps: first, the raw data were cleaned and organized into structured dataframes; second, spatial mini-batches were generated and solved exactly by the MIP model to produce ground-truth schedules; third, the dataset was augmented to increase the variety of training examples; fourth, the neural model was tuned and trained under controlled optimization settings; and fifth, in the next chapters: both of models (one-shot GNN & Graph2Seq) were qualitatively evaluated against the validation set (which contains unseen mini-batch examples) and the better model between them is used for in-depth generalized evaluation against the test set (which contains unseen full-batch examples). This workflow ensured that the proposed method was evaluated on a realistic and scientifically rigorous benchmarking.

9. Mini-Batch Experiments and Qualitative Analysis

While the previous chapters detailed the internal training dynamics and classification performance of the neural architectures, assessing the practical viability of the generated schedules requires a rigorous evaluation against exact ground-truth solutions. In this chapter, we present the experimental results obtained on the mini-batch validation set. To provide a comprehensive assessment, the predicted schedules are evaluated using the KPIs introduced in [ch.7](#), focusing specifically on the multi-objective criteria of operational efficiency and workload fairness.

The primary objective of these experiments is to quantify the optimality gap between the schedules produced by the neural models—namely, the one-shot GNN and the Graph2Seq recurrent architecture—and the exact optimal schedules generated by the MIP solver. For detailed analysis, we use the one-shot model KPIs’ scores from [table 9.1](#); meanwhile, the recurrent model KPIs’ scores from [table 9.2](#).

To achieve this, the analysis is divided into two main dimensions:

1. **Primary criteria evaluation:** We analyze the models’ capacity to minimize the primary cost drivers of the warehouse scheduling problem. This includes the *makespan* (the time required to complete the final mission in the batch) and the *activation score* (the total number of operators deployed). The aggregate performance across these criteria is further evaluated using the *weighted combination score* which respects the α & β formulation in the main objective function (see [sec.2.2](#) in [ch.2](#)).
2. **Efficiency and fairness evaluation:** Beyond raw speed and resource minimization, a high-quality schedule must ensure equitable task distribution

among the active workforce. We evaluate this qualitative dimension using the *TFT*, which measures cumulative operational time, and the *CV*, which quantifies the statistical imbalance of workloads across the active operators.

Table 9.1: Raw KPI Scores on Mini-batch Validation Set (One-Shot GNN Model)

Batch ID	Makespan Score		Activation Score		Weighted Score		TFT Score		CV Score	
	Opt	Pred	Opt	Pred	Opt	Pred	Opt	Pred	Opt	Pred
[0]: 107	43.56	22.09	3	6	3.04	6.02	130.39	131.80	0.0036	0.0076
[1]: 126	82.45	82.64	2	3	2.08	3.08	144.68	145.75	0.1976	0.6422
[2]: 142	66.42	88.42	2	2	2.07	2.09	132.84	132.84	0.0000	0.4684
[3]: 145	112.89	113.06	2	3	2.11	3.11	203.28	202.73	0.1565	0.6730
[4]: 156	66.25	44.28	2	4	2.07	4.04	131.50	130.58	0.0107	0.4067
[5]: 159	41.60	41.60	4	4	4.04	4.04	166.40	166.40	0.0000	0.0000
[6]: 15	186.72	104.25	1	2	1.19	2.10	186.72	187.80	0.0000	0.1559
...	...	...	...	...	...	...	...	...	...	...
[82]: 670	154.23	154.23	1	1	1.15	1.15	154.23	154.23	0.0000	0.0000
[83]: 70	42.42	42.48	4	4	4.04	4.04	148.58	148.68	0.2825	0.2837
[84]: 71	82.58	102.84	2	3	2.08	3.10	165.15	166.51	0.0002	0.7612
[85]: 72	60.36	61.08	2	3	2.06	3.06	101.08	101.08	0.2748	0.7039
[86]: 76	44.29	44.38	3	3	3.04	3.04	110.55	110.54	0.3491	0.3506
[87]: 80	67.30	89.95	2	3	2.07	3.09	134.49	134.11	0.0012	0.8766
[88]: 95	102.83	21.45	1	5	1.10	5.02	102.83	106.21	0.0000	0.0055

Table 9.2: Raw KPI Scores on Mini-batch Validation Set (Graph2Seq Recurrent Model)

Batch ID	Makespan Score		Activation Score		Weighted Score		TFT Score		CV Score	
	Opt	Pred	Opt	Pred	Opt	Pred	Opt	Pred	Opt	Pred
[0]: 107	43.56	43.56	3	3	3.04	3.04	130.39	130.39	0.0036	0.0037
[1]: 126	82.45	82.54	2	2	2.08	2.08	144.68	144.71	0.1976	0.1991
[2]: 142	66.42	66.42	2	2	2.07	2.07	132.84	132.84	0.0000	0.0000
[3]: 145	112.89	113.09	2	2	2.11	2.11	203.28	203.35	0.1565	0.1588
[4]: 156	66.25	66.46	2	2	2.07	2.07	131.50	131.48	0.0107	0.0155
[5]: 159	41.60	41.60	4	4	4.04	4.04	166.40	166.40	0.0000	0.0000
[6]: 15	186.72	186.73	1	1	1.19	1.19	186.72	186.73	0.0000	0.0000
...	...	...	...	...	...	...	...	...	...	...
[82]: 670	154.23	154.23	1	1	1.15	1.15	154.23	154.23	0.0000	0.0000
[83]: 70	42.42	42.51	4	4	4.04	4.04	148.58	148.78	0.2825	0.2835
[84]: 71	82.58	82.63	2	2	2.08	2.08	165.15	165.22	0.0002	0.0003
[85]: 72	60.36	60.72	2	2	2.06	2.06	101.08	101.08	0.2748	0.2849
[86]: 76	44.29	44.37	3	3	3.04	3.04	110.55	110.54	0.3491	0.3520
[87]: 80	67.30	67.30	2	2	2.07	2.07	134.49	134.47	0.0012	0.0014
[88]: 95	102.83	102.96	1	1	1.10	1.10	102.83	102.96	0.0000	0.0000

We discuss the performance of both neural architectures in depth. We also provide a comparative analysis of the optimality gaps using both visual curve alignments and exact

tabular results, highlighting the respective strengths of the one-shot and recurrent formulations in satisfying the aforementioned metrics. Therefore, we use the same metrics to measure the better-performing model with respect to the lower bounds (introduced in sec.7.2 of ch.7) on the same validation set.

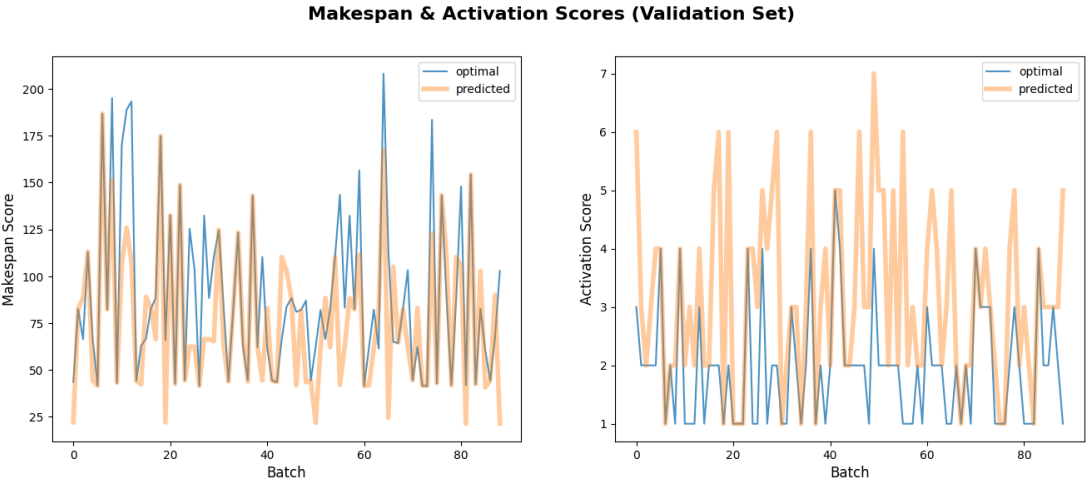
9.1 Mini-Batch Validation Results Analysis: Efficiency vs. Fairness

To assess the predictive quality of the trained architectures against the ground truth optimal solutions generated by the MIP solver, we evaluated the models on **mini-batch validation** instances. We compared both models across various benchmarking metrics (introduced in ch.7): primary objective metrics, and efficiency-oriented compared to fairness-oriented metrics. The predicted schedules were evaluated directly against the exact MIP solutions (see sec.3.5 in ch.3) to determine the quality of the neural approximation.

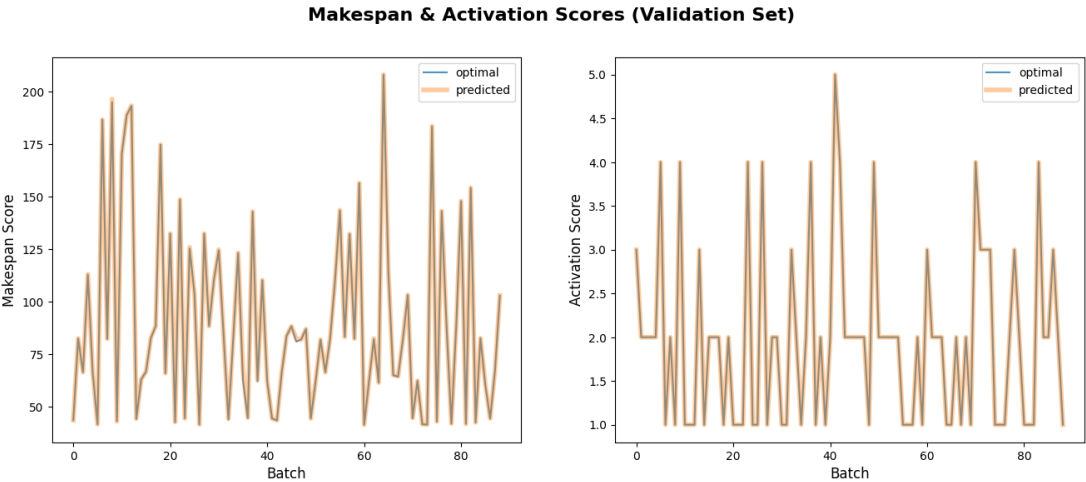
Primary Metrics Evaluation

The real-world business needs in warehouse scheduling are primarily driven by minimizing the overall makespan and the activation score (both defined in sec.7.1 of ch.7).

- **One-shot model performance:** as shown in fig. 9.1a, the one-shot model exhibits a moderate deviation from the optimal makespan, frequently overestimating the required time. Furthermore, the model consistently over-predicts the activation score, utilizing more operators than strictly necessary to complete the mini-batch missions. This conservative allocation strategy prevents task delays but sacrifices resource usage.
- **Recurrent model performance:** In contrast, the Graph2Seq recurrent model (fig. 9.1b) tracks the optimal makespan much more closely, preserving the tight bounds of the MIP solution. Even more notably, the recurrent model matches the optimal activation score almost perfectly across the entire validation set. The *autoregressive* decoding mechanism allows the model to continuously track the current load of active operators, successfully preventing unnecessary activations.



(a) Makespan & Activation Scores (One-Shot GNN Model)

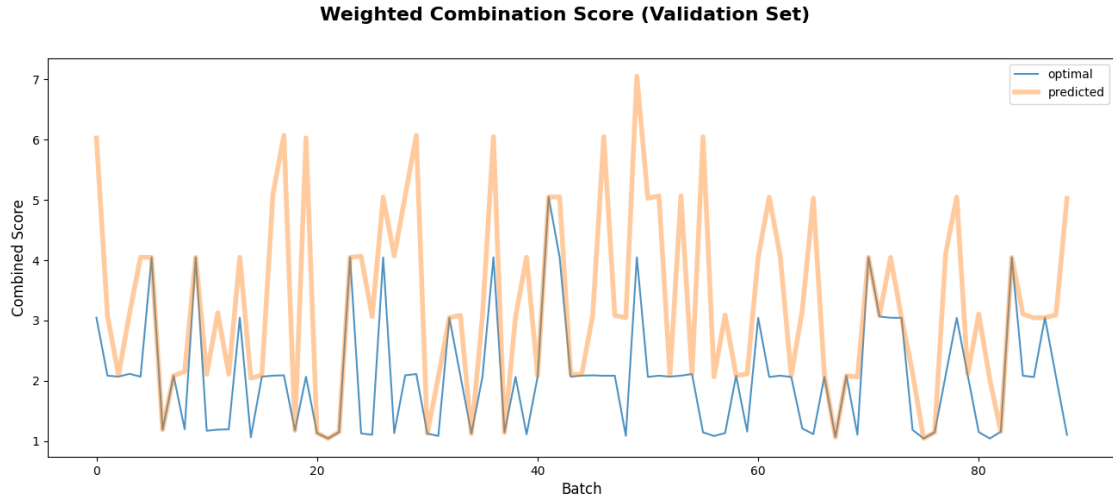


(b) Makespan & Activation Scores (Graph2Seq Recurrent Model)

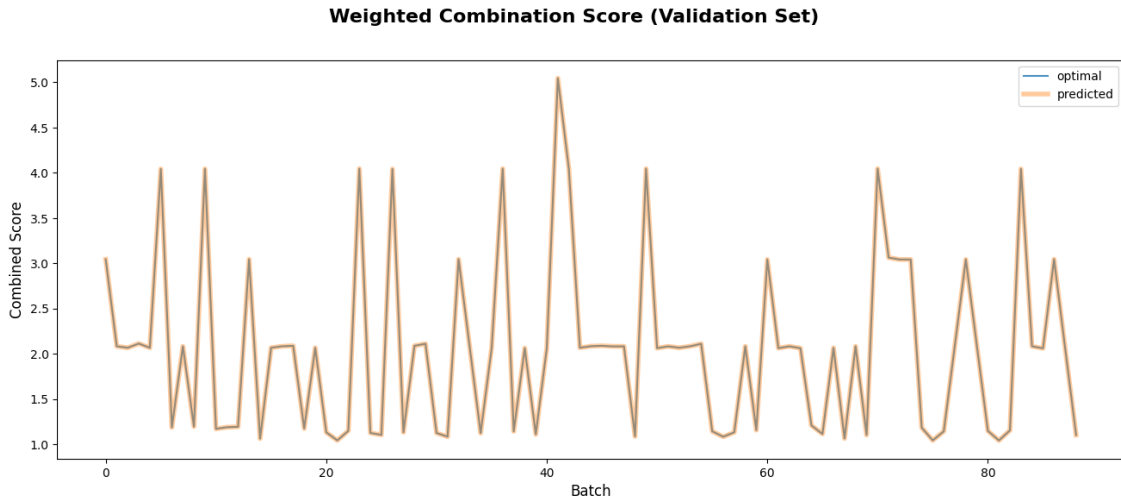
Figure 9.1: Makespan & Activation Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

Weighted Combination Score

The weighted combination score (defined in sec.7.1 of ch.7) aggregates the multi-objective efficiency criteria into a single scalar value, allowing for a standardized comparison between the predicted and optimal routing topologies.



(a) Weighted Combination Score (One-Shot GNN Model)



(b) Weighted Combination Score (Graph2Seq Recurrent Model)

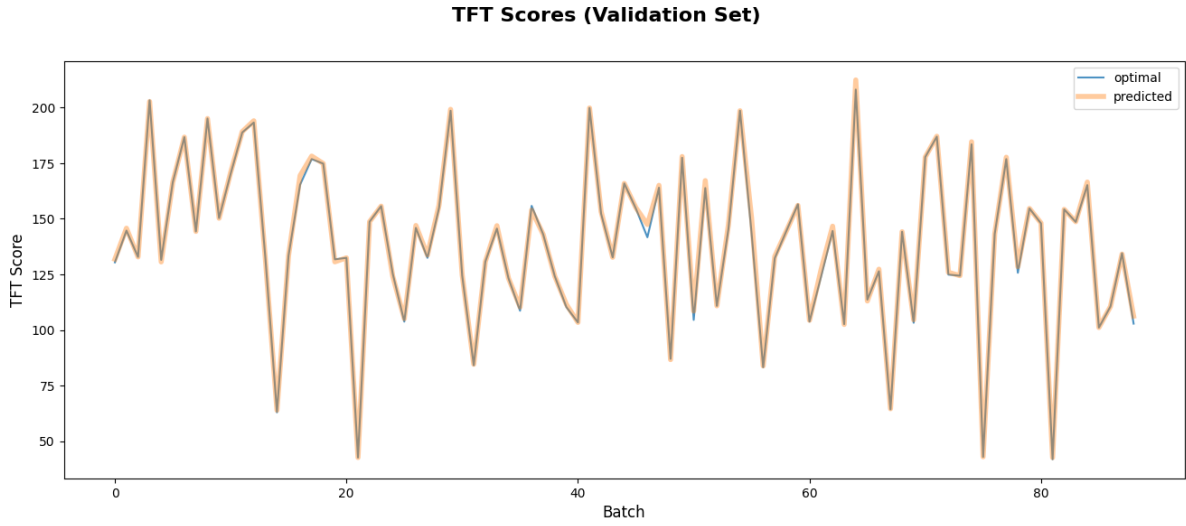
Figure 9.2: Weighted Combination Score Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

- **One-shot model:** Consistent with the individual primary metrics, the one-shot model's combined score (fig. 9.2a) exhibits noticeable over-prediction peaks. While it successfully captures the general trend of the validation batches, the lack of sequential context forces the model into suboptimal structural decisions, increasing the aggregate penalty.
- **Recurrent model:** The recurrent model (fig. 9.2b) demonstrates an exceptional fit to the optimal combined score. The predicted curve tightly wraps the optimal ground truth, confirming that the autoregressive formulation successfully learns the complex trade-off between makespan minimization and operator allocation embedded in the training data.

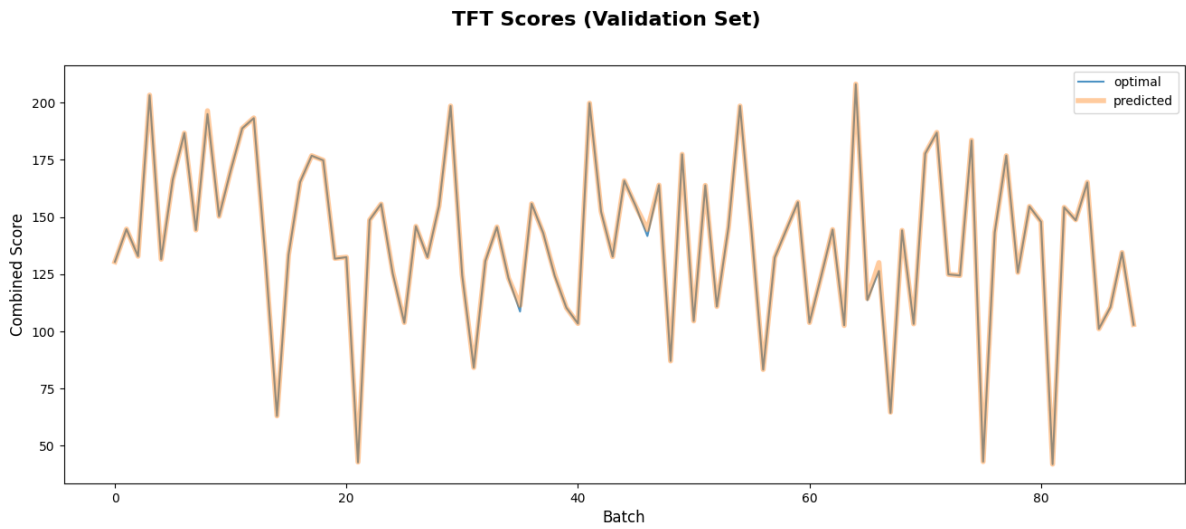
Efficiency and Fairness: TFT and CV Scores

Beyond raw business-view objectives, a high-quality schedule must ensure workload fairness among the activated operators training to maintain their overall efficiency. The overall efficiency can be measured by the TFT, which evaluates cumulative operational time, and the CV, which measures the unfairness of operator workloads using the standard deviation to the relative mean. (both metrics are defined in sec.7.1 of ch.7).

- **TFT score:** For the TFT metric, both the one-shot (fig. 9.3a) and recurrent (fig. 9.3b) architectures demonstrate near-perfect alignment with the optimal solutions. Because TFT is a structurally additive metric over the graph, both models successfully optimize the global sum of travel and operation times without requiring complex sequential dependencies.



(a) TFT Score (One-Shot GNN Model)



(b) TFT Score (Graph2Seq Recurrent Model)

Figure 9.3: TFT Score Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

- **CV Score:** The CV score captures the fairness of the resulting schedules by measuring how evenly the workload is distributed across the active operators. For the one-shot model (fig. 9.4a), the predicted CV curve shows larger deviations from the optimal baseline, with several pronounced spikes that indicate less balanced workload allocations in some batches.

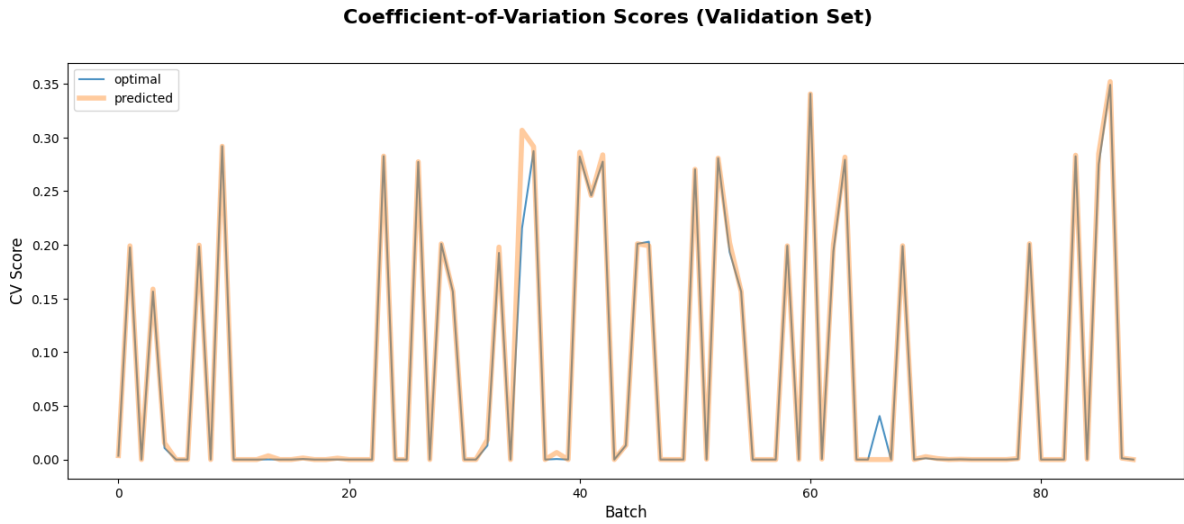
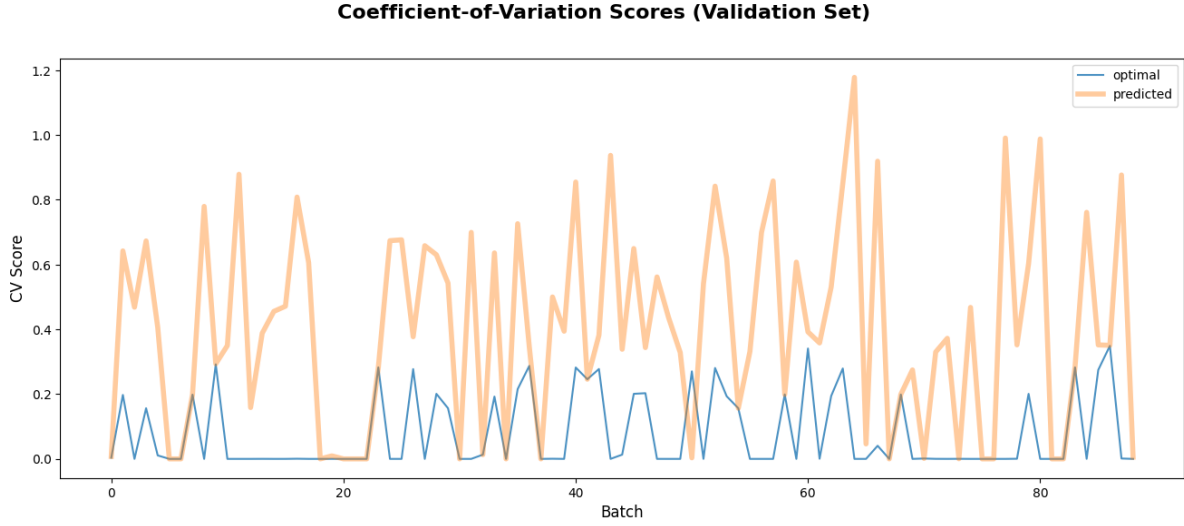


Figure 9.4: CV Score Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

By contrast, the recurrent model (fig. 9.4b) almost follows the optimal CV profile much more closely and with lower variability, suggesting a more equitable distribution of missions among operators. This tighter alignment indicates that the recurrent architecture not only preserves efficiency-oriented objectives, but also better captures the fairness structure of the MIP solutions, making it more reliable when balanced workload assignment is a priority.

Pareto Trade-off: Observations on the Relationship between TFT and CV

The analysis of the 66th mini-batch reveals a clear Pareto trade-off between routing efficiency, measured through TFT or makespan, and workload fairness, measured through the CV. The exact MIP solver aggressively minimizes TFT by concentrating tasks on the most suitable operators, thereby achieving a highly efficient schedule but inducing a greater imbalance in workload distribution. In contrast, the recurrent model introduces a penalty-based decoding strategy that encourages load balancing. As a result, it accepts a marginal increase in routing cost in exchange for a substantially fairer and more human-centric allocation of missions, reflected in a lower CV.

The **Pareto trade-off**, or Pareto frontier, is a fundamental concept in multi-objective optimization. It describes the set of solutions for which no objective can be improved without worsening at least one other objective. A solution on this frontier is said to be Pareto optimal. In this context, the inverse relationship observed between TFT and CV for the 66th batch is consistent with a Pareto trade-off, as documented in the multi-objective optimization literature [53].

From this perspective, the recurrent model does not simply approximate the MIP solution; rather, it identifies an alternative Pareto-optimal compromise in some cases that better reflects practical scheduling preferences in human-centered environments. While the MIP baseline achieves the mathematically minimum TFT, the recurrent model produces a schedule with slightly higher TFT but improved **fairness** through CV. Since both schedules use the same number of resources, the recurrent solution may be preferable in real-world operations where balanced workload distribution is valued alongside efficiency.

Why CV Was Not Included in the MIP Objective Function?

A natural question is why the CV was not directly incorporated into the exact MIP formulation as an additional optimization objective. The main reason is that CV is structurally much harder to optimize within a mixed-integer framework than the primary objectives considered in this work, namely makespan and activation.

First, CV is inherently non-linear because it depends on the standard deviation of operator workloads (as defined in eq.7.5 in ch.7). The CV definition immediately introduces a square root term, which is incompatible with a standard linear MIP objective. In contrast, makespan and activation can be expressed with linear variables and linear constraints (see sec.2.1 in ch.2), which is exactly the setting in which modern MIP solvers are most effective.

Second, CV is not only non-linear, but also fractional. The standard deviation appears in the numerator, while the mean workload appears in the denominator. This creates a rational objective function rather than a linear one. Once such a fractional expression is introduced, the model is no longer a classical MILP, but moves toward a non-linear mixed-integer program. That shift is highly undesirable from a computational perspective, especially in a scheduling problem that already has a constrained-combinatorial nature.

Third, introducing CV into the objective severely weakens the optimization landscape explored by the solver. The exact MIP used in this thesis relies on linear relaxations to compute strong lower bounds during B&B. When the objective is linear, these relaxations remain informative and help prune large portions of the search tree efficiently. By contrast, a CV-based objective introduces non-convex and ratio-based structure, making the relaxations much weaker and much less useful. In practice, this means the solver loses one of its main strengths: the ability to quickly certify high-quality bounds and eliminate suboptimal regions of the search space.

Fourth, this directly impacts B&B efficiency. The MIP solver can guarantee **optimality** for the originally defined primary objectives because those objectives are formulated in a way that preserves a tractable mixed-integer linear structure. Once CV is added, the solver would have to deal with a much more difficult class of problems, often requiring quadratic, conic, or general non-linear reformulations. These formulations tend to scale poorly, produce weaker bounds, and dramatically increase the number of nodes explored before convergence. For realistic warehouse instances, this would make exact optimization computationally impractical.

For these reasons, the MIP solver introduced in this work guarantees optimality

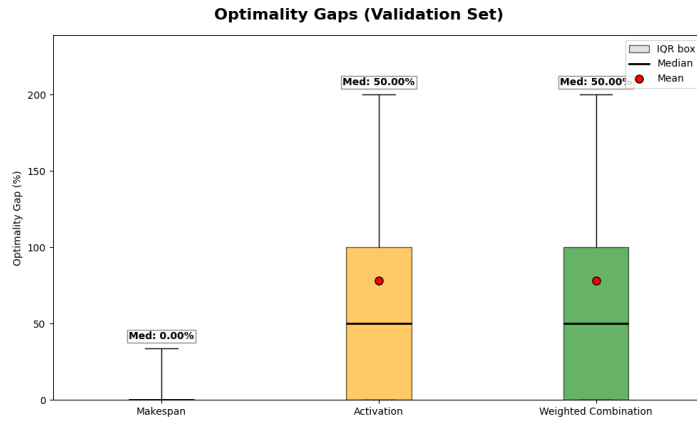
only with respect to the primary objectives that were explicitly encoded in the model (see sec.2.1 in ch.2). Workload balancing was not treated as a primary optimization target in that exact formulation.

Consequently, the MIP may sometimes produce a schedule with an excellent, or even optimal, CV as a by-product of optimizing makespan. This can happen because a well-balanced workload may indirectly help reduce completion time in some instances. However, this behavior is only incidental: since CV is not part of the objective function, the solver has no formal incentive to optimize it, and therefore no optimality guarantee can be claimed for workload balance.

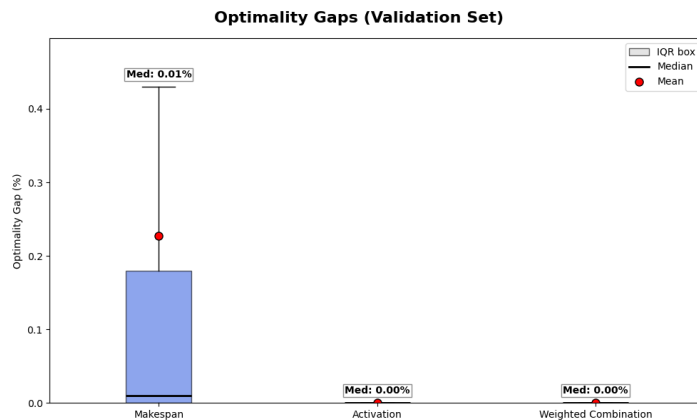
This distinction is important for interpreting the results of the neural models. When the learned scheduler achieves very low CV values, it is not simply reproducing something the MIP was already guaranteed to optimize. Instead, it is capturing an additional desirable property of the schedule that lies outside the scope of the exact primary-objective formulation. In other words, the *fairness* results should be viewed as complementary to the exact MIP guarantees, not as quantities for which the MIP provides a strict optimal benchmark.

9.2 Optimality Gap Analysis

To evaluate the quality and robustness of the neural schedules with respect to the MIP optimal solutions, the distribution of the optimality gaps was analyzed over the validation set for the primary objective metrics and the efficiency-fairness-related metrics. Figures 9.5a and 9.5b summarize the gaps for makespan, activation, and the weighted combination score, while figures 9.6a and 9.6b report the corresponding results for TFT and CV. Together, these figures provide a compact view of the median, dispersion, and outlier behavior of the gap distributions, enabling a direct comparison between the one-shot and recurrent architectures.



(a) Primary Objective Optimality Gaps (One-Shot GNN Model)



(b) Primary Objective Optimality Gaps (Graph2Seq Recurrent Model)

Figure 9.5: Primary Objective Optimality Gaps Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

Primary Objective Optimality Gaps

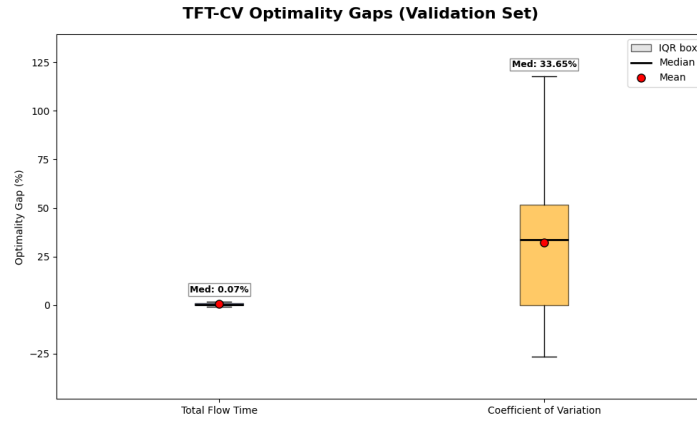
Fig. 9.5a shows that the one-shot model exhibits a wider spread of optimality gaps for the primary efficiency metrics. For makespan, the median gap remains low, but the distribution has a noticeable interquartile range and a visible upper tail, indicating that the model is not equally stable across all batches. The situation is more pronounced for activation and the weighted combination score with a median gap = 50%, where the central tendency is substantially higher and the spread is larger, revealing a less reliable approximation of the exact MIP solutions.

By contrast, fig. 9.5b shows that the recurrent model achieves much tighter and more favorable gap distributions. For makespan, activation, and the weighted combination score, the median gaps are essentially zero and the boxes are far more compact, which indicates that the recurrent decoder tracks the optimal MIP solutions more closely and more consistently across batches. This tighter concentration is the clearest evidence that the recurrent architecture is the more robust predictor for the main objectives.

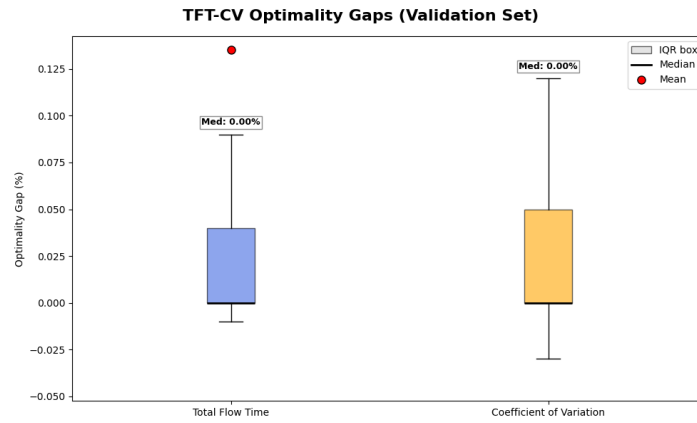
Efficiency and Fairness Optimality Gaps

The efficiency-to-fairness-oriented results in figures 9.6a and 9.6b tell a more nuanced story. For the one-shot model (fig. 9.6a), the median gap for TFT remains essentially zero, indicating that the model is often able to reproduce the optimal TFT values. However, the CV gap is markedly larger, with a median above 33% and a broad dispersion. This confirms that the one-shot architecture struggles to consistently balance operator workloads across all instances, and that fairness is only partially captured by its schedules.

Conversely, the recurrent model (fig. 9.6b) preserves the near-optimal behavior for both TFT and CV while exhibiting a noticeably tighter gap distribution. The interquartile range for the CV gap is significantly narrower compared with the one-shot model, meaning the recurrent decoder keeps efficiency and fairness in better balance with less batch-to-batch fluctuation. In practical terms, the recurrent model not only approaches the MIP solution more closely on the main objectives, but it also does so without degrading the fairness-related metrics.



(a) TFT & CV Optimality Gaps (One-Shot GNN Model)



(b) TFT & CV Optimality Gaps (Graph2Seq Recurrent Model)

Figure 9.6: Efficiency to Fairness Optimality Gaps Comparison: One-Shot GNN Model versus Graph2Seq Recurrent Model.

Model Comparison and Selection

In summary, the figures show that the recurrent model outperforms the one-shot model in terms of both accuracy and stability of the optimality gap across the complete KPI set. Its advantage is strongest for the primary efficiency metrics, where it achieves lower medians and reduced spread, but it also maintains robust behavior for TFT and CV. This balance makes the recurrent architecture the more reliable neural solver for the problem under study. Therefore, from this point onward, the **recurrent** model is retained as the *final* neural architecture, since it demonstrated the most reliable optimality-gap behavior across the complete set of KPIs.

9.3 Recurrent Model: Per-Horizon Optimality Gaps Analysis

Having established the recurrent model as the preferred neural architecture, we now analyze its performance across different time horizons (60, 90, 120, and 480 minutes) to understand how the scheduling difficulty scales with horizon limit. Figures 9.7 and 9.8 display the optimality gaps for the primary objective metrics (makespan, activation, weighted combination) and the efficiency-to-fairness metrics (TFT, CV), respectively, separated by time horizon.

Per-Horizon Primary Objective KPIs

As shown in fig. 9.7, the recurrent model maintains near-perfect performance for activation and weighted combination scores across all time horizons. For these two KPIs, the median gaps are consistently 0.00%, and the interquartile ranges (IQR) show virtually no spread, indicating that the model accurately predicts the required operator counts regardless of the scheduling window length.

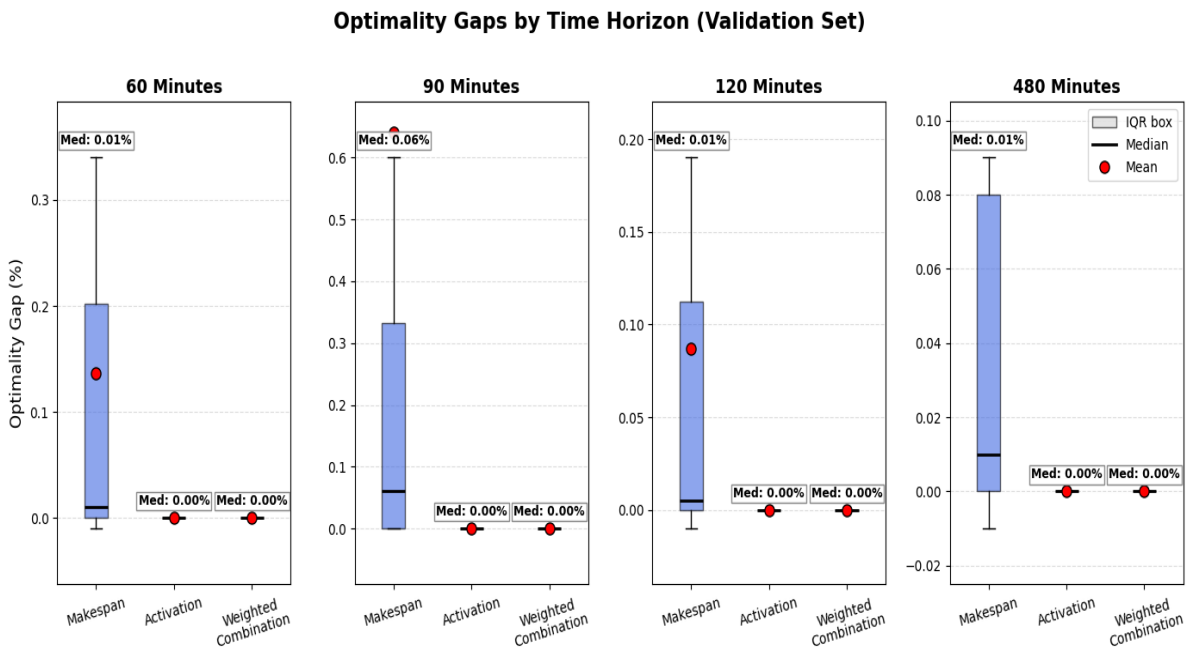


Figure 9.7: Recurrent Model: Per-Horizon Optimality Gaps on Primary Objective KPIs.

However, the makespan metric reveals horizon-dependent variability. The 90-minute horizon presents the most significant challenge for makespan optimization. While its median gap is still relatively low at 0.06%, it exhibits the highest dispersion among all horizons, with the upper quartile reaching roughly 0.33% and the maximum whisker extending near 0.6%. Interestingly, for the other horizons (60, 120, and 480 minutes), the median makespan gap drops to 0.01%, and the IQR is noticeably tighter. The 480-minute horizon, despite being the longest scheduling window, shows a very compact distribution, suggesting that the model scales well to larger time limits once past the intermediate bottleneck observed at 90 minutes, since we have a large maximum whisker $\sim 35\%$ also in the 60-minute horizon.

Per-Horizon Efficiency and Fairness KPIs

Figure 9.8 illustrates the model's performance on efficiency-oriented and fairness-oriented KPIs across the same horizons. For TFT, the model performs exceptionally well across the board. The median gaps are at or near 0.00% for all horizons. While the 480-minute horizon shows a slightly wider boxplot for TFT compared to the shorter horizons (with the upper quartile near 0.08%), the overall deviation from optimality remains remarkably small.

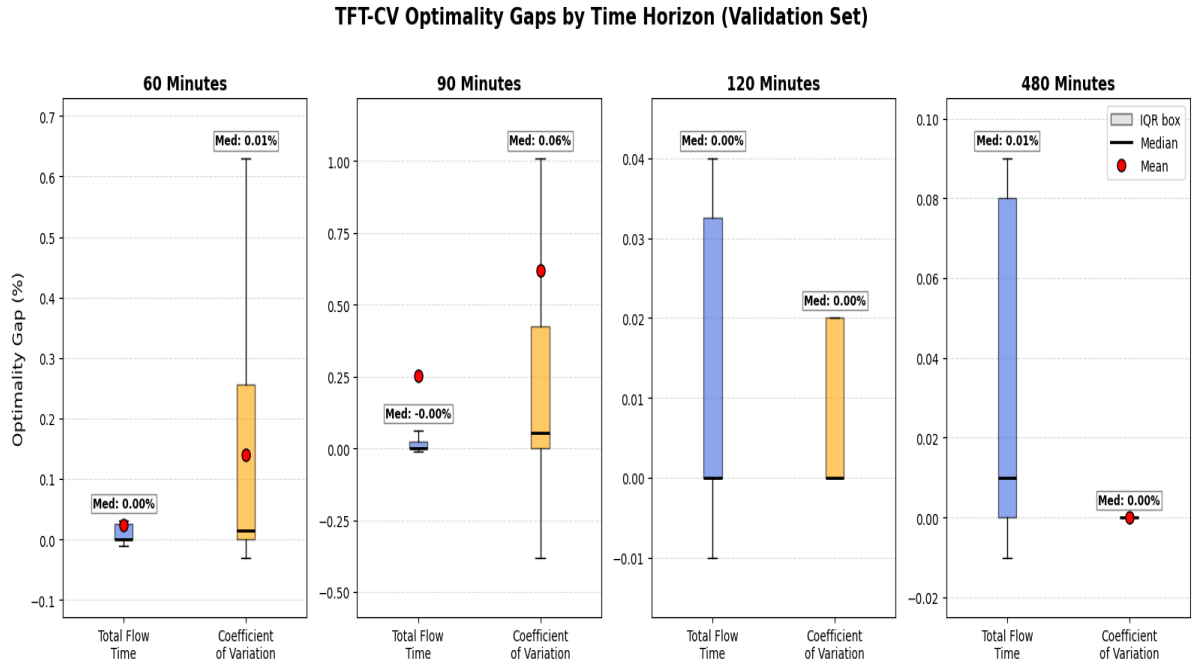


Figure 9.8: Recurrent Model: Per-Horizon Optimality Gaps on Efficiency and Fairness KPIs.

The CV, which measures workload balance, is the most sensitive metric to horizon length. Consistent with the makespan results, the 90-minute horizon stands out as an anomaly. While the median CV gaps remain low for 60, 120, and 480 minutes, the 90-minute horizon shows a median gap of 0.06% and a much larger spread, with the upper whisker reaching 1.00% and the mean pulled upward to approximately 0.6%. The 60-minute horizon also shows some similar dispersion in CV, but it is still substantially milder than the 90-minute case. By contrast, the 480-minute horizon again appears more stable, with the CV distribution tightly concentrated around the zero median.

One possible explanation for this pattern is that the random distribution of time horizons across mini-batches may have concentrated more difficult instances in the 90-minute batches, whereas this effect would be less visible at larger horizons, where the validation distributions appear more stable. This interpretation should be treated cautiously, but it provides a plausible explanation for why the intermediate horizon behaves differently from the others.

This behavior reinforces a key operational finding: **low-intermediate** scheduling windows, especially around 90 minutes, appear to present the most difficult structural challenges for the recurrent model. While the model reliably predicts the required number of operators across all horizons, it is less consistent at limited horizons in balancing workloads, as reflected by CV, and in minimizing completion time, as reflected by makespan, relative to the exact MIP solver.

Interpretation of Time Horizon Effects

The reduction in optimality gaps at larger time horizons does not necessarily imply that longer-horizon instances are intrinsically easier. A more plausible explanation is that the observed pattern results from the interaction between instance composition and the behavior of the recurrent decoder. In particular, the 90-minute horizon may have received a relatively higher concentration of difficult mini-batches due to the random horizon assignment, which would partially explain the limited larger gaps observed for makespan and CV.

At the same time, the recurrent architecture itself may benefit from less restrictive horizon settings. When the horizon is very tight, small local sequencing or assignment errors can immediately propagate into worse completion times and less balanced workloads. Under longer horizons, the decoder has more flexibility to absorb such local inefficiencies, allowing the schedule to recover and keeping the final KPI gaps closer to zero; considering also that at the same mini-batch's size, there won't be the absolute need to active

more number of operators that would be activated under lower time horizon. Therefore, the improved behavior at 120 and 480 minutes is likely driven by both the distribution of instance difficulty and the greater tolerance of longer-horizon schedules to small prediction errors, having lower number of operators to activate on a limited mini-batch size.

9.4 Recurrent Model: Lower Bound Gaps Analysis

While comparisons against the exact MIP solver provide insight into the model’s accuracy on solvable mini-batches, analyzing performance purely relative to MIP solutions becomes computationally intractable as problem sizes increase. Therefore, we introduce a theoretical lower bound (LB) as a stable, scale-invariant baseline (see sec.7.2 of ch.7). The theoretical lower bound represents an ideal mathematical threshold that does not guarantee practical feasibility, meaning it ignores real-world routing combinatorial nature and complex operator-order constraints. However, because it can be computed instantly regardless of problem scale, it serves as a crucial **bridge** baseline, enabling multi-scale comparisons and validating the model’s generalization capabilities on both mini-batch and full-batch instances (explained into the next chapter) [40, 54].

Table 9.3: Raw KPI Scores on Mini-batch Validation Set (Lower Bound)

Batch ID	Makespan Score		Activation Score		Weighted Score		TFT Score		CV Score	
	Min	Pred	Min	Pred	Min	Pred	Min	Pred	Min	Pred
[0]: 107	52.42	43.56	2	3	3.04	2.05	104.85	130.39	0.0000	0.0037
[1]: 126	65.05	82.54	2	2	2.08	2.06	130.11	144.71	0.0000	0.1991
[2]: 142	57.19	66.42	2	2	2.07	2.06	114.37	132.84	0.0000	0.0000
[3]: 145	89.06	113.09	2	2	2.11	2.09	178.11	203.35	0.0000	0.1588
[4]: 156	56.48	66.46	2	2	2.07	2.06	112.96	131.48	0.0000	0.0155
[5]: 159	44.85	41.60	3	4	4.04	3.04	134.55	166.40	0.0000	0.0000
[6]: 15	180.85	186.73	1	1	1.19	1.18	180.85	186.73	0.0000	0.0000
...	...	...	...	...	...	...	...	...	...	...
[82]: 670	148.04	154.23	1	1	1.15	1.15	148.04	154.23	0.0000	0.0000
[83]: 70	40.20	42.51	3	4	4.04	3.04	120.59	148.78	0.0000	0.2835
[84]: 71	74.23	82.63	2	2	2.08	2.07	148.45	165.22	0.0000	0.0003
[85]: 72	89.58	60.72	1	2	2.06	1.09	89.58	101.08	0.0000	0.2849
[86]: 76	44.43	44.37	2	3	3.04	2.04	88.86	110.54	0.0000	0.3520
[87]: 80	57.35	67.30	2	2	2.07	2.06	114.70	134.47	0.0000	0.0014
[88]: 95	93.06	102.96	1	1	1.10	1.09	93.06	102.96	0.0000	0.0000

To evaluate how close the recurrent model approaches this ideal limit, we report the LB curves (fig. 9.9) and gaps also for the same primary objective KPIs and efficiency-fairness KPIs. Since the LB represents an optimistic idealization rather than a feasible schedule, a 0% gap indicates perfect theoretical matching, though practically, optimal feasible schedules usually exhibit some positive gap from the LB.

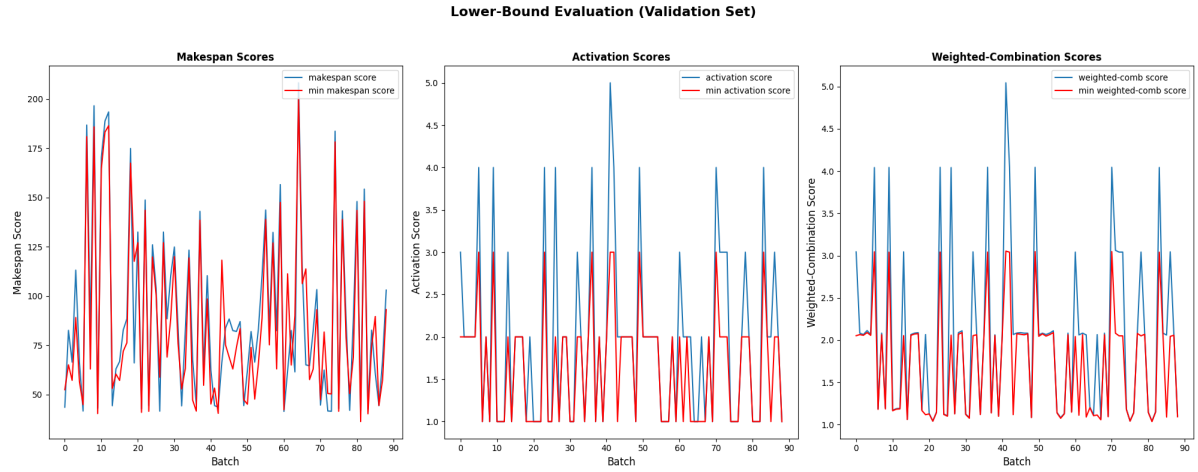


Figure 9.9: Primary Objective Lower-Bound KPIs (Recurrent Model)

Primary Objective Lower Bound Gaps

Figure 9.10 shows the LB gap distribution. A critical characteristic of the LB framework is the intrinsic trade-off between makespan and activation: the theoretical lower bound cannot be outperformed simultaneously in both metrics.

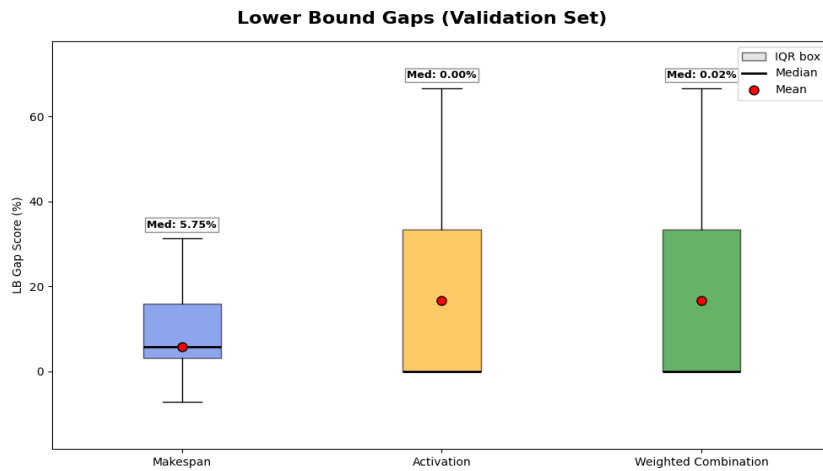


Figure 9.10: Lower Bound Gaps on Primary Objective KPIs

If the recurrent model achieves an unusually strong (or negative) makespan gap at a specific point on the curve (fig. 9.11), it implies the schedule completed tasks faster than the ideal makespan. However, this is only possible if the model utilized more operators than strictly necessary, which will be reflected as a penalty (positive gap) in the corresponding activation curve. Thus, a schedule might seem to "beat" the theoretical makespan bound, but this simply indicates a correspondent over-allocation of resources relative to the minimal ideal activation threshold.

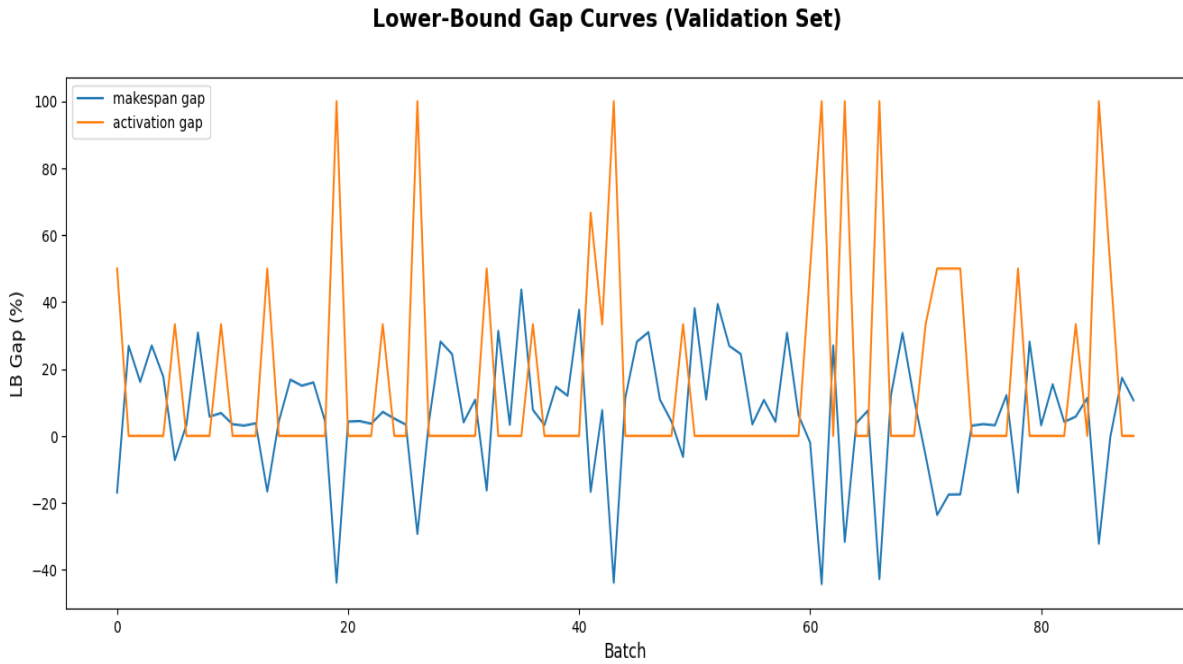


Figure 9.11: Trade-off: Makespan and Activation Lower Bound Gap Curves

Across the validation set, the recurrent model exhibits a median makespan gap of 5.75% against the lower bound. This positive gap is a natural consequence of the model adhering strictly to the activation bounds in the majority of instances. Concurrently, the activation gap median sits perfectly at 0.00%, with only moderate variations (the mean sits near 16%), demonstrating that the model decoding generally respects the minimum workforce threshold without excessive over-hiring. Because the model prioritizes workforce minimization, the completion time naturally stretches slightly beyond the theoretical minimum, resulting in the 5.75% median makespan gap. The overall weighted combination score reflects this balanced trade-off, maintaining an exceptionally tight median gap of 0.02%.

Efficiency and Fairness Lower Bound Gaps

We further evaluate the model against the theoretical minimums for TFT and CV. As shown in fig. 9.12, the model achieves a median TFT gap of 12.18%.

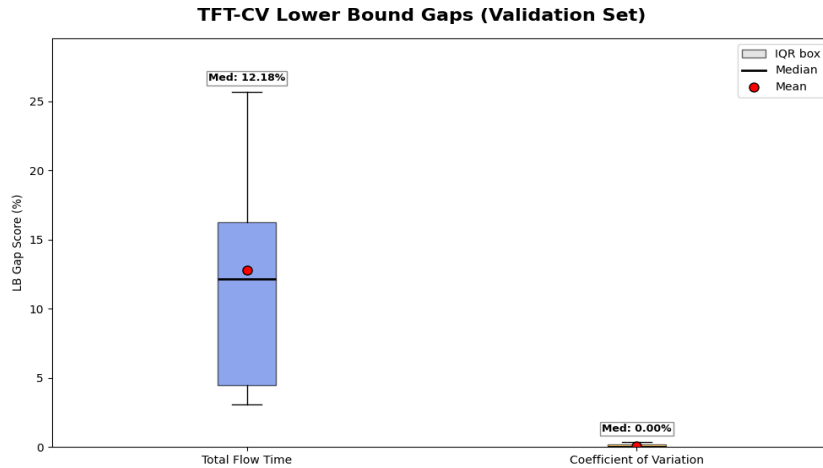


Figure 9.12: Lower Bound Gaps on Efficiency and Fairness KPIs

This positive gap is expected: the theoretical TFT lower bound assumes perfect, conflict-free routing for every order. In feasible reality, the recurrent decoder must route operators to avoid collisions and respect precedence constraints, naturally incurring routing delays and idle times that push the TFT roughly 12% above the optimistic theoretical ideal.

Finally, because the ideal theoretical scenario assumes workload can be divided perfectly continuously across operators, the absolute minimum CV is exactly zero. Our recurrent model achieves a median CV gap of 0.00%, meaning that in the majority of validation instances, the neural decoder perfectly balances the assigned tasks among the active workforce, closely tracking the theoretical ideal despite the discrete, combinatorial nature of the actual scheduling environment.

10. Full Problem Experiments and Qualitative Analysis

In the previous chapters, we evaluated the neural scheduling models on mini-batch instances where the exact MIP solver could reliably provide optimal ground-truth solutions of limited scale. However, the true operational value of a neural scheduler lies in its ability to generalize to large-scale, full-problem environments where traditional exact methods become computationally intractable.

In this chapter, we evaluate the recurrent model on the full test set. Because the MIP solver cannot compute optimal solutions for these massive instances within a reasonable time, we cannot rely on direct optimality gaps. Instead, we establish a theoretical lower bound (LB) framework to serve as a scale-invariant baseline. By comparing the LB gaps obtained on the validation set (small scale) with the LB gaps obtained on the test set (large scale), we can rigorously demonstrate the model’s generalization capabilities. Following the theoretical lower bound analysis, we then compare the recurrent model’s performance against a traditional heuristic greedy baseline to highlight the practical advantages of the neural approach in large-scale scenarios.

The theoretical lower bound represents an idealized mathematical limit for the scheduling metrics, calculated by relaxing complex operational constraints of the combinatorial nature (formally defined in [sec.7.2](#) of [ch.7](#)).

10.1 Recurrent Model: Test Set Lower Bound Gaps Analysis

Before comparing across scales, we first analyze the model’s performance relative to the theoretical lower bounds purely on the full-scale test set.

Table 10.1: Raw KPI Scores on Full-batch Test Set (Lower Bound)

Batch ID	Makespan Score		Activation Score		Weighted Score		TFT Score		CV Score	
	Min	Pred	Min	Pred	Min	Pred	Min	Pred	Min	Pred
[0]: 28	376.71	389.87	17	18	17.38	18.39	6404.13	6854.62	0.0000	0.0270
[1]: 28_1	376.71	389.82	17	18	17.38	18.39	6404.13	6855.12	0.0000	0.0230
[2]: 28_2	376.71	389.62	17	18	17.38	18.39	6404.13	6849.65	0.0000	0.0236
[3]: 28_3	376.71	387.96	17	18	17.38	18.39	6404.13	6853.80	0.0000	0.0202
[4]: 28_4	376.71	389.04	17	18	17.38	18.39	6404.13	6853.73	0.0000	0.0223
[5]: 29	357.67	348.79	23	26	23.36	26.35	8226.50	8833.64	0.0000	0.0290
[6]: 29_1	357.67	349.25	23	26	23.36	26.35	8226.50	8839.05	0.0000	0.0263
...	...	...	...	...	...	...	...	...	...	...
[44]: 31	469.82	479.34	34	36	34.47	36.48	15973.86	17030.96	0.0000	0.0074
[45]: 31_1	469.82	479.93	34	36	34.47	36.48	15973.86	17047.18	0.0000	0.0079
[46]: 31_2	469.82	479.88	34	36	34.47	36.48	15973.86	17057.81	0.0000	0.0103
[47]: 31_3	469.82	479.21	34	36	34.47	36.48	15973.86	17015.67	0.0000	0.0100
[48]: 31_4	469.82	479.28	34	36	34.47	36.48	15973.86	16997.02	0.0000	0.0101
[49]: 31_5	469.82	479.35	34	36	34.47	36.48	15973.86	17008.96	0.0000	0.0083

Figure 10.1 illustrates the distribution of these gaps for the primary objective KPIs. On the full test set, the median makespan gap narrows to -1.06% , while the median activation gap registers at 12.50% .

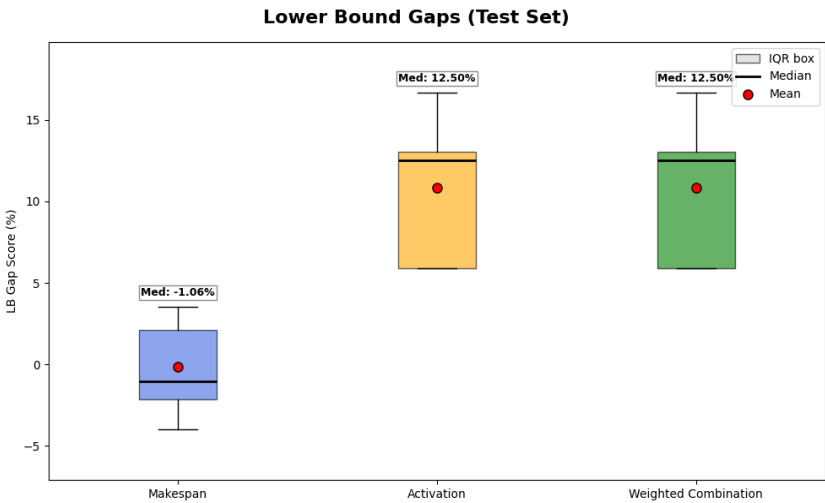


Figure 10.1: Test Set Lower Bound Gaps on Primary Objective KPIs

This shift reflects the strict mathematical trade-off inherent in the LB framework: completing a massive schedule slightly faster than the theoretical makespan minimum requires a minor over-allocation of resources relative to the mathematically absolute minimum activation. Despite this internal trade-off, the overall schedule efficiency remains remarkably balanced, yielding a median weighted combination gap of 12.50%.

The robustness of the recurrent model extends to the efficiency and fairness KPIs: TFT and CV.

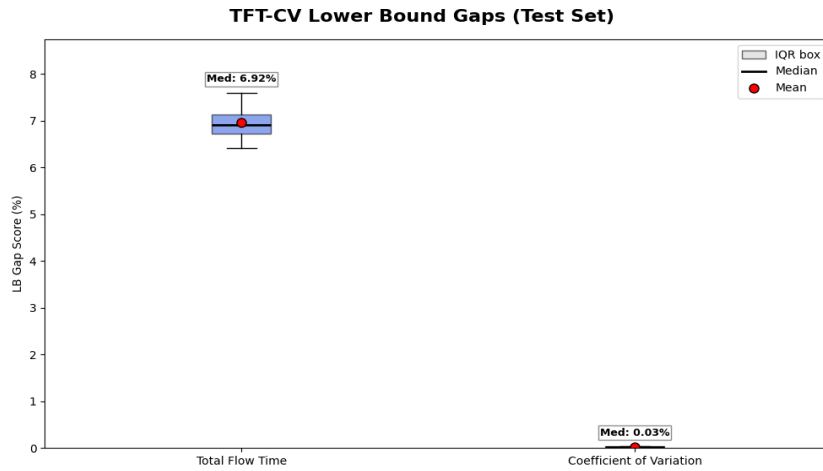


Figure 10.2: Test Set Lower Bound Gaps on Efficiency and Fairness KPIs

As shown in fig. 10.2, the median CV gap is a near-perfect 0.03%. This indicates that the recurrent decoder’s ability to perfectly balance workloads among the active workforce is exceptionally strong, even in a full-scale environment. For the TFT metric, the model achieves a median gap of 6.92%. This positive gap is an expected consequence of routing operators around realistic physical collisions—which the theoretical LB ignores—confirming that the model’s routing logic functions effectively without severe delays as the warehouse becomes more congested.

10.2 Generalization via Validation-Test Lower Bound Gaps

Since the LB can be computed instantly regardless of problem scale. This makes the LB a powerful scale-invariant baseline [40]. If the neural model maintains similar relative gaps to the LB on both the validation mini-batches and the full test instances

(e.g., $Gap_{LB} \leq 5\%$ for excellent generalization, $5\% < Gap_{LB} \leq 10\%$ for acceptable generalization), we can conclude that the model generalizes robustly to the full-scale problem without performance degradation.

To evaluate model's **generalization** capability, we directly compare the LB gaps recorded on the *mini-batch validation* set (calculated in sec.9.4 of the previous chapter) with the corresponding gaps generated on the *full-batch test* set.

Validation-Test Lower Bound Gaps on Primary Objective KPIs

Figure 10.3 illustrates the alignment of these gaps for the primary objectives. The median gap for the weighted combination score demonstrates remarkable stability, measuring 5.88% on the validation set and maintaining an identical 5.88% median during portions of the multi-horizon testing (as shown in subsequent horizon breakdowns). This confirms that the overall quality of the schedules generated by the recurrent model does not fundamentally degrade when faced with significantly larger problem spaces.

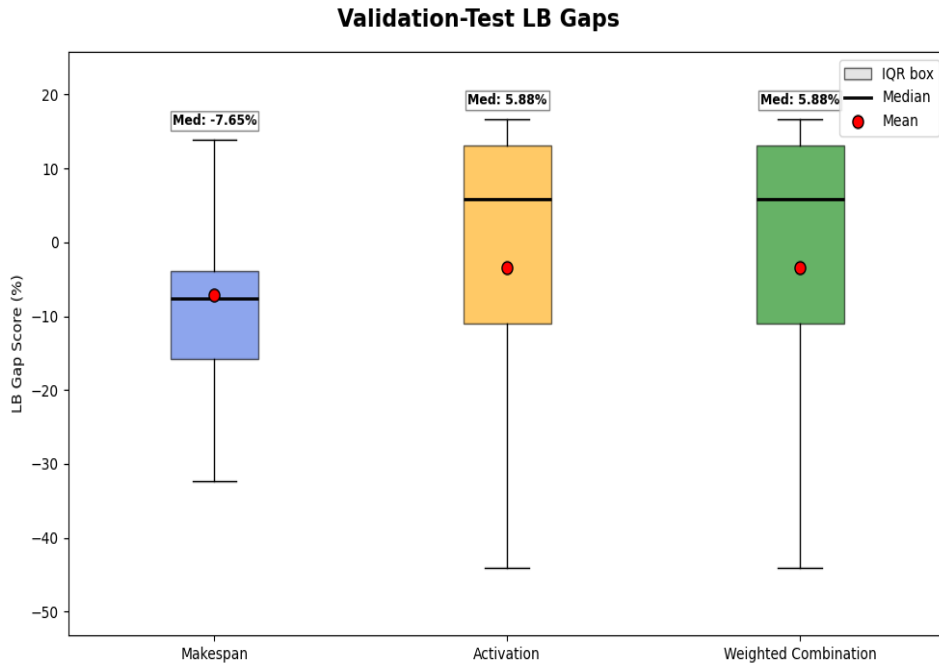


Figure 10.3: Validation vs. Test Lower Bound Gaps (Primary Objective KPIs)

Validation-Test Lower Bound Gaps on Efficiency and Fairness KPIs

The robustness of the recurrent model extends to the fairness and efficiency metrics.

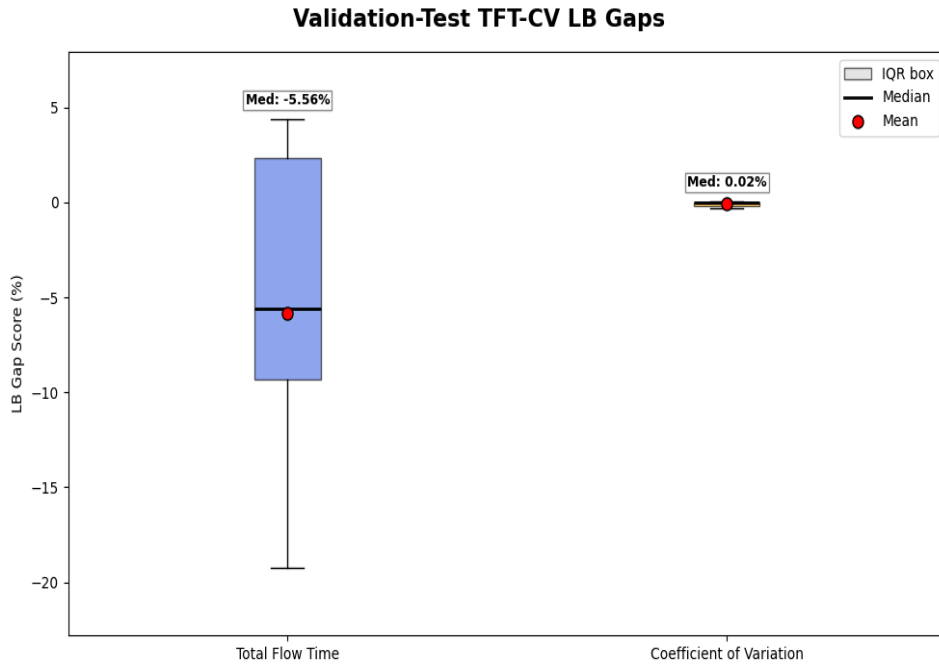


Figure 10.4: Validation vs. Test Lower Bound Gaps (Efficiency and Fairness KPIs)

As shown in figure 10.4, the model demonstrates excellent generalization on these metrics. The median CV gap remains absolutely fixed at a near-perfect 0.02% for both the validation and test sets. This indicates that the recurrent decoder’s ability to perfectly balance workloads among the active workforce is entirely unaffected by the size of the schedule.

For the TFT metric, the model actually shows a slight improvement at scale, moving from a validation median gap of $\sim 12.18\%$ (see fig. 9.12 in ch.9) down to a test median gap of 6.92% (fig. 10.2). The negative TFT LB gap near -5.56% reflects such improvement across larger scales.

10.3 Recurrent Model: Per-Horizon Lower Bound Gaps Analysis (Full Batch)

To ensure the recurrent model handles different temporal constraints efficiently at scale, we analyze the test set lower bound gaps separated by time horizons (360, 390, 420, and 480 minutes). This breakdown confirms whether the neural model maintains its generalization capabilities regardless of how tight or loose the scheduling window becomes.

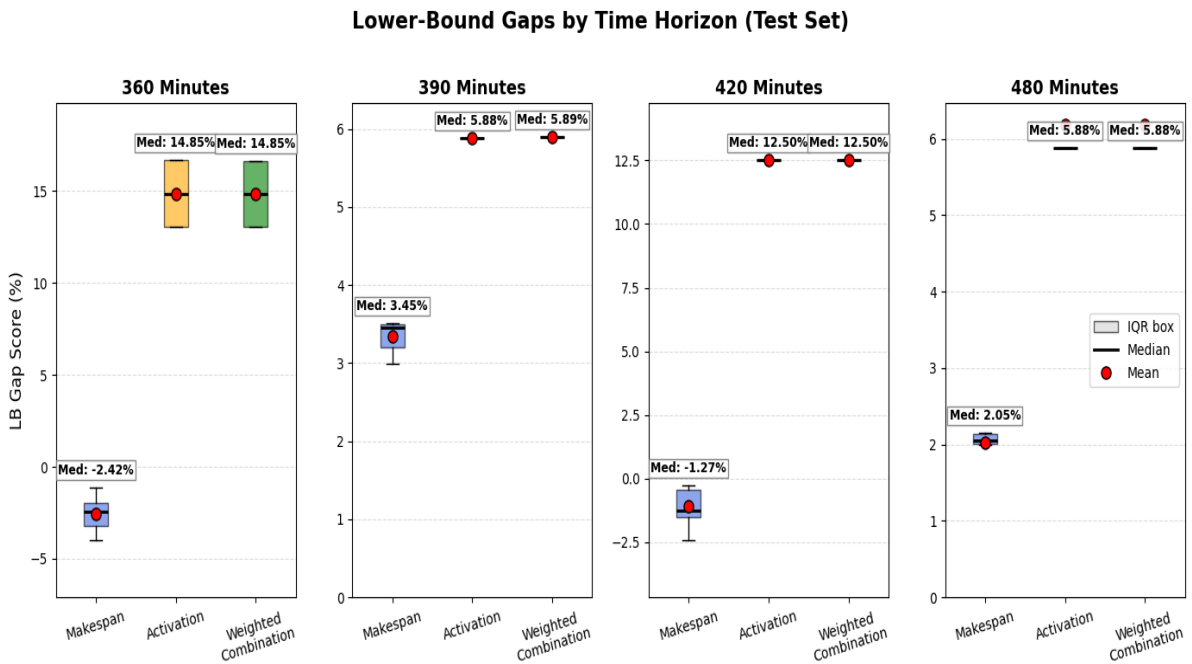


Figure 10.5: Per-Horizon LB Gaps on Primary Objective KPIs

As depicted in fig. 10.5, the weighted combination gap remains extremely stable and narrow across the longer horizons, registering medians of 5.89%, 12.50%, and 5.88% for the 390, 420, and 480-minute horizons, respectively. The 360-minute horizon shows a slightly higher median weighted combination gap (14.85%) alongside a makespan gap of -2.42% . This follows the behavior observed in the mini-batch phase: tighter horizons force the model to aggressively parallelize tasks, slightly inflating the activation gap to meet the strict makespan limit. Conversely, the longest horizon (480 minutes) allows the schedule to loose through the extended time limit per operator, resulting in a tightly compacted makespan gap of 2.05% and an activation gap of 5.88%. The model clearly scales its internal trade-off logic efficiently according to the requested horizon.

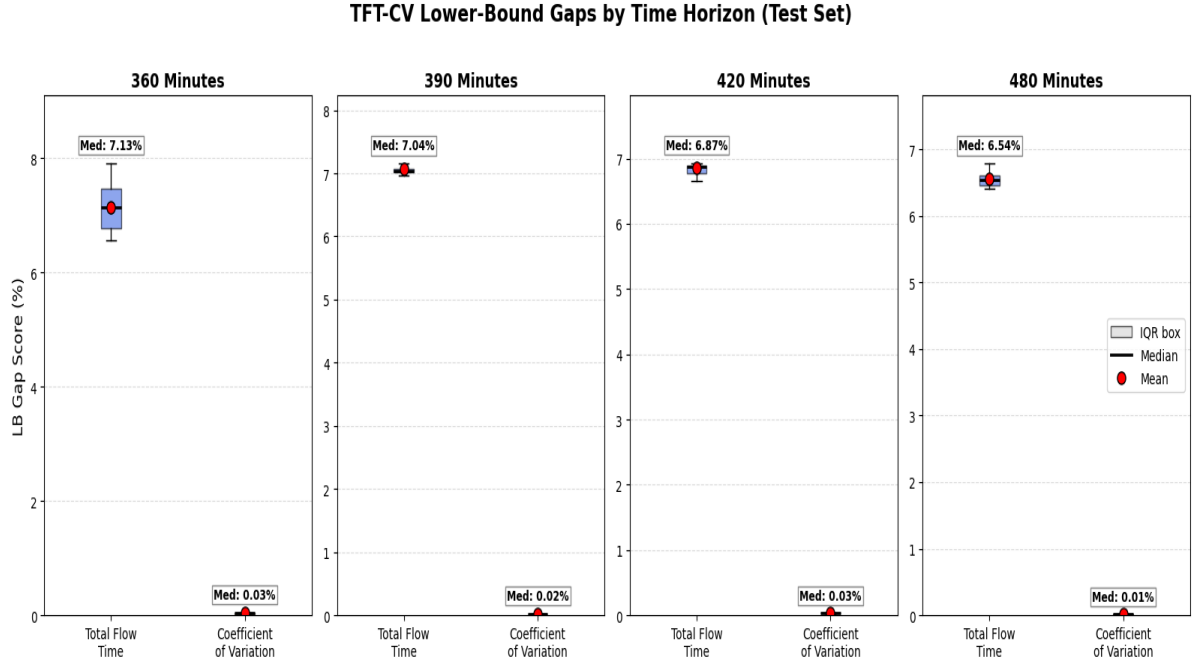


Figure 10.6: Per-Horizon LB Gaps on Efficiency and Fairness KPIs

A similar, robust stability is observed in the fairness metrics across horizons (fig. 10.6). The CV median gap remains tightly locked between 0.01% and 0.03% for all horizons, proving that the model maintains invariant fairness and perfectly balanced workloads regardless of the time limit. The TFT median gap also remains tightly bounded between 6.54% and 7.13% across all four time limits, confirming that the recurrent model generates balanced, highly efficient flow times even under varying degrees of temporal pressure in a full-scale environment.

10.4 Heuristic Greedy Baseline Comparison

While the LB analysis proves the recurrent model’s scale-invariant mathematical efficiency, the LB remains an idealized, unachievable boundary which doesn’t consider the sophisticated routing feasibility at all. To evaluate the model’s practical utility against a realistic standard, we must compare it to a feasible scheduling baseline. Because the full-scale instances are too massive for the exact MIP solver, we utilize a heuristic greedy baseline (defined in sec.6.3 of ch.6). This algorithm sequentially assigns operators to tasks prioritizing immediate availability maintaining system’s primary objective (defined in sec.2.1 of ch.2), ensuring strict routing feasibility and hard constraint satis-

faction without requiring extensive search time or considering the combinatorial nature of the COP. The gaps reported in this section represent the percentage improvement (negative gaps) or degradation (positive gaps) of our neural model relative to this greedy baseline.

Table 10.2: Raw KPI Scores on Full-batch Test Set (Heuristic Greedy Baseline)

Batch ID	Makespan Score		Activation Score		Weighted Score		TFT Score		CV Score	
	Greedy	Pred	Greedy	Pred	Greedy	Pred	Greedy	Pred	Greedy	Pred
[0]: 28	382.33	389.87	66	18	66.32	18.39	8095.19	6854.62	0.5505	0.0270
[1]: 28_1	379.86	389.82	49	18	49.33	18.39	8265.59	6855.12	0.3711	0.0230
[2]: 28_2	378.77	389.62	39	18	39.34	18.39	8383.95	6849.65	0.2822	0.0236
[3]: 28_3	381.24	387.96	60	18	60.32	18.39	8136.23	6853.80	0.4706	0.0202
[4]: 28_4	379.01	389.04	50	18	50.33	18.39	8258.38	6853.73	0.3220	0.0223
[5]: 29	359.91	348.79	148	26	148.21	26.35	17357.72	8833.64	0.6390	0.0290
[6]: 29_1	359.49	349.25	131	26	131.23	26.35	17534.13	8839.05	0.6359	0.0263
...	...	...	...	...	...	...	...	...	...	...
[44]: 31	471.18	479.34	122	36	122.35	36.48	31188.15	17030.96	0.3051	0.0074
[45]: 31_1	472.79	479.93	111	36	111.36	36.48	31311.96	17047.18	0.2631	0.0079
[46]: 31_2	472.08	479.88	117	36	117.35	36.48	31237.56	17057.81	0.3054	0.0103
[47]: 31_3	470.44	479.21	118	36	118.35	36.48	31224.47	17015.67	0.3008	0.0100
[48]: 31_4	472.78	479.28	115	36	115.36	36.48	31251.32	16997.02	0.2889	0.0101
[49]: 31_5	473.59	479.35	117	36	117.36	36.48	31266.49	17008.96	0.2863	0.0083

Recurrent Model to Greedy Baseline Score Curves

Before analyzing baseline gaps, we first inspect the raw score curves produced by the heuristic greedy baseline and the recurrent model on the test set. This step is important because, unlike the mini-batch setting where the exact MIP solver provides an optimal reference, the full-scale problem does not admit a realistic optimal baseline. Instead, the greedy heuristic serves as a feasible but limited comparator: it respects routing feasibility and hard time-horizon constraints, yet it makes *local* decisions without global foresight.

Figure 10.7 shows that the recurrent model closely tracks the greedy baseline in makespan while using substantially fewer operators. The makespan curves follow the same batch-level regime changes, which indicates that the neural policy preserves the baseline’s feasibility structure and does not sacrifice completion time. In contrast, the activation curve reveals the main weakness of the heuristic: the greedy baseline reacts sharply to workload changes and repeatedly overshoots the operator requirement, whereas the recurrent model remains much flatter and closer to the minimum feasible staffing level.

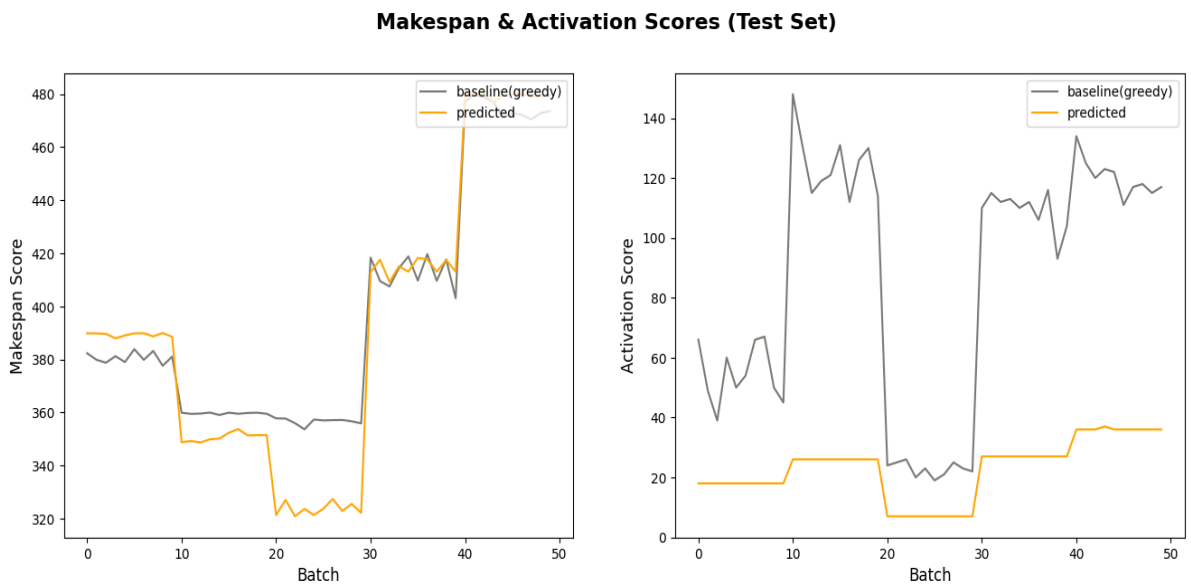


Figure 10.7: Makespan and Activation Scores: Recurrent Model vs. Greedy Baseline

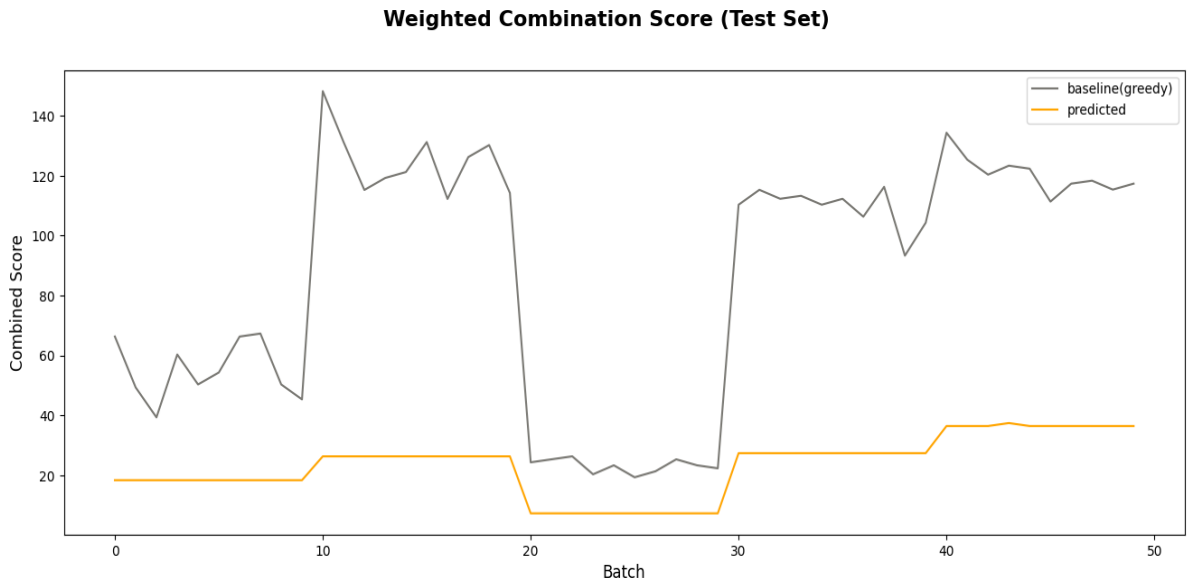
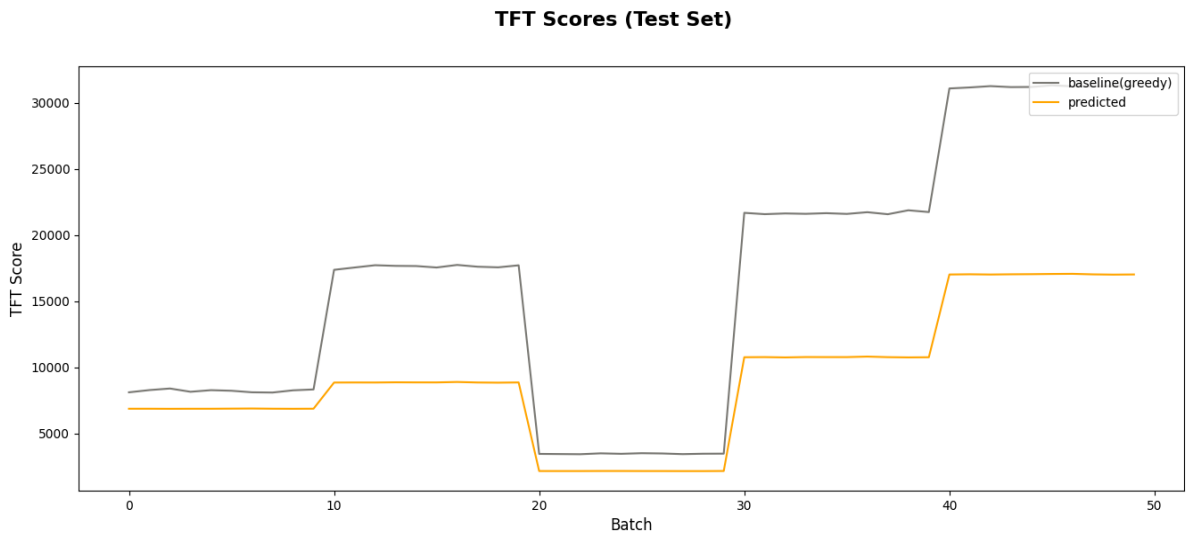
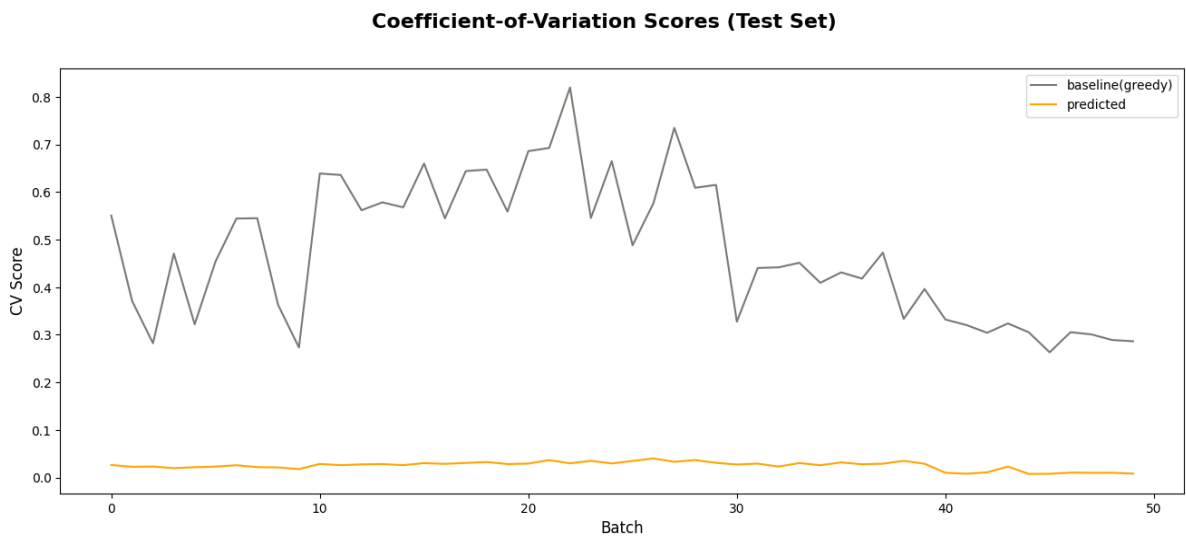


Figure 10.8: Weighted Combination Score: Recurrent Model vs. Greedy Baseline

The weighted-combination trajectory in fig. 10.8 makes the same pattern even clearer. The greedy baseline exhibits large swings across batches, reflecting its sensitivity to local decisions and workload peaks, while the recurrent model remains much smoother and consistently lower. This means that the neural policy is not simply matching the greedy heuristic on isolated batches; it is producing a more balanced schedule across the full test distribution.



(a) TFT Scores



(b) CV Scores

Figure 10.9: TFT and CV Curves: Recurrent Model vs. Greedy Baseline

Figure 10.9a shows the strongest difference in flow efficiency. The greedy baseline produces large TFT plateaus and abrupt jumps, which are typical of a *limited* routing policy that accumulates waiting and travel inefficiencies. By contrast, the recurrent model maintains a much lower and more stable TFT profile throughout the test set, indicating better anticipation of downstream routing conflicts and more efficient operator sequencing.

The same conclusion holds for workload balance in fig. 10.9b. The greedy baseline exhibits substantial fluctuations in the coefficient of variation, which reflects unstable and uneven operator allocation across batches. The recurrent model stays close to zero throughout most of the test set, showing that it preserves a far more uniform workload distribution while still maintaining routing feasibility. Taken together, these score curves justify the subsequent gap analysis: the recurrent model is not only feasible, but also consistently more efficient and better balanced than the greedy heuristic.

Primary Objective Baseline Gaps

Figure 10.10 illustrates the performance of the recurrent model compared to the greedy baseline across the primary objective KPIs.

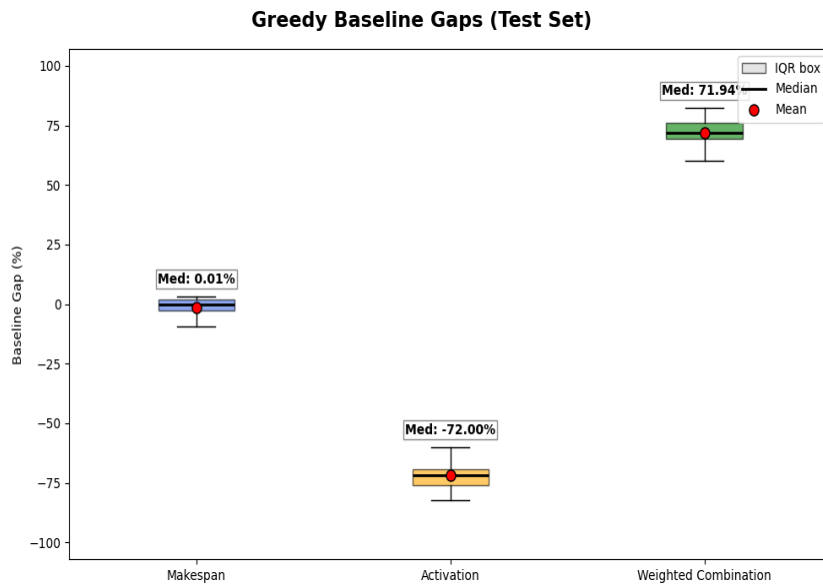


Figure 10.10: Recurrent Model vs. Greedy Baseline Gaps (Primary Objective KPIs)

The recurrent model matches the greedy baseline on the completion time metric, maintaining a median makespan gap of exactly 0.01%. However, this identical speed is

achieved with drastically higher resource efficiency. The activation gap shows a massive median improvement of -72.00% over the baseline. In practical terms, this means the greedy algorithm routinely over-allocates operators—hiring substantially more workers than necessary just to blindly push tasks through. Because the recurrent model effectively predicts the global structural requirements of the schedule, it completes the work in the exact same amount of time using less than a third of the workforce. This dominant efficiency is reflected in the overall weighted combination gap, where the neural model outperforms the greedy baseline by a median of -71.94% .

Efficiency and Fairness Baseline Gaps

The superiority of the recurrent model’s routing and assignment logic is further magnified when evaluating fairness and flow efficiency, as shown in fig. 10.11.

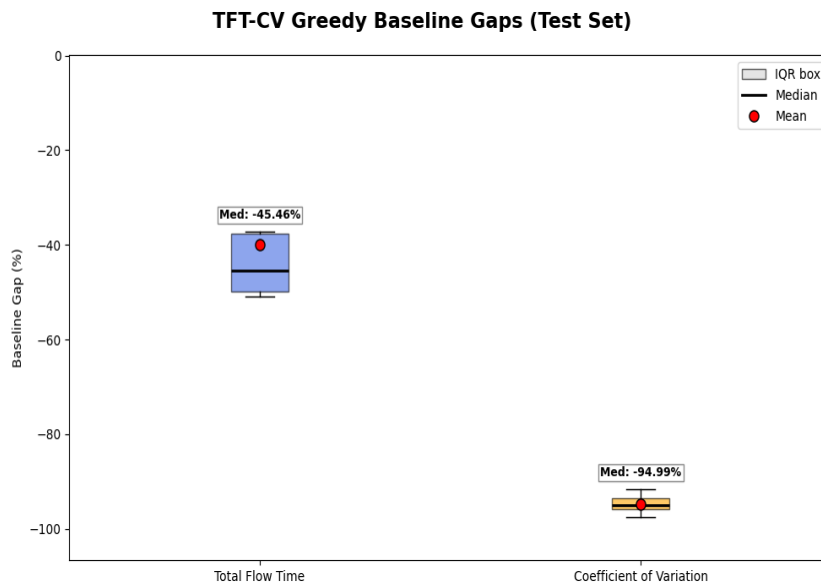


Figure 10.11: Recurrent Model vs. Greedy Baseline Gaps (Efficiency and Fairness)

The TFT gap registers a median of -45.46% , indicating that orders processed by the neural schedule spend nearly half as much cumulative time waiting or traveling compared to the heuristic approach. The greedy algorithm’s limited routing creates localized traffic and suboptimal sequencing that compounds over the massive warehouse space, whereas the recurrent model leverages its global graph to context-aware streamline routes.

Most dramatically, the neural model achieves a median CV gap of -94.99% relative to the greedy baseline. The baseline’s purely reactive assignment logic leads to severe

workload imbalances—some operators are continuously dispatched while others remain idle. The recurrent model, optimized also for fairness through the capacity penalty within the used loss function (see eq.4.17 in ch.4), successfully balances the workload across its highly minimized workforce, proving that massive reductions in operators count do not have to come at the expense of operator equity.

The evaluations presented in this thesis demonstrate that graph neural networks can effectively learn to solve complex, large-scale warehouse scheduling problems. By moving away from exact MIP modelling, which suffers from severe scalability limits, and instead using a supervised learning framework trained on optimal mini-batches, we have developed a neural scheduler that generalizes robustly to **industrial-scale** environments. In this chapter, we synthesize the key architectural findings, highlight the practical implications of the results for real-world deployment.

11.1 The Superiority of Recurrent Decoding at Scale

A primary contribution of this work is the comparison between one-shot (end-to-end) prediction and sequential recurrent decoding. The results conclusively demonstrate that the recurrent model is superior in both feasibility and optimization quality, particularly when generalizing to large-scale batches.

The core limitation of the one-shot model lies in its reliance on static capacity penalties and a unified loss function to **imitate** MIP optimality. When the environment scales up, the natural changes in state variables—such as operator availability and spatial congestion under strict time constraints—become highly dynamic. A one-shot architecture must predict the entire schedule matrix simultaneously, which makes it extremely hard to respect these shifting constraints, leading to massive constraint violations and infeasible operator allocations.

Conversely, the success of the recurrent model is a direct result of the **teacher-forcing** methodology applied during training, combined with the dynamic *feature*

update mechanism during inference. By sequentially masking assigned nodes and updating the graph features at each step, the recurrent decoder explicitly grounds its subsequent predictions in the current, partially completed state of the warehouse. This step-by-step resolution mimics the sequential nature of time constraints, ensuring that the model never assigns an operator to a task if that operator is currently busy or if the assignment would violate precedence rules. Furthermore, because the recurrent model predicts assignments as *non-overlapping* events based on updated graph states, it avoids the combinatorial explosion of searching a massive decision space, significantly speeding up the total inference time compared to single-loop assignment algorithms.

Inference Speed and Real-World Reliability

For a neural scheduler to be reliable in a real-world warehouse, it must not only produce high-quality schedules but also do so within a strict operational time limit. Our recurrent neural solver has registered a proven average inference time of 158.46 seconds for the largest available full-batch set, which consists of approximately 1000 orders distributed among up to 100 operators under a 480-minute time horizon. Generating a near-optimal schedule for a problem of this magnitude in almost three minutes proves that the GNN recurrent approach is fully capable of driving real-time warehouse execution systems where traditional MIP solvers would simply time out.

11.2 The Weighted Combination Score as the Ultimate Business Metric

Throughout this thesis, we have evaluated schedules using a variety of metrics, including makespan, activation, weighted combination, total flow time, and coefficient of variation. However, from a practical business perspective, the most critical reference is the weighted combination score.

The weighted combination score directly reflects the real assigned costs (α and β) of operational completion time (makespan) and labor cost (operator activation). In a real-world scenario, warehouse managers must constantly balance the pressure to fulfill orders quickly against the financial cost of deploying a large workforce. As depicted in fig. 11.1, the recurrent model consistently produces schedules that operate exceptionally close to the theoretical lower bound of this combined cost. In contrast, the greedy base-

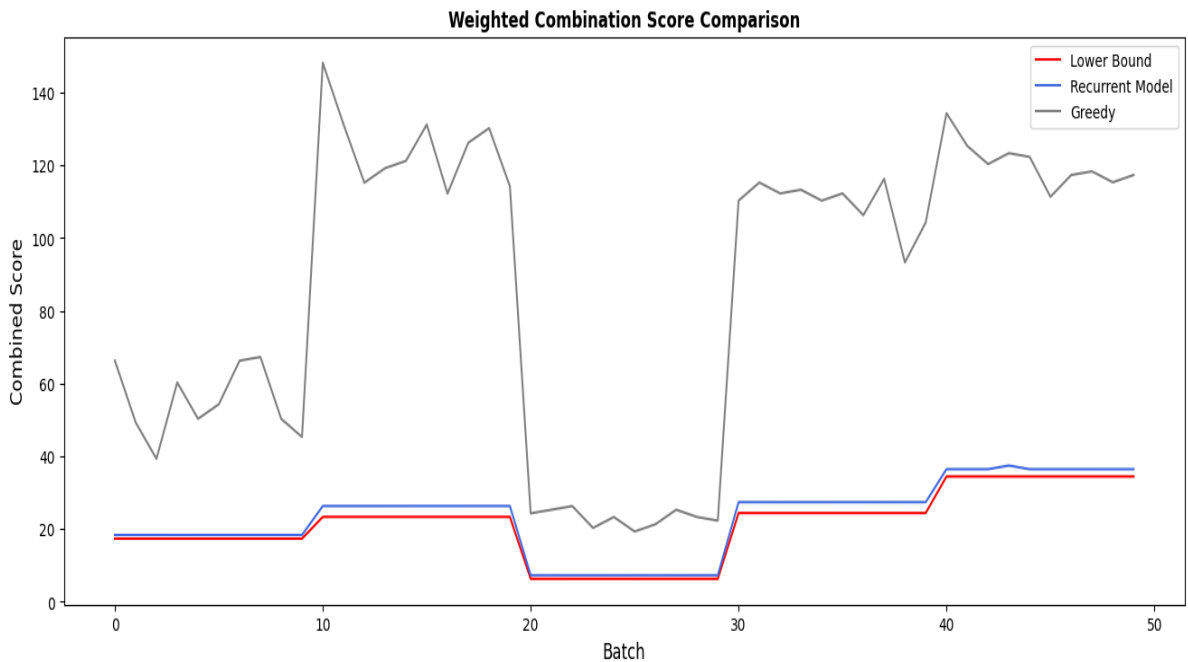


Figure 11.1: Weighted Combination Score Comparison: Lower Bound, Recurrent Model, and Heuristic Greedy Algorithm

line exhibits severe volatility, drastically inflating the business cost by over-activating operators to achieve similar completion speeds, making it an impractical choice. The recurrent model’s ability to minimize this combined metric proves its value as a cost-effective operational tool.

12. Computational Environment and Used Technologies

12.1 Computational Environment

The experiments were implemented in python using `Pandas` for data manipulation, `NumPy` for numerical operations, `PyTorch` together with `PyTorch Geometric` for graph-based learning, `Matplotlib` for data visualization. The exact optimization problems were solved with a MIP-based framework (`Pyomo` for modelling, `IBM ILog CPLEX` for solving), and the resulting schedules were used as supervision for the neural model. The software stack also included libraries for `Scikit-learn` for essential preprocessing and clustering, whereas `Matplotlib` for data visualization.

All experiments were executed in a GPU-capable environment to accelerate both training and inference. This was necessary because the workflow combined exact optimization, graph neural network training, and repeated evaluation across many generated mini-batches. The computational environment was therefore selected to ensure both reproducibility and practical runtime efficiency.

The implementation relied on a modular pipeline that separated data preparation, graph construction, solver-based label generation, and neural training. This structure made it possible to reuse the same preprocessing logic across the different experimental settings, including the mini-batch validation analysis and the full-scale test evaluation. In addition, the environment supported repeated runs with fixed seeds and consistent data transformations, helping to reduce variance across experiments and improve the reliability of the reported results.

For the exact optimization stage, the solver was used offline to generate reference schedules rather than during deployment. This allowed the neural model to learn from high-quality supervision while keeping the training procedure computationally manageable. Once trained, the recurrent model could be evaluated independently of the exact solver, which is essential for studying scalability on large instances where exact optimization becomes impractical.

Overall, the computational environment was chosen to balance three requirements: numerical stability, reproducibility, and scalability. The combination of GPU acceleration, graph-based deep learning tools, and exact offline supervision provided a practical foundation for comparing neural and heuristic scheduling strategies under realistic warehouse constraints.

The full implementation of our thesis is available on this `github` repository: [\[1\]](#).

13. Conclusion and Future Work

13.1 Open Challenges and Future Directions

While the GNN recurrent scheduler presented here is highly efficient, real-world warehouse environments are subject to unpredictable disruptions and shifting business priorities that require further investigation.

Architectural Refinements and Advanced Learning Paradigms

Looking beyond operational adaptations, there are several promising structural and algorithmic modifications that could further enhance the neural scheduler’s performance and robustness.

First, the current recurrent architecture utilizes multiple prediction heads—specifically, separating the assignment and sequencing logic. Empirical observations during training indicate that maintaining a distinct sequencing head introduces conflicting gradients and noise into the optimization landscape, which negatively impacts the f1-score of the primary assignment task. Furthermore, training this complex recurrent structure is affected by the slow convergence and optimization difficulties inherent to backpropagation through time (BPTT). A highly effective structural change would be to transition to a fully auto-regressive model with a unified assignment head that takes into account also the sequencing logic. By removing the auxiliary sequencing head and predicting the next optimal allocation strictly step-by-step, the architecture would simplify the gradient flow, alleviate the BPTT bottleneck, and likely achieve higher precision in its core assignment predictions.

Second, the current training paradigm relies entirely on **offline** imitation learning

(specifically, behavioral cloning) using optimal ground-truth data generated by the exact MIP solver. Consequently, the neural network has only ever been exposed to optimal, “perfect” state trajectories. This introduces a well-known vulnerability called **exposure bias** [55]: if the neural solver makes a slightly suboptimal decision during real-world inference, it shifts the warehouse into an out-of-distribution state—a “bad scenario” it has never seen during training. Lacking any experience in these suboptimal states, the model does not know how to recover, potentially leading to compounding errors.

To overcome this, future iterations should consider more advanced types of imitation learning, such as **dataset aggregation** (DAgger). DAgger iteratively trains the policy by allowing the neural model to navigate the environment, but queries the exact MIP solver to provide the optimal corrective action for the states the model actually visits, directly teaching it how to recover from mistakes.

However, DAgger requires continuous, expensive queries to the exact MIP solver during training. To bypass this computational bottleneck, **adversarial imitation learning** (AIL) could be employed. By training a discriminator to distinguish between the neural scheduler’s generated trajectories and the offline optimal dataset, AIL allows the model to refine its policy using reinforcement learning without needing constant expert interventions. Alternatively, the system could be evolved using pure **deep reinforcement learning** (DRL). By directly optimizing a customized reward function based on the weighted combination score, a pure DRL framework removes the dependency on the MIP expert entirely. Both adversarial and pure DRL frameworks allow the model to explore the scheduling environment through trial and error. By doing so, the model naturally learns to navigate suboptimal scenarios, ultimately discovering robust, generalized routing and assignment policies independent of exact solver limitations.

This shift towards environment-driven exploration also addresses a final, critical concern for real-world deployment: spatial and topological generalization. Under the current offline training framework, this question can be raised: would there be any sudden *catastrophic errors* when deploying the model on a significantly larger and massive scale, or more importantly, on an entirely different warehouse map? Because the current model relies on graph features—specifically travel times and spatial coordinates—derived from a fixed warehouse layout (fig.13.1), changing the floor plan (e.g., adding new aisles, moving depots, or altering rack structures) fundamentally changes the distance metrics. If the model encounters coordinates from a new layout, it

will process them using the geometric assumptions of the old map, producing completely out-of-distribution structural embeddings. This would cause the learned policy to break down unexpectedly, resulting in inefficient routing and operator collisions.

By integrating DRL or AIL with procedural domain randomization—where the agent trains continuously across dynamically generated warehouse maps of varying scales and geometries—the neural scheduler would be forced to learn truly invariant spatial and routing logic, ensuring seamless adaptation to any future physical warehouse changes.

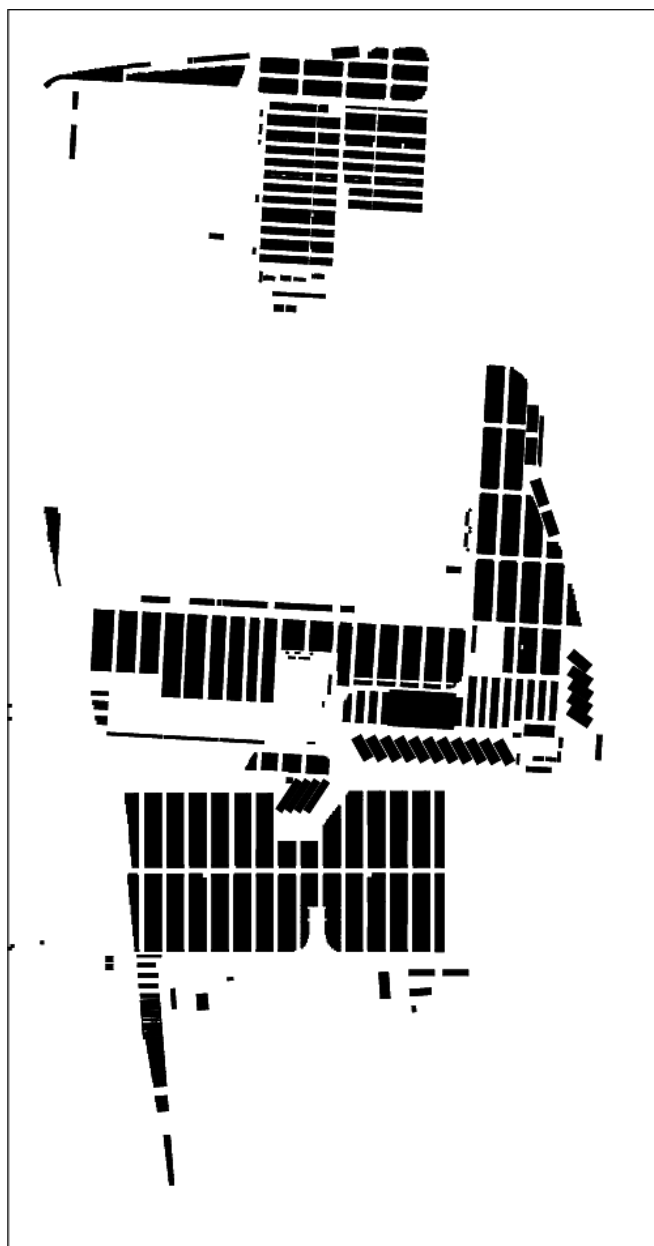


Figure 13.1: Warehouse Map

Adapting to Changing Business Perspectives

The current model was trained to optimize a specific balance between makespan and activation, governed by fixed α and β weights, prioritizing active operators minimization within the recursive decoding. A critical question is what happens if the **business perspective** suddenly changes. For example, during a peak holiday season, a warehouse might prioritize extreme speed (high α) regardless of labor costs, whereas during off-peak times, the focus might shift entirely to minimizing workforce activation (high β). Because the neural model learns its policy offline based on the specific cost function used by the teacher MIP, it cannot dynamically shift its core strategy on the fly in order to adapt new needs. Future work could explore conditioning the Graph2Seq architecture directly on continuous α and β parameters during training, allowing the end-user to adjust the scheduler's behavior at inference time without retraining.

Fault Tolerance and Dynamic Rescheduling

Another practical challenge is **fault tolerance** in a decentralized, distributed system. If an active operator breaks down or a piece of equipment fails mid-execution, the schedule must be adjusted. This raises questions regarding how to execute a "truncated batch" on the neural solver. If a disruption occurs, should the solver receive only the remaining, unassigned workload, or must it re-evaluate the entire current state, including the subsequent tasks already assigned to non-faulty operators, in addition to the faulty tasks? Furthermore, should the system enforce a *consensus* protocol [56] among the remaining active operators to re-distribute the dropped tasks, or should the central solver completely wipe and re-generate the subsequent schedule?

Resolving these issues requires integrating the neural solver into a middleware system capable of real-time state monitoring and rapid re-inference. While our average inference time is fast enough to support occasional re-scheduling, the complex analysis of state-recovery protocols, distributed consensus, and the exact middleware architecture required to notify operators of schedule changes lies beyond the scope of this work. Addressing these fault-tolerance mechanisms represents a rich and necessary approach for future research in neural combinatorial optimization for robotics.

Bibliography

- [1] Abdelrahman Sobh. *Manual Warehouse Optimization*. <https://github.com/Abudo-S/ManualWarehouseOptimization>. 2026.
- [2] D. S. Garey M. R. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Bell Telephone Laboratories Incorporated, 1979.
- [3] Alexander Mendiburu Andoni I. Garmendia Josu Ceberio. “Neural Combinatorial Optimization: a New Player in the Field [about Generalization to different instance sizes].” In: *Arxiv - Cornell University* (2025). URL: <https://arxiv.org/pdf/2205.01356>.
- [4] Jens Clausen. “Branch and Bound Algorithms - Principles and Examples.” In: *University of Copenhagen* (1999). URL: https://janders.eecg.toronto.edu/1387/readings/b_and_b.pdf.
- [5] Davide Angioni, Claudia Archetti, and M. Grazia Spera. “Neural combinatorial optimization: A tutorial.” In: *Science Direct* (2025). URL: <https://www.sciencedirect.com/science/article/pii/S0305054825001303>.
- [6] Michael L. Pinedo. *Scheduling Theory, Algorithms, and Systems*. 3rd ed. Springer, 2008.
- [7] Yejian Zhao, Xiaochuan Luo, and Yulin Zhang. “The application of heterogeneous graph neural network and deep reinforcement learning in hybrid flow shop scheduling problem.” In: *Science Direct* (2024). URL: <https://www.sciencedirect.com/science/article/abs/pii/S0360835223008264>.
- [8] Paolo Toth and Daniele Vigo. “Models, relaxations and exact approaches for the capacitated vehicle routing problem.” In: *Elsevier Science B.V.* (2002). URL: https://mate.unipv.it/~gualandi/famo2conti/papers/routing_models.pdf.

- [9] Gabor Pataki. “Teaching Integer Programming Formulations Using the Traveling Salesman Problem.” In: *Society for Industrial and Applied Mathematic (SIAM)* (2003). URL: https://mate.unipv.it/~gualandi/famo2conti/papers/teaching_tsp.pdf.
- [10] L. Wolsey. *Integer Programming*. Wiley Interscience, 2021.
- [11] Juan Pablo Vielma. “Mixed Integer Linear Programming Formulation Techniques.” In: *Society for Industrial and Applied Mathematic (SIAM)* (2015). URL: <https://juan-pablo-vielma.github.io/publications/Mixed-Integer-Linear-Programming-Formulation-Techniques.pdf>.
- [12] Leonardo Lai et al. “Solving Mixed Pareto-Lexicographic Multi-Objective Optimization Problems: The Case of Priority Levels.” In: *ARPI - University of Pisa* (2021). URL: https://arpi.unipi.it/bitstream/11568/1109185/1/LAI_J2_PL.pdf.
- [13] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [14] Dylan J. Foster, Adam Block, and Dipendra Misra. “Is Behavior Cloning All You Need? Understanding Horizon in Imitation Learning.” In: *arXiv - Cornell University* (2024). URL: <https://arxiv.org/pdf/2407.15007>.
- [15] Bernard T. Agyeman et al. “Graph-Based Imitation and Reinforcement Learning for Efficient Benders Decomposition.” In: *arXiv - Cornell University* (2025). URL: <https://arxiv.org/pdf/2511.11870v1>.
- [16] Benjamin Sanchez-Lengeling et al. “A Gentle Introduction to Graph Neural Networks.” In: *Distill* (2021). <https://distill.pub/2021/gnn-intro>. DOI: [10.23915/distill.00033](https://doi.org/10.23915/distill.00033).
- [17] Muhammed Fatih Balın and Ümit V. Çatalyürek. “Layer-Neighbor Sampling — Defusing Neighborhood Explosion in GNNs.” In: *OpenReview* (2023). URL: <https://openreview.net/pdf?id=Kd5W4JRsfV>.
- [18] Simi Job et al. “Towards Causal Classification: A Comprehensive Study on Graph Neural Networks.” In: *arXiv - Cornell University* (2024). URL: <https://arxiv.org/pdf/2401.15444v1>.
- [19] Shaked Brody, Eran Yahav, and Uri Alon. “How Attentive are Graph Attention Networks?” In: *Sicence Direct* (2022). URL: <https://arxiv.org/pdf/2105.14491>.

- [20] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors.” In: *Nature* (1986). DOI: [10.1038/323533a0](https://doi.org/10.1038/323533a0).
- [21] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators.” In: *Neural Networks* (). DOI: [10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- [22] William L. Hamilton, Zhitao Ying, and Jure Leskovec. “Inductive Representation Learning on Large Graphs.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/5dd9dbd5d08734c0557ed2273a8476d6-Abstract.html>.
- [23] Yang Hu et al. “Graph-MLP: Node Classification without Message Passing in Graph.” In: *arXiv - Cornell University* (2021). URL: <https://arxiv.org/pdf/2212.09034>.
- [24] Chenxiao Yang et al. “Graph Neural Networks are Inherently Good Generalizers: Insights by Bridging GNNs and MLPs.” In: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/pdf/2106.04051>.
- [25] Mohamed Abdelhamid and Abhyuday Desai. “Balancing the Scales: A Comprehensive Study on Tackling Class Imbalance in Binary Classification.” In: *arXiv - Cornell University* (2024). URL: <https://arxiv.org/pdf/2409.19751v1>.
- [26] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. “Deep Sparse Rectifier Neural Network.” In: *MLR press* (2011). URL: <https://proceedings.mlr.press/v15/glorot11a/glorot11a.pdf>.
- [27] Kun Xu et al. “Graph2Seq: Graph to Sequence Learning with Attention-based Neural Networks.” In: *arXiv - Cornell University* (2018). URL: <https://arxiv.org/abs/1804.00823>.
- [28] Alessandro Betti et al. “A zone-based training approach for last-mile routing using Graph Neural Networks and Pointer Networks.” In: *arXiv - Cornell University* (2024). URL: <https://arxiv.org/abs/2401.07635>.
- [29] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.” In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014. URL: <https://arxiv.org/abs/1406.1078>.

- [30] Junyoung Chung et al. “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.” In: *arXiv - Cornell University* (2014). URL: <https://arxiv.org/abs/1412.3555>.
- [31] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. “An Empirical Exploration of Recurrent Network Architectures.” In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015. URL: <https://proceedings.mlr.press/v37/jozefowicz15.html>.
- [32] Paul J. Werbos1990. “Backpropagation through time: what it does and how to do it.” In: *IEEE* (1990). URL: https://www.researchgate.net/profile/Paul-Werbos/publication/2984354_Backpropagation_through_time_what_it_does_and_how_to_do_it/links/55ef061c08aef559dc44b02d/Backpropagation-through-time-what-it-does-and-how-to-do-it.pdf.
- [33] Florian Grötschla, Joël Mathys, and Roger Wattenhofer. “Learning Graph Algorithms With Recurrent Graph Neural Networks.” In: *arXiv preprint arXiv:2212.04934* (2022). URL: <https://arxiv.org/abs/2212.04934>.
- [34] Dingyi Zhuang et al. “GETS: Ensemble Temperature Scaling for Calibration in Graph Neural Networks.” In: *International Conference on Learning Representations (ICLR)*. 2025. URL: <https://openreview.net/forum?id=u8rK7ZId4w>.
- [35] Yu Quan Chong, Jiaoyang Li, and Katia Sycara. “Optimal Task Assignment and Path Planning using Conflict-Based Search with Precedence and Temporal Constraints.” In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024)*. 2024. URL: <https://arxiv.org/abs/2402.08772>.
- [36] Daria Pugacheva et al. “Enhancing GNNs Performance on Combinatorial Optimization by Recurrent Feature Update.” In: *arXiv - Cornell University* (2024). URL: <https://arxiv.org/abs/2407.16468>.
- [37] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. 5th. Springer, 2016.
- [38] B. G. Kingsman et al. “Responding to customer enquiries in make-to-order companies: Problems and solutions.” In: *International Journal of Production Economics* (1996). URL: <https://www.sciencedirect.com/science/article/abs/pii/0925527396000564>.

- [39] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths.” In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: [10.1109/TSSC.1968.4082128](https://doi.org/10.1109/TSSC.1968.4082128).
- [40] Vivek Myers, Catherine Ji, and Benjamin Eysenbach. “Horizon Generalization in Reinforcement Learning.” In: *International Conference on Learning Representations (ICLR)*. 2025. URL: <https://arxiv.org/pdf/2501.02709>.
- [41] Tycho F. A. van der Ouderaa and Mark van der Wilk. “Learning invariant weights in neural networks.” In: *Proceedings of the Thirty-Eighth Conference on Uncertainty in Artificial Intelligence*. Proceedings of Machine Learning Research. PMLR, 2022. URL: <https://proceedings.mlr.press/v180/ouderaa22a.html>.
- [42] Gennaro Gala, Daniele Grattarola, and Erik Quaeghebeur. “E(n)-equivariant Graph Neural Cellular Automata.” In: *arXiv - Cornell University* (2023). URL: <https://arxiv.org/pdf/2301.10497>.
- [43] Sylvestre-Alvise Rebuffi et al. “Data Augmentation Can Improve Robustness.” In: 2021. URL: <https://proceedings.neurips.cc/paper/2021/hash/fb4c48608ce8825b558ccf07169a3421-Abstract.html>.
- [44] Kezhi Kong et al. “Robust Optimization as Data Augmentation for Large-scale Graphs.” In: *arXiv - Cornell University* (2020). URL: <https://arxiv.org/pdf/2010.09891>.
- [45] Sebastian Ruder. “An overview of gradient descent optimization algorithms.” In: *arXiv - Cornell University* (2016). URL: <https://arxiv.org/pdf/1609.04747>.
- [46] Tianhe Yu et al. “Gradient Surgery for Multi-Task Learning.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. URL: <https://arxiv.org/pdf/2001.06782>.
- [47] Ozan Sener and Vladlen Koltun. “Multi-Task Learning as Multi-Objective Optimization.” In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/hash/432aca3a1e345e339f35a30c8f65edce-Abstract.html.
- [48] Zhao Chen et al. “GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks.” In: *International Conference on Machine Learning (ICML)*. 2018. URL: <https://arxiv.org/pdf/1711.02257>.

- [49] *Accuracy Paradox*. https://en.wikipedia.org/wiki/Accuracy_paradox.
- [50] Thomas G Dietterich. “Overfitting and Undercomputing in Machine Learning.” In: *ACM computing surveys (CSUR)* (1995).
- [51] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer Science & Business Media, 2009.
- [52] Jesse Davis and Mark Goadrich. “The relationship between Precision-Recall and ROC curves.” In: *Proceedings of the 23rd International Conference on Machine Learning*. ACM. 2006. URL: <https://ftp.cs.wisc.edu/machine-learning/shavlik-group/davis.icml06.pdf>.
- [53] Matthias Ehrgott. *Multicriteria Optimization*. Springer, 2005.
- [54] Christopher W. F. Parsonson, Alexandre Laterre, and Thomas D. Barrett. “Reinforcement Learning for Branch-and-Bound Optimisation Using Retrospective Trajectories.” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2023. URL: <https://arxiv.org/pdf/2205.14345>.
- [55] Kavosh Asadi et al. “Combating the Compounding-Error Problem with a Multi-step Model.” In: *arXiv - Cornell University* (2019).
- [56] Andrew S. Tanenbaum and Maarten Van Steen. *Distributed Systems: Principles and Paradigms*. 2nd. Upper Saddle River, NJ, USA: Prentice Hall, 2007.

If you want to mention this work, the complete annotation in L^AT_EX is the following:

```
@thesis{NCO-MWLO,  
author="Abdelrahman Sobh",  
title="Optimization of Logistic Operations in Decentralized Manual  
Warehouses",  
school="Department of Computer Science - Faculty of Science and  
Technology, University of Milan",  
year=2026  
}
```

